

MATERIAL AUTORAL

Processamento de Dados e Fine-Tuning de Modelos

Disciplina 9 • Engenharia de Software com IA Aplicada

Autoria: Ahirton Lopes

Como usar este material

Este material foi reorganizado a partir do conteúdo integral das aulas fornecidas. O texto-base permanece fiel à transcrição; a edição acrescenta estrutura didática, navegação interna, diagramas conceituais e atividades de revisão sem substituir as afirmações apresentadas pelo professor.

PRINCÍPIO DE FIDELIDADE

- O conteúdo conceitual das aulas é preservado; a edição atua em organização, hierarquia visual e correções editoriais mínimas.
- Diagramas reorganizam relações explicitamente apresentadas nas aulas e não são apresentados como medições quantitativas.
- Gráficos numéricos não são criados quando a transcrição não fornece valores adequados para comparação.
- Números, casos e exemplos quantitativos permanecem vinculados ao contexto em que aparecem nas aulas.

REPOSITÓRIO OFICIAL DA DISCIPLINA

[Processamento de Dados e Fine-Tuning de Modelos no GitHub](#)

Use o repositório junto com o texto: localize ferramentas, datasets, scripts e materiais complementares citados e compare a implementação com as decisões apresentadas nas aulas.



COMO ESTUDAR CADA AULA

- Leia os objetivos e observe o fluxo visual da unidade antes de avançar.
- Estude o conteúdo integral da aula, agora organizado para facilitar a leitura.
- Use o bloco Na prática para transformar conceitos em uma pequena aplicação ou registro verificável.
- Responda ao checkpoint sem consultar o texto; retorne aos trechos em que houver dúvida.
- Conclua pela revisão da unidade e utilize os links internos para voltar diretamente às aulas que precisarem de reforço.

Introdução da disciplina

Nesta disciplina, eu começo por uma pergunta que precisa vir antes de qualquer dataset, API de treinamento ou escolha de hiperparâmetro: será que fine-tuning é realmente a ferramenta certa para o problema que eu tenho em mãos?

A disciplina anterior deixou uma situação em aberto de propósito. No Trial Forge, a síntese de CSR sempre era encaminhada para um modelo mais caro por uma regra fixa, porque errar naquela etapa custava demais. Só que uma regra fixa não significa que o problema esteja resolvido. A pergunta que abre esta nova disciplina é justamente se poderíamos ter um modelo menor, especializado naquela tarefa, barato e preciso o suficiente para deixar de escalar todas as chamadas.

Essa é a porta de entrada para Processamento de Dados e Fine-Tuning de Modelos. Antes de ensinar a preparar dados ou rodar um treinamento, eu quero organizar a decisão. Fine-tuning é uma das ferramentas mais caras e mais lentas de iterar entre as opções disponíveis. Por isso, deveria ser uma das últimas alternativas consideradas, e não a primeira.

Percurso da disciplina

- Unidade 1: Quando Fazer Fine-Tuning (Decision Framework).
- Unidade 2: Preparação de Datasets para Fine-Tuning.
- Unidade 3: Fine-Tuning via API (upload e train).
- Unidade 4: LoRA e PEFT (Conceitos Práticos).
- Unidade 5: Avaliar Modelos Fine-Tunados.
- Unidade 6: Projeto Final: Modelo Customizado para Domínio Específico.

Mapa da disciplina

Navegue por uma unidade ou aula. Os títulos abaixo são links internos para os respectivos pontos do material.

Unidade 1: Quando Fazer Fine-Tuning (Decision Framework) [↗ ir para a unidade](#)

[Aula 1: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-1\)](#)

[Aula 2: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-2\)](#)

[Aula 3: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-3\)](#)

Unidade 2: Preparação de Datasets para Fine-Tuning [↗ ir para a unidade](#)

[Aula 1: Preparação de Datasets para Fine-Tuning \(PT-1\)](#)

[Aula 2: Preparação de Datasets para Fine-Tuning \(PT-2\)](#)

Unidade 3: Fine-Tuning via API (upload e train) [↗ ir para a unidade](#)

[Aula 1: Fine-Tuning via API \(upload e train\) \(PT-1\)](#)

[Aula 2: Fine-Tuning via API \(upload e train\) \(PT-2\)](#)

[Aula 3: Fine-Tuning via API \(upload e train\) \(PT-3\)](#)

[Aula 4: Fine-Tuning via API \(upload e train\) \(PT-4\)](#)

[Aula 5: Fine-Tuning via API \(upload e train\) \(PT-5\)](#)

Unidade 4: LoRA e PEFT (Conceitos Práticos) [↗ ir para a unidade](#)

[Aula 1: LoRA e PEFT \(Conceitos Práticos\) \(PT-1\)](#)

[Aula 2: LoRA e PEFT \(Conceitos Práticos\) \(PT-2\)](#)

[Aula 3: LoRA e PEFT \(Conceitos Práticos\) \(PT-3\)](#)

[Aula 4: LoRA e PEFT \(Conceitos Práticos\) \(PT-4\)](#)

Unidade 5: Avaliar Modelos Fine-Tunados [↗ ir para a unidade](#)

[Aula 1: Avaliar Modelos Fine-Tunados \(PT-1\)](#)

[Aula 2: Avaliar Modelos Fine-Tunados \(PT-2\)](#)

[Aula 3: Avaliar Modelos Fine-Tunados \(PT-3\)](#)

[Aula 4: Avaliar Modelos Fine-Tunados \(PT-4\)](#)

Unidade 6: Projeto Final: Modelo Customizado para Domínio Específico [↗ ir para a unidade](#)

[Aula 1: Projeto Final: Modelo Customizado para Domínio Específico \(PT-1\)](#)

[Aula 2: Projeto Final: Modelo Customizado para Domínio Específico \(PT-2\)](#)

[Aula 3: Projeto Final: Modelo Customizado para Domínio Específico \(PT-3\)](#)

Unidade 1

Quando Fazer Fine-Tuning (Decision Framework)

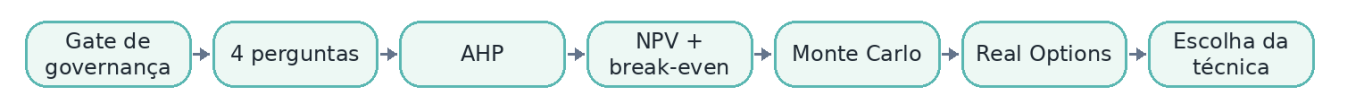
Decisão estruturada sobre quando fine-tuning faz sentido, combinando governança, quatro condições necessárias, análise multicritério e avaliação econômica antes de escolher uma técnica de treinamento.

GITHUB E PRÁTICA

- [Abrir pasta do Módulo 1](#)
- [Decision Framework Tool](#)
- [Pôster Fine-Tuning Zoo](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

FLUXO VISUAL DA UNIDADE



Aula	Tema
1	Quando Fazer Fine-Tuning (Decision Framework) (PT-1)
2	Quando Fazer Fine-Tuning (Decision Framework) (PT-2)
3	Quando Fazer Fine-Tuning (Decision Framework) (PT-3)

Aula 1 — Quando Fazer Fine-Tuning (Decision Framework) (PT-1)

Unidade 1 • Quando Fazer Fine-Tuning (Decision Framework)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Quando Fazer Fine-Tuning (Decision Framework) (PT-1)”.
- Relacionar Fine-tuning e RAG às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Decision Framework Tool \(JavaScript\)](#)
- [Casos da Amplitude Seguros](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Fine-tuning	• RAG
• Decision Framework	• Context engineering
• Gate de governança	• Tarefa estreita e repetida

Conteúdo da aula

O caso da Amplitude Seguros

Para aplicar esse raciocínio, eu utilizo o caso da Amplitude Seguros. A empresa nasceu como cooperativa de produtores rurais, dividindo risco e, décadas depois, se transformou em sociedade anônima e ampliou a atuação para linhas como automóvel, saúde e empresarial.

Essa origem ainda influencia a cultura da companhia. Existe uma tendência forte à decisão colegiada e à aversão a risco. Nesse ambiente, Camila Andrade, head de dados e IA, já não precisa convencer o comitê de que IA generativa importa. Prompting e RAG já aparecem em produção em diferentes frentes.

A questão agora é mais difícil: quando vale a pena ir além de prompt e recuperação de contexto e assumir o custo de treinar um modelo próprio? Em uma organização acostumada a evitar gastos desnecessários, essa decisão precisa ser fundamentada, não movida apenas pela vontade de adotar uma técnica mais sofisticada.

O mito mais importante sobre fine-tuning

Antes do framework, eu preciso derrubar o erro mais comum de quem chega ao fine-tuning pela primeira vez: a ideia de que ele serve para ensinar fatos novos ao modelo.

Esse não é o papel principal do fine-tuning. Fine-tuning ensina comportamento, formato, padrão de resposta, tom e estrutura. Ele ajuda o modelo a responder de uma maneira consistente para uma classe específica de solicitações.

Quem resolve conhecimento novo é o RAG. Nesse caso, o conhecimento fica em uma fonte externa que o modelo consulta a cada chamada. O texto relevante é recuperado e inserido no contexto. O modelo em si não muda.

Confundir esses dois problemas é caro porque pode levar uma equipe a treinar um modelo inteiro quando o que realmente precisava era de uma fonte atualizável de conhecimento.

RAG e fine-tuning resolvem problemas diferentes

A diferença mecânica explica por que essa separação precisa ser tão rígida.

No RAG, o modelo permanece o mesmo. Na hora da requisição, o sistema procura a informação certa em um índice e coloca esse conteúdo no prompt. Se eu alterar o índice, a próxima chamada já pode utilizar o conhecimento novo.

No fine-tuning, eu pego um modelo previamente treinado e continuo o treinamento usando meus próprios exemplos. Durante esse processo, os pesos internos são ajustados por gradiente descendente. Mais adiante na disciplina, nós vamos observar esse treinamento acontecendo de verdade, inclusive acompanhando a perda cair ao longo das iterações.

Isso torna o RAG muito mais reversível. Eu posso trocar ou corrigir a base de conhecimento e ver o efeito imediatamente. No fine-tuning, o comportamento aprendido passa a fazer parte do próprio modelo treinado. Ele não depende de um texto externo ser reinserido em toda chamada para aparecer.

Casos que mostram a diferença na prática

A aula apresenta exemplos de mercado para mostrar os dois lados.

Em um caso ligado ao ecossistema do Ray, textos de Shakespeare foram modificados substituindo sistematicamente o nome Romeo por Bob. Depois do fine-tuning, quando o modelo recebeu uma pergunta sobre o amado de Julieta e ainda recebeu a pista de que o nome começava com R, continuou respondendo Romeo. Mesmo uma substituição tão direta não se transformou de forma confiável em conhecimento factual novo.

Outro estudo citado, publicado por pesquisadores da Microsoft em 2024, comparou fine-tuning e RAG para injeção de conhecimento novo em diferentes modelos. No Llama 2, o fine-tuning isolado chegou a piorar a acurácia em fatos recentes, enquanto o RAG elevou a acurácia de aproximadamente 35% para quase 60%.

O ponto não é concluir que fine-tuning nunca serve. O ponto é perceber que ele estava sendo aplicado ao problema errado.

No outro lado está a Indeed. A plataforma já havia validado o comportamento de explicações de recomendação de vagas com técnicas como few-shot prompting. Depois, usou fine-tuning em um modelo menor para manter a qualidade com menor custo de inferência. O caso citado registra uma redução de cerca de 60% no consumo de tokens em uma escala de milhões de mensagens por mês.

A distinção que eu quero manter é simples: RAG resolve conhecimento. Fine-tuning resolve comportamento repetido em volume e pode tornar esse comportamento mais barato do que reconstruir um prompt grande a cada chamada.

A primeira pergunta: a tarefa é estreita e repetida?

A primeira pergunta do framework é se a tarefa possui uma forma estreita e repetida ou se ela é aberta e variável.

Fine-tuning funciona melhor quando eu consigo descrever a tarefa em uma frase que continua válida para praticamente todos os casos. Um exemplo seria extrair segurado, placa e valor de um documento de sinistro. A entrada muda, mas a natureza da tarefa e o contrato de saída continuam reconhecíveis.

Esse é um sinal verde porque existe repetição suficiente para que um comportamento especializado faça sentido.

O sinal vermelho aparece quando a própria natureza da tarefa muda. Em uma chamada eu quero resumir, em outra traduzir, em outra responder uma pergunta aberta e em outra produzir um formato completamente diferente. Nesse cenário, eu não tenho um comportamento estreito para especializar. Um bom prompt tende a ser uma ferramenta melhor.

A segunda pergunta: as alternativas mais baratas já foram esgotadas?

A segunda pergunta é a que muita gente pula: eu já tentei de verdade prompt engineering, RAG, roteamento de modelo e cache de contexto ou de prompt antes de decidir treinar?

A Indeed é importante justamente porque seguiu essa ordem. Primeiro validou que o comportamento funcionava com few-shot prompting. Só depois partiu para fine-tuning com o objetivo de reduzir custos em escala.

Existe um teste prático. Se eu consigo explicar em texto exatamente como quero que o modelo se comporte, primeiro preciso tentar colocar essa descrição em um system prompt bem escrito, utilizando exemplos reais dentro do próprio contexto.

O sinal verde aparece quando esse trabalho já foi feito de maneira séria e ainda assim existe uma falha consistente de qualidade, custo ou latência. O sinal vermelho é considerar fine-tuning antes mesmo de tentar melhorar o prompt adequadamente.

A regra é importante porque treinar não deveria ser uma forma de evitar o trabalho de especificar o comportamento que eu quero.

A terceira pergunta: existem dados suficientes e de qualidade?

A terceira pergunta é se eu tenho exemplos suficientes, diversos e corretos para treinar.

Essa questão será aprofundada no módulo 2, mas precisa entrar no decision framework desde o começo. Sem dados bons, não existe fine-tuning bom.

Suficiente não significa necessariamente milhares de exemplos. Significa possuir exemplos reais e representativos da variação que existe na tarefa-alvo.

Um sinal verde seria já ter um histórico real de entradas e saídas corretas para aquela operação, mesmo que inicialmente sejam apenas algumas dezenas de casos, desde que cubram de forma útil a diversidade do problema.

O sinal vermelho aparece quando eu preciso inventar todo o dataset do zero sem possuir nenhum caso real como referência. Dados artificiais podem aparecer em estratégias mais avançadas de expansão, mas não deveriam ser a única fundação de um dataset quando eu sequer conheço a variação real do processo que quero modelar.

A quarta pergunta: a tarefa é estável?

A quarta pergunta avalia se a tarefa é estável o suficiente para justificar o custo de especialização.

Se o formato de saída muda toda semana, cada mudança importante pode exigir revisão do dataset, nova avaliação e, em alguns casos, novo treinamento. Esse custo de manutenção precisa ser considerado antes do primeiro ciclo de fine-tuning.

O sinal verde é ter um esquema de saída tratado como contrato. Ele muda raramente e, quando muda, isso acontece por uma decisão deliberada de produto ou arquitetura.

O sinal vermelho aparece quando cada área da empresa quer uma resposta diferente e ainda não existe um contrato consolidado. Treinar antes dessa definição é tentar acertar um alvo que ainda está se movendo.

Nos módulos seguintes, eu vou transformar esse custo em algo mais concreto, inclusive mostrando o custo real de um ciclo de treinamento.

A escada de customização ficou maior

Existe um adendo importante porque o mercado evoluiu. Hoje os modelos de fronteira aceitam janelas de contexto muito maiores do que alguns anos atrás. Modelos de raciocínio também conseguem gastar mais tempo de inferência antes de responder, e o custo por token caiu em muitos cenários.

Isso não muda a diferença entre RAG e fine-tuning. O primeiro continua ligado ao conhecimento recuperado. O segundo continua ligado ao comportamento aprendido. O que mudou foi a quantidade de alternativas intermediárias que eu posso tentar antes de treinar.

Uma delas é context engineering. Aqui eu penso em tudo que entra no contexto do modelo, não apenas no prompt escrito manualmente. Histórico de conversa, documentos recuperados, saída de ferramentas e exemplos escolhidos dinamicamente fazem parte dessa disciplina maior. RAG é uma técnica dentro desse conjunto.

Outra alternativa são agent skills, pacotes de instruções e, em alguns casos, código que um agente pode carregar sob demanda quando precisa de uma capacidade específica. Eles não precisam permanecer no contexto o tempo inteiro e também não exigem alteração dos pesos do modelo.

A escada passa, então, por prompt, context engineering, RAG, agent skills e outras técnicas antes de chegar ao fine-tuning.



Figura — Escada de customização apresentada no Decision Framework.

Onde fine-tuning continua ganhando

Mesmo com modelos excelentes e contexto grande, existem situações em que fine-tuning continua tendo vantagens claras.

A primeira é volume. Se eu preciso reconstruir um contexto grande em milhões de chamadas, o custo de tokens e latência pode superar o investimento de ter parte daquele comportamento embutido em um modelo especializado.

A segunda é consistência de formato. Em tarefas em que o esquema de saída precisa ser extremamente estável, fine-tuning pode reduzir a dependência de repetir instruções de formato em todo prompt.

Ainda assim, isso não elimina a validação determinística. A própria OpenAI oferece mecanismos como Structured Outputs para garantir aderência a um esquema JSON. Fine-tuning pode aumentar a consistência, mas não deveria ser a única camada de proteção.

A terceira situação aparece quando eu preciso rodar um modelo pequeno, local, fora da nuvem e sem acesso a um contexto enorme ou a skills externas. Essa é uma das situações que será demonstrada mais adiante com um modelo pequeno executando localmente.

O checklist das quatro perguntas

O decision framework é transformado em um checklist com as quatro perguntas, os sinais verdes, os sinais vermelhos e uma primeira aplicação ao caso da Amplitude Seguros.

As perguntas são: a tarefa é estreita e repetida; as alternativas mais baratas já foram esgotadas; existem dados suficientes, diversos e de qualidade; e a tarefa é estável o bastante para não criar uma esteira de retreino constante.

Se uma resposta importante cai no vermelho, eu interrompo a recomendação de fine-tuning naquele ponto. Não faz sentido forçar as outras perguntas apenas para justificar uma decisão que já começou errada.

O checklist não substitui julgamento. Ele organiza o julgamento. Nos próximos vídeos, ele deixa de ser somente uma tabela e passa a ser aplicado a casos distintos da Amplitude Seguros, até chegar a uma implementação executável que recomenda caminhos diferentes para problemas diferentes.

O portão de governança

Existe ainda uma quinta preocupação que as quatro perguntas não cobrem completamente: governança.

No caso da Amplitude, um dos cenários envolve recibos médicos e dados de saúde sensíveis. Treinar um modelo com esse tipo de informação não é a mesma decisão que consultar um dado externo por RAG.

Quando a informação entra no processo de treinamento, retenção, auditoria e direito ao esquecimento fica mais difícil de operacionalizar. Por isso, a governança passa a funcionar como um portão anterior às quatro perguntas nos capítulos seguintes.

Se o dado não pode ser utilizado de forma adequada para treinamento, pouco importa que a tarefa seja estreita, estável e tenha volume. O candidato não deveria avançar para fine-tuning.

Fine-tuning também cria novas responsabilidades

Antes de seguir para os estudos de caso, eu deixo registrado que fine-tuning não encerra problemas. Ele troca parte dos problemas de inferência por problemas de dados e manutenção.

Dados ruins não avisam que estão errados. Eles viram comportamento aprendido. Um dataset pequeno demais ou uma configuração agressiva pode levar a overfitting, em que o modelo memoriza exemplos em vez de aprender um padrão mais geral.

Cada modelo fine-tunado também vira um artefato que precisa ser versionado, reavaliado e eventualmente retreinado quando o modelo base muda.

E fine-tuning sem uma avaliação rigorosa contra um baseline é especialmente perigoso porque pode gerar falsa confiança. Uma melhora aparente em alguns exemplos não prova que a solução funciona melhor no problema real.

As quatro perguntas desta aula servem para filtrar candidatos. Elas não substituem preparação de dados, treinamento, avaliação e governança. Nos próximos capítulos, o mesmo framework será aplicado a casos da Amplitude Seguros que parecem semelhantes na origem, mas não recebem a mesma recomendação quando cada pergunta é analisada com cuidado.

NA PRÁTICA

- Escolha uma tarefa do seu contexto e responda às quatro perguntas do Decision Framework.
- Registre, para cada pergunta, o sinal verde ou vermelho e a evidência que sustenta a classificação.
- Verifique se o gate de governança permite que o caso avance antes de discutir treinamento.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Quando Fazer Fine-Tuning (Decision Framework) (PT-1)”?
- Qual relação existe entre Fine-tuning e RAG no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- Nesta disciplina, eu começo por uma pergunta que precisa vir antes de qualquer dataset, API de treinamento ou escolha de hiperparâmetro: será que fine-tuning é realmente a ferramenta certa para o problema que eu tenho em mãos?
- A disciplina anterior deixou uma situação em aberto de propósito.
- Essa é a porta de entrada para Processamento de Dados e Fine-Tuning de Modelos.

[↑ Voltar ao mapa da disciplina](#)

Aula 2 — Quando Fazer Fine-Tuning (Decision Framework) (PT-2)

Unidade 1 • Quando Fazer Fine-Tuning (Decision Framework)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Quando Fazer Fine-Tuning (Decision Framework) (PT-2)”.
- Relacionar Condições necessárias e Gate de governança às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Casos da Amplitude Seguros](#)
- [Decision Framework Tool](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Condições necessárias	• Gate de governança
• Representatividade de dados	• Amplitude Auto
• Saúde Empresarial	• Atendimento ao Cliente

Conteúdo da aula

No capítulo anterior, eu apresentei o framework de decisão para avaliar quando fine-tuning realmente faz sentido. Agora eu aplico esse framework a casos concretos da Amplitude Seguros. A ideia é sair da regra decorada e observar como duas tarefas aparentemente parecidas podem receber recomendações diferentes quando eu analiso cada condição necessária de forma explícita.

Vou trabalhar com três casos. Os dois primeiros pertencem à mesma empresa e têm uma origem semelhante: documentos externos, formatos inconsistentes e necessidade de extrair informação de forma confiável. Mesmo assim, Amplitude Auto e Amplitude Saúde Empresarial não chegam à mesma recomendação. O terceiro caso, atendimento ao cliente, é importante porque mostra outra nuance: ter muito dado disponível não significa automaticamente que fine-tuning seja adequado.

O objetivo é aplicar as quatro perguntas com justificativa registrada e acrescentar um gate de governança que roda antes de qualquer ponderação. A implementação executável fica para o próximo capítulo. Aqui, o foco é entender exatamente onde a decisão muda.

As quatro perguntas continuam sendo condições necessárias

O framework continua com as mesmas quatro perguntas. Primeiro, a tarefa é estreita e repetida? Segundo, prompt engineering, RAG e as alternativas mais baratas já foram realmente esgotadas? Terceiro, existem dados suficientes, diversos e representativos para treinamento? Quarto, a tarefa é estável o bastante para justificar o investimento e a manutenção de um modelo fine-tunado?

As quatro precisam estar verdes. Isso não representa uma aprovação arbitrária por unanimidade. Cada pergunta verifica uma condição necessária diferente.

Uma tarefa estreita sem dados suficientes não é um bom candidato. Um grande volume de dados aplicado a uma tarefa aberta também não resolve. Uma tarefa repetitiva que muda de contrato toda semana pode transformar o fine-tuning em uma esteira de retreino.

Por isso, as perguntas não compensam umas às outras. Um sinal vermelho já muda a recomendação, independentemente de quão fortes sejam os sinais verdes restantes.

Relembrando o papel do RAG

Como a segunda pergunta compara fine-tuning com alternativas anteriores, vale relembrar rapidamente o papel do RAG.

No RAG, a pergunta é representada de forma que o sistema consiga localizar trechos semanticamente relevantes em uma base de conhecimento. Esses trechos são recuperados e enviados junto com a solicitação para o modelo. Nada é retreinado.

É justamente por isso que, se prompt e RAG já resolvem o problema com qualidade, custo e latência aceitáveis, fine-tuning não deveria entrar em cena. Eu só avanço para treinamento quando existe uma necessidade que essas técnicas não resolveram de forma suficiente.

O gate de governança vem antes da ponderação

Antes das quatro perguntas, eu adiciono um portão de governança.

Esse gate verifica uma questão binária: os dados podem ser processados por um provedor de fine-tuning do ponto de vista legal e de compliance? Preciso ter uma base legal definida e, quando houver dados de categoria sensível, preciso considerar os acordos exigidos com o provedor, como um DPA adequado ao contexto.

Esse gate não funciona como uma quinta pergunta ponderável. Ele é um blocker. Nenhum volume de dados, ganho financeiro ou pontuação favorável em um método de decisão compensa uma base legal ausente.

Por isso ele precisa rodar antes do AHP e antes da análise econômica. Não faz sentido gastar tempo avaliando retorno financeiro de um caso que juridicamente não pode prosseguir.

Na Amplitude Seguros, Saúde Empresarial carrega uma exigência adicional porque envolve dados de saúde, categoria sensível. Auto e atendimento ao cliente não possuem essa mesma característica no exemplo. Os três casos utilizados na aula passam pelo gate, mas o resultado é verificado caso a caso e fica acompanhado de justificativa.

Governança não termina no gate

Passar no gate não significa que toda a discussão de privacidade esteja resolvida.

Existe um passo técnico adicional entre a autorização de uso e a construção do dataset: higienizar informações pessoalmente identificáveis antes que elas cheguem ao treinamento.

CPF, nome e outros identificadores podem ser memorizados pelo modelo e eventualmente aparecer em uma resposta futura se forem mantidos de forma inadequada no dataset. Esse risco será tratado com mais profundidade no módulo de preparação de dados.

A disciplina antecipa inclusive uma ferramenta específica para higienização, com validação real de CPF pelo dígito verificador, além de materiais complementares sobre privacidade diferencial, fine-tuning federado e risco de memorização.

A separação é importante. O gate responde se o piloto pode prosseguir. Mascaramento, auditoria e demais controles determinam como o dado precisa ser preparado e governado durante o restante do pipeline.

Caso 1: Amplitude Auto

O primeiro caso é Amplitude Auto. A tarefa é extrair segurado, placa e valor de orçamentos de oficina.

O problema é realista porque o formulário principal do sinistro pode até chegar estruturado pela plataforma, mas o documento de apoio vem da oficina. Cada oficina pode utilizar um formato diferente, criado por software próprio, impresso ou até preenchido de maneira menos padronizada.

Antes de analisar o caso, eu deixo uma ressalva: os números financeiros utilizados ao longo do exemplo são ilustrativos. Eles existem para construir uma história econômica coerente e deverão ser substituídos pelos números reais de cada organização quando o framework for utilizado na prática.

Amplitude Auto: pergunta 1

A tarefa é estreita e repetida? Sim.

A saída é sempre a mesma: segurado, placa e valor. Os documentos de entrada variam bastante entre oficinas, mas a variação está na forma de entrada, não na natureza da tarefa.

Esse é exatamente o tipo de situação em que fine-tuning pode aprender um comportamento consistente. A primeira pergunta recebe sinal verde.

Amplitude Auto: pergunta 2

Prompt e RAG já foram esgotados? Sim.

A equipe já trabalhou com prompt bem escrito e RAG em produção. Ainda assim, a variação de layout entre oficinas mantém inconsistências de saída em escala.

O ponto não é que prompt tenha sido ignorado. Pelo contrário, ele já foi testado e validado até o limite em que consegue ajudar. Fine-tuning aparece como próximo degrau porque existe um problema residual de consistência que continua relevante.

A segunda pergunta também fica verde.

Amplitude Auto: perguntas 3 e 4

Existe dado suficiente? Sim. A linha de Auto é madura e possui milhares de sinistros processados historicamente, cobrindo a diversidade real dos orçamentos.

A tarefa também é estável. O sistema espera os mesmos campos e o contrato de saída não muda semanalmente.

Portanto, as perguntas 3 e 4 ficam verdes. O resultado é quatro sinais verdes. Para Amplitude Auto, o framework considera que fine-tuning vale a pena como candidato.

Caso 2: Amplitude Saúde Empresarial

O segundo caso parece muito parecido na superfície. Em Saúde Empresarial, a tarefa é extrair beneficiário, procedimento e valor de recibos médicos e ainda cruzar essas informações com o cadastro da empresa cliente para identificar se o pedido pertence ao titular ou a um dependente.

A estrutura é semelhante à de Auto, mas o domínio adiciona uma camada regulatória importante porque os documentos carregam dados de saúde.

O gate de governança precisa, portanto, ser tratado com atenção adicional antes de qualquer análise das quatro perguntas.

Saúde Empresarial: perguntas 1 e 2

A primeira pergunta recebe sinal verde. A tarefa é estreita e repetida. O contrato continua previsível: beneficiário, procedimento, valor e resultado do cruzamento entre titular e dependente.

A segunda pergunta também fica verde. A equipe já testou prompt e RAG, seguindo o mesmo tipo de processo de validação utilizado em Auto.

Até aqui, os dois casos parecem equivalentes.

Saúde Empresarial: a pergunta que muda a decisão

A divergência aparece na terceira pergunta: existe dado suficiente, diverso e representativo? Neste momento, não.

Saúde Empresarial é uma linha mais nova da companhia e possui um histórico menor. Existem dados, mas eles ainda não cobrem de forma satisfatória a variedade real de recibos, prestadores e situações.

O problema não é a ausência completa de exemplos. O problema é a representatividade.

A aula coloca números ilustrativos para tornar a diferença concreta. Auto processa aproximadamente 8 mil sinistros por mês, enquanto Saúde Empresarial processa cerca de 1.200. É uma diferença próxima de sete vezes.

O score utilizado no framework acompanha essa distância. Auto recebe aproximadamente 0,9 nesta dimensão, enquanto Saúde Empresarial fica em 0,35, abaixo de um limiar de 0,6 utilizado para separar aprovação e reprovação nesta pergunta.

O framework não cria a diferença. Ele formaliza uma diferença que já existe nos dados disponíveis.

Saúde Empresarial: tarefa estável, mas ainda não

A quarta pergunta volta a ser verde. O contrato de saída é estável e integrado ao mesmo tipo de sistema de sinistros.

O resultado final, portanto, é três perguntas verdes e uma vermelha.

Nesse caso, a recomendação não é que fine-tuning nunca servirá. A recomendação é ainda não.

O gargalo é dado histórico representativo. Se o volume crescer e a variedade do problema passar a estar adequadamente coberta, a decisão poderá ser reavaliada.

Essa é uma diferença importante em relação ao terceiro caso, no qual o problema não pode ser resolvido simplesmente esperando mais dados.

Caso 3: Atendimento ao Cliente

O terceiro caso é atendimento ao cliente.

A tarefa envolve negociar e resolver contestações de sinistro em conversas abertas com segurados, incluindo exceções de cobertura analisadas caso a caso.

Um segurado pode questionar um item de guincho, outro pode discutir uma exceção de cobertura, outro pode precisar de escalonamento para um especialista. O contexto, o histórico, o produto contratado e o tipo de decisão mudam entre atendimentos.

Esse cenário possui mais dados do que os anteriores, mas ainda assim reprova no framework.

Atendimento ao Cliente: a tarefa não é estreita

A primeira pergunta recebe sinal vermelho.

A tarefa é aberta. Não existe uma única estrutura de saída comparável a segurado, placa e valor.

Cada contestação pode exigir uma combinação diferente de interpretação, consulta de cobertura, justificativa, negociação e decisão.

Nesse caso, mais dados não tornam a tarefa estreita. O problema não é a falta de exemplos. É a ausência de um padrão único que faça sentido embutir como comportamento especializado.

Atendimento ao Cliente: alternativas e dados estão fortes

A segunda pergunta fica verde de propósito. Prompt bem escrito, RAG sobre apólices e regras de cobertura e roteamento para atendimento humano em situações sensíveis já estão em produção.

A terceira pergunta também fica verde. Esse é o caso com maior volume de dados entre os três. Existem milhares de conversas por mês, em quantidade superior à de Auto e Saúde Empresarial combinadas.

Essa combinação é importante porque impede uma interpretação simplista do framework. Dado de sobra não basta para justificar fine-tuning.

Atendimento ao Cliente: a tarefa também é instável

A quarta pergunta recebe outro sinal vermelho.

Políticas de cobertura podem mudar com revisões regulatórias e com a criação de novos produtos. Uma exceção aceitável hoje pode deixar de ser válida depois de uma alteração de regra.

Treinar um comportamento fixo sobre uma política que muda com frequência cria risco de obsolescência rápida.

O resultado é diferente de Saúde Empresarial. Ali, o framework diz que ainda não, porque falta representatividade de dados. Em atendimento ao cliente, a recomendação é não usar fine-tuning para essa tarefa como ela está definida, porque os problemas centrais são abertura e instabilidade.

Três casos, três recomendações

Os três casos produzem recomendações diferentes.

Amplitude Auto passa: tarefa estreita, alternativas anteriores esgotadas, dados representativos e contrato estável.

Saúde Empresarial não passa agora: a tarefa é estreita, já houve trabalho com prompt e RAG e o contrato é estável, mas o histórico ainda não representa suficientemente a variação real.

Atendimento ao Cliente também não passa, porém por outro motivo. Ele possui muitos dados, mas a própria tarefa é aberta e instável.

Essa comparação mostra por que o framework precisa ser aplicado pergunta por pergunta. Não existe uma resposta única para a empresa inteira nem para a categoria genérica de documento ou atendimento.

Por que a dimensão de dados recebe peso maior

A aula também antecipa que, na ponderação AHP utilizada por trás do framework, a dimensão de dados recebe maior peso do que as demais.

A justificativa não é preferência arbitrária. Prompt melhor pode ser escrito amanhã. Um contrato pode ser redesenhado. Uma arquitetura de RAG pode ser revisada. Dado histórico representativo não aparece por decisão imediata da equipe.

Ele depende de tempo, coleta ou de uma fonte alternativa confiável.

Por isso, a insuficiência de dados possui uma característica estrutural diferente das outras limitações e recebe um peso maior na modelagem que será detalhada no próximo capítulo.

Mesmo assim, a ponderação não transforma uma condição necessária em algo compensável. O gate de governança continua anterior à análise e as perguntas vermelhas continuam tendo significado próprio.

O próximo passo do framework

No próximo capítulo, esse raciocínio deixa de ser apenas uma análise manual e passa a ser executado no terminal.

A implementação apresenta os pesos derivados pelo AHP, a razão de consistência e a recomendação para os três casos.

A análise também incorpora dimensões econômicas como NPV e breakeven, além de simulação de Monte Carlo e o valor de esperar utilizando Real Options.

A ideia é transformar o framework em uma ferramenta reproduzível. Em vez de uma equipe dizer apenas que acredita que fine-tuning vale a pena, cada caso passa por gates, perguntas, justificativas e critérios econômicos que podem ser registrados e revisitados.

Essa disciplina começa, portanto, por uma regra de decisão antes de começar o treinamento. A tecnologia só entra quando o problema, os dados, a estabilidade e a governança realmente sustentam o investimento.

NA PRÁTICA

- Compare três tarefas do seu contexto e aplique as quatro condições necessárias separadamente.
- Diferencie um caso de “ainda não” por falta de dados de um caso de “não” por tarefa aberta ou instável.
- Documente a justificativa sem permitir que um score médio compense uma condição essencial reprovada.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Quando Fazer Fine-Tuning (Decision Framework) (PT-2)”?
- Qual relação existe entre Condições necessárias e Gate de governança no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No capítulo anterior, eu apresentei o framework de decisão para avaliar quando fine-tuning realmente faz sentido.
- Vou trabalhar com três casos.
- O objetivo é aplicar as quatro perguntas com justificativa registrada e acrescentar um gate de governança que roda antes de qualquer ponderação.

[↑ Voltar ao mapa da disciplina](#)

Aula 3 — Quando Fazer Fine-Tuning (Decision Framework) (PT-3)

Unidade 1 • Quando Fazer Fine-Tuning (Decision Framework)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Quando Fazer Fine-Tuning (Decision Framework) (PT-3)”.
- Relacionar AHP e Razão de consistência às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Decision Framework Tool](#)
- [Pôster: Fine-Tuning Zoo](#)
- [Companion: mecanismo e estado da arte](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• AHP	• Razão de consistência
• NPV	• Break-even
• Monte Carlo	• Real Options
• LoRA e QLoRA	• DPO

Conteúdo da aula

Nos capítulos anteriores, eu construí e apliquei o framework de quatro perguntas para decidir quando o fine-tuning realmente faz sentido. Agora eu tiro essa decisão do slide e coloco o ferramental para rodar no terminal.

O código executa os três casos da Amplitude Seguros, aplica o gate de governança, deriva pesos por AHP, verifica a consistência do julgamento, calcula NPV e break-even, executa dez mil simulações de Monte Carlo e, quando a reprovação é temporária por insuficiência de dados, calcula o valor de esperar usando Real Options.

A mudança é importante porque o fine-tuning não deveria ser aprovado apenas por um sinal verde informal. Quando o investimento envolve dados, treinamento, operação e manutenção, eu quero números, probabilidades e justificativas que possam ser apresentadas a um comitê e revisitadas depois.

Executando o Decision Framework no terminal

A demonstração começa com o Decision Framework Tool executado no terminal. Antes da análise, a suíte automatizada roda vinte e nove testes para validar o comportamento esperado.

Depois, o script imprime os pesos derivados, a razão de consistência, os resultados financeiros, a simulação de Monte Carlo, o valor de espera para o caso elegível a Real Options e uma sessão de comitê com pesos agregados de avaliadores hipotéticos.

O gate de governança permanece anterior a tudo isso. Ele não é uma variável compensável. Os casos só seguem para ponderação e análise financeira quando podem prosseguir do ponto de vista legal e de compliance.

AHP como formalização do julgamento

A primeira técnica é o AHP, Analytic Hierarchy Process.

Os pesos das quatro perguntas não são escolhidos diretamente como percentuais arbitrários. Eles são derivados de uma matriz de comparação pareada, em que cada critério é comparado aos demais por uma escala de importância relativa.

Na demonstração, a pergunta sobre dados suficientes recebe importância maior porque representa uma restrição que não se resolve apenas por decisão imediata da equipe. Um prompt pode ser reescrito. Um contrato pode ser redesenhado. Histórico representativo depende de tempo, coleta ou outra fonte de dados.

Da matriz saem pesos aproximados de 0,141 para as perguntas 1 e 2, 0,455 para dados suficientes e 0,263 para estabilidade.

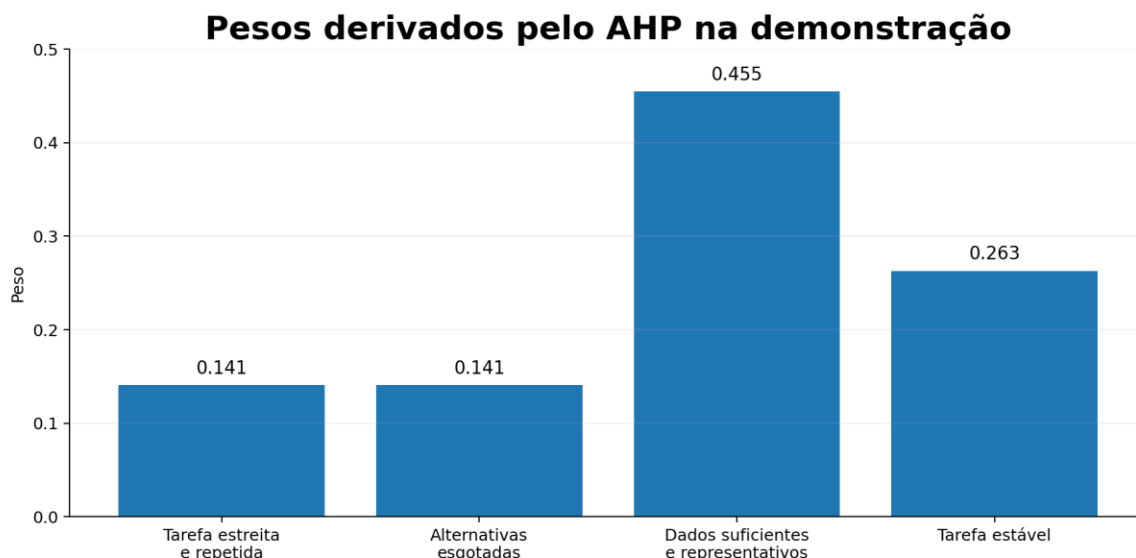


Gráfico — Pesos derivados pelo AHP na demonstração da disciplina.

Razão de consistência e decisão em grupo

O AHP também calcula uma razão de consistência. Isso permite detectar comparações contraditórias antes de confiar nos pesos.

Na matriz da demonstração, a razão fica em aproximadamente 0,038, abaixo do limiar de 10% utilizado como referência. O julgamento é, portanto, suficientemente coerente.

O código também simula um comitê com três perfis: produto, compliance e engenharia. Cada perfil possui sua própria matriz, e as matrizes são agregadas pela média geométrica célula a célula.

Os pesos mudam um pouco, mas os três veredictos continuam iguais: Auto aprovado, Saúde Empresarial ainda não e Atendimento ao Cliente reprovado. Isso mostra que a decisão não depende totalmente de quem está na sala.

NPV, break-even e Monte Carlo

Com os pesos definidos, o framework passa para a análise econômica.

O NPV representa o valor presente dos benefícios futuros menos o investimento necessário. O break-even mostra em que mês o retorno acumulado cobre o investimento.

Na demonstração, a taxa de desconto é de 1% ao mês, coerente com o perfil mais avesso a risco da Amplitude Seguros. Para Auto, o NPV em 24 meses fica positivo e o break-even aparece por volta do décimo mês.

Mesmo assim, eu não quero depender de um único cenário determinístico. Volume de chamadas, economia por requisição e custo operacional variam na vida real.

Por isso, o framework executa dez mil simulações de Monte Carlo. Para Amplitude Auto, a demonstração apresenta probabilidade de retorno positivo de 100% no horizonte analisado, inclusive nos cenários simulados menos favoráveis.

A utilidade da simulação é transformar premissas incertas em uma distribuição de resultados possíveis, em vez de tratar uma única planilha como verdade absoluta.

A demonstração também deixa claro que a análise financeira utiliza números ilustrativos, calibrados para o caso da Amplitude Seguros. O valor real de um projeto precisa ser recalculado com volume, custo de inferência, custo de treinamento, taxa de erro e impacto econômico do próprio contexto da empresa. A ferramenta organiza o cálculo, mas não cria premissas de negócio que não existem.

Saúde Empresarial e o valor de esperar

Saúde Empresarial é o caso mais interessante porque não recebe um simples não.

O framework reprova o treinamento agora porque o score de dados ainda não alcança o limiar necessário. Mas essa é uma condição que pode melhorar com tempo e coleta.

Por isso, o caso é elegível a Real Options. A pergunta passa a ser quanto vale manter a opção de treinar mais tarde, quando a qualidade do dataset melhorar.

O código deriva a volatilidade a partir das simulações de Monte Carlo e monta uma árvore binomial de nove passos, um por mês, até o momento estimado em que o score de dados pode cruzar o limiar.

A decisão é induzida de trás para frente. Em cada nó, o algoritmo compara treinar naquele momento com continuar esperando.

Decidir imediatamente possui valor econômico baixo porque o risco associado ao dado insuficiente supera a economia esperada. A opção de esperar, por outro lado, recebe valor positivo, na ordem de algumas centenas de reais na demonstração.

O valor varia entre execuções porque a volatilidade vem da própria simulação. O ponto não é decorar o número, mas transformar espera em decisão econômica mensurável.

Também fica registrada uma reavaliação futura. Esperar não significa abandonar o caso, mas assumir o compromisso de rodar novamente o framework quando houver mais dados.

Por que Atendimento ao Cliente não recebe Real Options

Atendimento ao Cliente mostra o limite dessa técnica.

Seu score composto pode parecer relativamente alto e até superar o de Saúde Empresarial. Se eu decidisse pela média, poderia aprovar o caso incorretamente.

O framework, porém, continua avaliando condições essenciais. A tarefa não é estreita e repetida e também não é suficientemente estável. A dimensão de dados está verde.

Uma diferença importante dessa etapa é que o score ponderado não substitui os gates. Ele ajuda a quantificar importância e robustez, mas não transforma uma pergunta essencial em algo que possa ser compensado pela média. Isso explica por que Atendimento ao Cliente pode apresentar score composto superior ao de Saúde Empresarial e ainda assim ser rejeitado. O framework continua olhando para as condições necessárias que ficaram vermelhas.

Esperar mais meses não resolve esses problemas. Real Options só faz sentido quando a reprovação pode melhorar com o tempo, como insuficiência de dados. Ela não transforma uma tarefa aberta em estreita nem estabiliza uma política que muda por natureza.

Por isso, Atendimento ao Cliente é reprovado sem valor de espera e continua mais adequado a técnicas como prompt, RAG, context engineering e roteamento humano.

Aprovar fine-tuning não escolhe a técnica

A saída do framework registra outra distinção importante: aprovar fine-tuning responde se vale a pena treinar, não como treinar.

Essa segunda decisão será aprofundada nos módulos 3 e 4. Mesmo depois de um caso passar por dados, estabilidade, governança e análise financeira, ainda preciso escolher técnica, provedor e infraestrutura.

O tipo de fine-tuning depende do problema, do hardware, do orçamento e da distância entre o comportamento desejado e aquilo que o modelo base já sabe fazer.

Full Fine-Tuning, LoRA e QLoRA

Full Fine-Tuning ajusta todos os parâmetros do modelo. É mais caro e exige mais infraestrutura, mas oferece maior liberdade de adaptação quando o domínio está distante do comportamento original.

O material cita como referência histórica o Codex, derivado de treinamento especializado em código e associado ao surgimento de experiências como o GitHub Copilot.

LoRA segue outro caminho. O modelo base permanece congelado e apenas adaptadores menores são treinados. Isso reduz memória, custo e tempo de treinamento.

A aula apresenta LoRA como uma escolha muito comum em produção e cita um caso da Checkr, em classificação de registros de background check, como exemplo de redução de custo e aumento de velocidade em relação a uma solução anterior baseada em modelo maior.

QLoRA combina adaptadores com quantização, tipicamente em 4 bits, para reduzir ainda mais o consumo de memória. O exemplo do Guanaco mostra como modelos grandes puderam ser ajustados com hardware muito menor do que seria necessário em Full Fine-Tuning tradicional.

Instruction Tuning, RLHF, DPO e destilação

Instruction Tuning ensina o modelo a seguir instruções em linguagem natural em uma variedade de tarefas. O exemplo citado é o FLAN, ajustado sobre muitas tarefas para melhorar a generalização diante de novas instruções.

RLHF e DPO trabalham com preferência entre respostas. RLHF utiliza um modelo de recompensa e depois reinforcement learning. DPO simplifica o pipeline ao otimizar diretamente sobre pares preferidos e rejeitados.

A destilação busca transferir comportamento de um modelo grande para um menor, frequentemente usando exemplos produzidos pelo próprio modelo maior. O objetivo é reduzir custos de inferência mantendo capacidades relevantes.

O cheat sheet da disciplina ainda registra técnicas mais recentes, como GRPO e RFT, relacionadas a ajuste por recompensa verificável, para quem quiser aprofundar além do escopo desta aula.

O cheat sheet e a segunda decisão

O cheat sheet funciona como um mapa para escolher a técnica depois que o treinamento já foi aprovado.

Ele compara requisitos de dados, hardware, orçamento, hiperparâmetros e exemplos de mercado. Também propõe perguntas práticas: quão distante o domínio está do modelo genérico, quanto orçamento existe e qual latência a produção exige.

A lógica é a mesma do Decision Framework, mas aplicada à segunda decisão. Primeiro eu provo que fine-tuning merece existir. Depois escolho Full Fine-Tuning, LoRA, uma API gerenciada ou outra abordagem.

Essa separação evita escolher uma tecnologia de treinamento antes de provar que o investimento faz sentido.

Risco operacional de provedor

A aula também introduz o risco operacional do provedor.

Um serviço self-service pode mudar de estratégia, remover uma funcionalidade ou deixar de oferecer determinado fluxo de fine-tuning. O material registra mudanças observadas no mercado durante a preparação da disciplina como exemplo desse risco.

Isso não muda se fine-tuning vale a pena. Muda onde treinar.

O material ainda chama atenção para mudanças concretas em ofertas self-service de fine-tuning e para a possibilidade de um provedor abandonar um produto. A consequência prática é que a decisão técnica precisa vir acompanhada de uma estratégia de saída. Se o treinamento depende de uma única API sem portabilidade, o risco operacional pode se tornar maior do que o ganho inicial de conveniência.

Por isso, arquitetura e planejamento precisam considerar dependência de fornecedor, portabilidade de artefatos e alternativas locais ou em outros provedores.

O modelo de fronteira pode alcançar o modelo customizado

Existe ainda outra forma de obsolescência. Um modelo de fronteira futuro pode superar um modelo especializado que hoje representa vantagem.

O material utiliza um caso do setor jurídico para mostrar essa evolução. Um modelo customizado pode ser superior em determinado momento e, algum tempo depois, modelos generalistas mais novos podem alcançar ou ultrapassar seu desempenho.

Isso não significa que o treinamento anterior tenha sido um erro. Ele pode ter gerado valor durante o período em que possuía vantagem.

Significa que fine-tuning tem prazo de validade operacional e precisa ser reavaliado contra novas baselines, novos custos e novos modelos base.

Missão prática do módulo

O módulo termina com uma missão prática em quatro partes.

Primeiro, eu aplico as quatro perguntas a um caso real do meu próprio contexto e registro justificativa para cada resposta.

Segundo, pondero as perguntas. Posso utilizar o AHP completo ou um score mais simples, desde que documente por que determinado critério pesa mais.

Terceiro, estimo retorno financeiro. Não é obrigatório reproduzir toda a simulação de Monte Carlo. Um NPV determinístico, uma economia recorrente ou um break-even já tornam o raciocínio econômico explícito.

Se a única reprovação for dada como insuficiente, documento também quantos meses seriam necessários para acumular dados e por que esperar pode valer mais do que treinar agora. Se a reprovação vier de tarefa aberta ou instável, registro porque Real Options não se aplica.

Por fim, adapto o Decision Framework Tool para os números do meu caso e executo a recomendação.

Concluir que a tarefa não precisa de fine-tuning também é uma entrega válida. O objetivo é decidir corretamente, não decidir treinar sempre.

No próximo módulo, os casos elegíveis deixam a fase de decisão e entram na preparação de dados, transformando documentos brutos da Amplitude Seguros em datasets JSONL limpos e balanceados para treinamento.

NA PRÁTICA

- Adapte o Decision Framework aos números de um caso real ou plausível do seu contexto.
- Registre a ponderação dos critérios e uma análise econômica simples, como NPV, economia recorrente ou break-even.
- Se a única reprovação for insuficiência de dados, registre quando o caso deverá ser reavaliado e por que esperar pode ter valor.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Quando Fazer Fine-Tuning (Decision Framework) (PT-3)”?
- Qual relação existe entre AHP e Razão de consistência no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

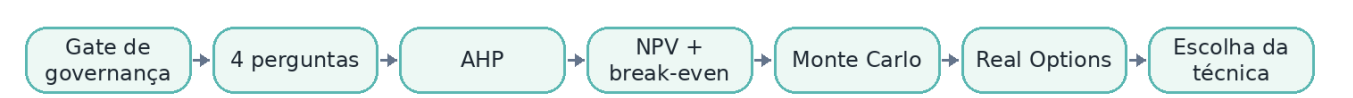
- Nos capítulos anteriores, eu construí e apliquei o framework de quatro perguntas para decidir quando o fine-tuning realmente faz sentido.
- O código executa os três casos da Amplitude Seguros, aplica o gate de governança, deriva pesos por AHP, verifica a consistência do julgamento, calcula NPV e break-even, executa dez mil simulações de Monte Carlo e, quando a reprovação é temporária por insuficiência de dados, calcula o valor de esperar usando Real Options.
- A mudança é importante porque o fine-tuning não deveria ser aprovado apenas por um sinal verde informal.

[↑ Voltar ao mapa da disciplina](#)

Revisão da Unidade 1

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de decisão, preparação ou operação.

FLUXO VISUAL DA UNIDADE



CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-1\)](#)
- [Revisar a Aula 2: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-2\)](#)
- [Revisar a Aula 3: Quando Fazer Fine-Tuning \(Decision Framework\) \(PT-3\)](#)

[↑ Voltar ao mapa da disciplina](#)

Unidade 2

Preparação de Datasets para Fine-Tuning

Preparação de dados de treinamento com gates de relevância, OCR, parsing, validação, redução de PII, representação canônica, deduplicação e balanceamento para aumentar qualidade e diversidade.

GITHUB E PRÁTICA

- [Abrir pasta do Módulo 2](#)
- [Comparativo OCR vs. LLM multimodal](#)
- [Ferramenta de extração multimodal](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

FLUXO VISUAL DA UNIDADE



Aula	Tema
1	Preparação de Datasets para Fine-Tuning (PT-1)
2	Preparação de Datasets para Fine-Tuning (PT-2)

Aula 1 — Preparação de Datasets para Fine-Tuning (PT-1)

Unidade 2 • Preparação de Datasets para Fine-Tuning

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Preparação de Datasets para Fine-Tuning (PT-1)”.
- Relacionar OCR e Tesseract às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Comparativo OCR vs. LLM multimodal](#)
- [Ferramenta de extração LLM multimodal](#)
- [Companion: privacidade e fine-tuning](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• OCR	• Tesseract
• Parser tolerante	• Validação de esquema
• PII scrubbing	• JSONL canônico
• Rastreabilidade	

Conteúdo da aula

O módulo anterior terminou com uma decisão importante. Para Amplitude Auto, o framework indicou que fine-tuning já fazia sentido. Para a Amplitude Saúde Empresarial, a recomendação foi esperar, principalmente porque ainda faltava dado suficiente. Só que decidir treinar ou decidir esperar não produz automaticamente um dataset. A partir daqui, eu começo o trabalho de transformar documentos reais em exemplos estruturados que possam ser utilizados mais adiante no treinamento.

O ponto de partida são documentos de apoio que chegam de fora da plataforma da Amplitude Seguros, como orçamentos de oficina e recibos médicos. Esses documentos podem existir como imagens PNG, fotos de papel ou digitalizações. Antes de limpar ou estruturar qualquer coisa, eu preciso extrair o texto. Por isso, o primeiro componente técnico deste pipeline é OCR, reconhecimento óptico de caracteres.

Mesmo Saúde Empresarial ainda não estando pronto para treinamento, faz sentido preparar os dados com o mesmo rigor. A espera recomendada no módulo 1.3 não é um período de inatividade. É justamente o período em que eu posso melhorar coleta, qualidade, governança e cobertura de variação até o caso cruzar o limiar exigido pelo framework.

Antes do OCR, o documento precisa merecer entrar no dataset

O primeiro erro que eu quero evitar é coletar tudo que existe no arquivo da empresa e chamar isso de dataset.

Preparar dados para fine-tuning não é acumular documentos. Eu preciso definir critérios de relevância e qualidade antes da ingestão.

Esse raciocínio se aproxima da ideia de data-centric AI: melhorar sistematicamente o dado e estabelecer critérios explícitos para aquilo que entra no treinamento, em vez de assumir que mais volume sempre significa mais qualidade.

A aula cita o trabalho LIMA como exemplo dessa lógica. Um conjunto pequeno, mas cuidadosamente curado, pode produzir resultados melhores do que volumes muito maiores de exemplos menos controlados. O número de exemplos importa, mas a qualidade e a representatividade importam antes.

O gate de relevância com quatro perguntas

Antes de coletar qualquer documento, eu aplico um gate com quatro perguntas.

Primeiro, o documento contém o ground truth da tarefa? Eu preciso conseguir observar uma entrada real e uma resposta correta.

Segundo, o documento vem do fluxo real de produção e não foi inventado apenas para ilustrar o problema?

Terceiro, ele ajuda a cobrir a variação real de formatos que a tarefa apresenta?

Quarto, ele pode ser utilizado em treinamento do ponto de vista de sensibilidade e compliance?

As quatro respostas precisam ser verdadeiras. Se uma falha, o documento não entra. Esse gate acontece antes de OCR e parsing. Ele decide se vale a pena gastar qualquer esforço de preparação com aquele item.

Aplicando o gate aos documentos da Amplitude Seguros

A demonstração aplica o gate a sete candidatos plausíveis.

No caso de Auto, eu considero orçamento de oficina, boletim de ocorrência, fotografia do veículo danificado e transcrição da ligação de abertura do sinistro.

Em Saúde Empresarial, aparecem recibo médico, prontuário médico completo e cadastro de beneficiários.

Dos sete candidatos, apenas dois passam: orçamento de oficina e recibo médico.

O boletim de ocorrência não entrega segurado, placa e valor de maneira suficientemente confiável para funcionar como gabarito da tarefa. A foto do veículo mostra o dano, mas não contém o documento textual com os campos desejados. A transcrição da ligação pode conter os dados, mas a estrutura varia demais entre atendentes para funcionar como exemplo consistente de extração.

No caso de Saúde Empresarial, o prontuário completo passa nos critérios técnicos, mas falha em compliance porque contém muito mais dados clínicos do que o necessário para extrair beneficiário, procedimento e valor. O princípio de necessidade da LGPD exige minimizar o dado utilizado para a finalidade.

O cadastro de beneficiários falha por outra razão. Ele é uma tabela de referência útil ao processo de negócio, mas não representa um par de entrada e saída da tarefa de extração. Dado real e limpo não é automaticamente dado adequado para treinar o extrator.

Do documento bruto ao exemplo validado

Depois do gate de relevância, eu organizo o pipeline de ponta a ponta.

O fluxo começa no documento bruto, passa por OCR, segue para um parser tolerante, entra em validação de esquema e termina em um exemplo estruturado pronto para compor o dataset.

Cada estágio resolve um problema diferente. O OCR transforma a imagem em texto. O parser interpreta variações de rótulo e localiza os campos. A validação de esquema verifica se o resultado faz sentido. O exemplo estruturado preserva entrada, saída e metadados para as etapas seguintes.

Misturar tudo em uma única função tornaria mais difícil descobrir onde um erro nasceu. Separar as etapas mantém rastreabilidade.

OCR com Tesseract e idioma português

O motor utilizado na demonstração é o Tesseract.

Ele é um OCR open source com uma longa história de desenvolvimento e utiliza um mecanismo baseado em rede neural nas versões modernas.

Para documentos em português, eu preciso instalar o pacote de idioma correspondente. Isso é importante porque nomes próprios, acentuação, cedilha e termos médicos podem ser reconhecidos de forma inadequada quando o motor opera apenas com configuração de inglês.

O OCR entra especificamente nos documentos de apoio. O formulário principal de sinistro pode chegar estruturado pelo aplicativo ou portal da própria Amplitude. Já o orçamento produzido por uma oficina ou o recibo emitido por uma clínica nasce fora desse sistema e não segue um padrão controlado pela seguradora.

Por que o parser precisa ser tolerante

O texto extraído pelo OCR ainda não resolve o problema.

Cada oficina pode escrever o mesmo campo de maneira diferente. Um documento usa "segurado", outro usa "nome do segurado". Um usa "placa", outro "placa do veículo". Em Saúde Empresarial, um recibo escreve "beneficiário", outro "paciente beneficiário". Um usa "valor", outro "valor cobrado".

Se o parser procurar apenas um rótulo fixo, funcionará em uma fonte e quebrará na próxima.

Por isso, a implementação utiliza expressões regulares tolerantes a pequenas variações de rótulo e não depende de uma posição fixa no texto.

Essa flexibilidade é importante porque o documento de amanhã pode vir de uma oficina que nunca apareceu durante o desenvolvimento. Um parser de demonstração reconhece um layout conhecido. Um parser de produção precisa tolerar variações plausíveis sem começar a inventar campo quando não encontrou o valor.

O risco de aceitar extração incompleta

Sem validação de esquema, uma falha de parsing pode entrar silenciosamente no dataset.

Se o parser procura apenas "segurado" e encontra "nome do segurado", ele pode devolver o campo vazio. Se eu aceitar esse exemplo mesmo assim, o modelo passa a aprender que, em alguns documentos, não identificar o segurado é uma saída normal.

Multiplicado por centenas de documentos, isso deixa de ser um erro isolado e vira ruído sistemático.

A regra, portanto, é rejeitar o exemplo quando campos obrigatórios estão ausentes ou quando o valor extraído é inválido. Quase certo não é suficiente para dado de treinamento.

Executando OCR, parsing e validação de verdade

Na demonstração, o pipeline roda com o binário real do Tesseract sobre quatro imagens sintéticas de documentos.

São dois orçamentos de oficina e dois recibos médicos, cada um com pequenas diferenças de layout e rótulos.

O texto bruto do OCR não é perfeito. Existem espaços extras e quebras de linha pouco elegantes. Isso é esperado e, por isso, eu preservo esse texto como entrada do exemplo em vez de tentar embelezá-lo manualmente.

O parser fica separado e extrai apenas os campos necessários.

Em um dos orçamentos, o OCR identifica segurado, placa do veículo e valor total do reparo. O parser converte o valor monetário brasileiro para número decimal e devolve os campos esperados.

Em um recibo com layout diferente, o parser reconhece "paciente beneficiário" e "valor cobrado" sem depender do rótulo usado no primeiro documento.

Confiança do OCR como metadado

Junto com o texto, eu também guardo a confiança média reportada pelo Tesseract.

Nos quatro documentos da demonstração, ela fica aproximadamente entre 94% e 96%.

Esse número não é inventado pelo pipeline. Ele vem da confiança calculada pelo OCR palavra a palavra e depois agregada.

Em um documento de digitalização ruim, a confiança tenderia a cair. Essa informação pode ser usada mais adiante para decidir se um exemplo precisa de revisão humana antes de entrar no treinamento.

No módulo seguinte, essa confiança se transforma em gate operacional. Aqui ela já entra no metadado para não ser perdida.

Testes automatizados e caminho de rejeição

Antes de confiar no dataset gerado, o pipeline executa vinte e oito testes automatizados.

Os primeiros isolam a conversão de valores em reais para garantir que a vírgula decimal brasileira não seja interpretada de maneira errada.

A maior parte dos testes roda contra os quatro documentos. O OCR precisa extrair texto real, o exemplo precisa passar pela validação, os campos precisam bater com os valores esperados e a confiança precisa permanecer em uma faixa válida.

Os testes finais verificam o caminho contrário. Um exemplo sem placa é rejeitado. Um exemplo com valor zero também é rejeitado.

Eu não quero apenas provar que o pipeline aceita o correto. Preciso provar que ele recusa o incorreto.

O esquema canônico de quatro campos

Todo exemplo aceito passa a carregar quatro campos em uma ordem conceitual estável.

O primeiro é instrução. Ela descreve a tarefa em linguagem natural e permanece a mesma dentro de um caso.

O segundo é entrada. Aqui eu guardo o texto bruto produzido pelo OCR, sem edição manual.

O terceiro é saída. É o objeto estruturado produzido pelo parser. Em Auto, contém segurado, placa e valor. Em Saúde Empresarial, beneficiário, procedimento e valor.

O quarto é metadata. Ele registra o caso, o arquivo de origem, a confiança de OCR e o resultado das validações.

Esse esquema é deliberadamente canônico. Ele não é ainda o formato específico de Vertex AI nem o formato usado pelo treinamento local com MLX. A adaptação para cada provedor fica para os módulos de treinamento.

Por que manter uma representação canônica

Separar preparação de formatação por provedor evita acoplamento desnecessário.

Se amanhã eu trocar o Vertex AI por outro serviço ou migrar parte do treinamento para LoRA local, não quero repetir OCR, parsing, compliance e validação.

Eu mantenho um dataset intermediário estável e crio conversores específicos para cada destino.

Esse desenho também facilita testes. O pipeline de qualidade pode ser validado sem depender da API do provedor e sem exigir que o formato de armazenamento de um fornecedor invada a lógica de preparação.

Rastreabilidade do texto bruto

Guardar o texto OCR bruto possui duas vantagens importantes.

A primeira é rastreabilidade técnica. Se um exemplo apresentar comportamento estranho, eu consigo voltar ao texto original e identificar se o problema nasceu no documento, no OCR ou no parser.

A segunda é auditoria. No caso de Saúde Empresarial, dado de saúde é categoria sensível. Manter a origem e o caminho de transformação ajuda a registrar as operações de tratamento e a entender de onde cada exemplo surgiu.

Existe, porém, uma tensão. Preservar o texto bruto com nome completo significa também preservar PII em um material que pode ser usado para treinamento. Em produção, isso exige uma etapa adicional de pseudonimização antes que o dado chegue ao modelo.

PII scrubbing antes do treinamento

A aula transforma essa recomendação em um gate executável de PII scrubbing.

O pipeline trata nome e CPF nos casos de Auto e Saúde Empresarial. O CPF é validado pelo dígito verificador, não apenas pelo formato textual.

Os nomes são identificados quando aparecem ancorados por rótulos conhecidos, como segurado ou beneficiário.

Depois da redação, nome e CPF deixam de aparecer no exemplo de treino. Placa e valor permanecem, porque são necessários para a tarefa e, isoladamente naquele contexto, não são tratados da mesma forma que o identificador direto.

O limite dessa abordagem é assumido explicitamente. Um nome solto em texto livre não seria capturado apenas por âncora de rótulo. Para esse cenário, eu precisaria de reconhecimento de entidades nomeadas mais robustas, semelhante ao tipo de estratégia utilizada por ferramentas como Microsoft Presidio.

O objetivo não é prometer anonimização completa. É reduzir PII de maneira verificável e deixar claro onde o método deixa de ser suficiente.

Validação continua sendo o portão final

Mesmo depois de OCR, parsing e PII scrubbing, o exemplo ainda precisa passar pela validação.

Campo obrigatório ausente é motivo de rejeição. Valor zero ou negativo também é rejeitado porque normalmente indica que o parser não conseguiu encontrar corretamente o número.

Nos quatro documentos usados na aula, todos passam. Isso não significa que o pipeline foi desenhado para aceitar tudo. Ele foi desenhado para aceitar apenas quando as condições mínimas são atendidas.

É mais barato rejeitar um exemplo ruim durante a preparação do que descobrir o problema depois de treinar o modelo sobre ele.

Quando quatro exemplos viram centenas

Até aqui, eu trabalhei documento por documento. Essa escala é útil para provar que extração e validação funcionam, mas um dataset de fine-tuning real contém centenas ou milhares de exemplos.

Quando a escala aumenta, surgem problemas que não aparecem em quatro documentos.

Um deles é viés de amostragem. Se 90% dos orçamentos vierem de poucas oficinas parceiras, o modelo pode aprender muito bem aqueles formatos e generalizar mal para oficinas novas.

Outro é a repetição quase idêntica. Duas unidades da mesma rede podem utilizar o mesmo sistema e gerar documentos com estrutura quase igual, mudando apenas nome e valor. Um dataset cheio de quase-duplicatas aumenta artificialmente a sensação de diversidade e desperdiça capacidade de treinamento repetindo o mesmo padrão.

Esses problemas não são de extração individual. São problemas de composição do dataset como conjunto.

Preparando o próximo passo

Esta aula fecha a transformação de documento em exemplo.

Eu começo filtrando relevância, extraio texto com OCR, interpreto variações com parser tolerante, valido o contrato, preservo metadados, reduzo PII e escrevo o resultado em JSONL canônico.

No próximo capítulo, a unidade de análise muda. Em vez de perguntar se um exemplo individual está correto, eu preciso perguntar se o conjunto inteiro está repetitivo, enviesado ou desequilibrado.

É aí que entram limpeza, deduplicação e balanceamento. O objetivo passa a ser transformar muitos exemplos individualmente válidos em um dataset que cubra a variação real da tarefa e ensine padrão, não repetição.

NA PRÁTICA

- Aplique o gate de relevância a pelo menos quatro candidatos de fonte de dados e mantenha ao menos um rejeitado com justificativa.
- Defina um esquema canônico com instrução, entrada, saída e metadata.
- Descreva onde OCR, parsing, validação e PII scrubbing entram no pipeline e qual falha cada etapa precisa impedir.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Preparação de Datasets para Fine-Tuning (PT-1)”?
- Qual relação existe entre OCR e Tesseract no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- O módulo anterior terminou com uma decisão importante.
- O ponto de partida são documentos de apoio que chegam de fora da plataforma da Amplitude Seguros, como orçamentos de oficina e recibos médicos.
- Mesmo Saúde Empresarial ainda não estando pronto para treinamento, faz sentido preparar os dados com o mesmo rigor.

[↑ Voltar ao mapa da disciplina](#)

Aula 2 — Preparação de Datasets para Fine-Tuning (PT-2)

Unidade 2 • Preparação de Datasets para Fine-Tuning

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Preparação de Datasets para Fine-Tuning (PT-2)”.
- Relacionar MinHash e LSH às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Projeto do módulo: deduplicação, balanceamento e dataset final](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• MinHash	• LSH
• Deduplicação	• Recall
• Balanceamento por temperatura	• Entropia de Shannon
• Diversidade	

Conteúdo da aula

No capítulo anterior, eu transformei documentos individuais em exemplos estruturados. O pipeline já consegue decidir se um documento é relevante, extrair texto com OCR, interpretar os campos com um parser tolerante, validar o esquema, registrar metadados e reduzir PII antes de escrever o exemplo no dataset. Isso prova que a unidade individual funciona. Ainda não prova que o dataset como conjunto está bom.

Quando eu saio de quatro exemplos e começo a trabalhar com dezenas, centenas ou milhares de registros, surgem problemas que não aparecem na validação documento por documento. Dois deles são especialmente importantes: quase-duplicatas e desbalanceamento de fonte.

Um dataset pode ser formado apenas por exemplos individualmente corretos e, mesmo assim, induzir o modelo ao erro. Se muitos registros repetem praticamente o mesmo padrão, o treinamento reforça informação redundante. Se uma única oficina ou clínica domina a distribuição, o modelo aprende muito bem aquele formato e generaliza pior para fontes novas.

Por isso, nesta etapa eu mudo a unidade de análise. Não quero apenas perguntar se cada exemplo é válido. Quero saber se o conjunto inteiro possui diversidade suficiente para ensinar padrão em vez de repetição.

Escala revela problemas que quatro exemplos escondem

Imagine que a Amplitude Seguros processa centenas de sinistros por mês. Uma oficina parceira de grande volume pode enviar orçamentos toda semana, sempre gerados pelo mesmo sistema e quase com o mesmo layout.

Se essa oficina representar mais da metade do dataset, o modelo verá aquele formato continuamente. Ele pode ficar excelente para documentos dessa fonte e mais frágil quando receber o orçamento de uma oficina nova.

O problema é semelhante em Saúde Empresarial. Uma clínica que processa muitos reembolsos pode dominar o conjunto e fazer com que layouts menos frequentes quase desapareçam do treinamento.

É exatamente nesse ponto que quantidade deixa de ser sinônimo de cobertura. Um dataset grande pode representar apenas a repetição de poucas fontes.

Quase-duplicatas

O primeiro problema é a quase-duplicata.

Um mesmo sinistro pode aparecer duas vezes: porque o operador reenviou um documento ou porque houve uma nova digitalização. O OCR pode produzir pequenas diferenças entre as duas versões, como espaço extra, uma quebra de linha diferente ou um caractere reconhecido de outra forma.

Para uma comparação visual rápida, os registros parecem diferentes. Para o treinamento, eles carregam praticamente a mesma informação.

Se eu mantiver as duas cópias, o modelo recebe um reforço artificial daquele padrão. Ele não aprende uma nova variação do problema. Apenas vê de novo algo que já estava no conjunto.

Em escala, essa repetição desperdiça computação e pode aumentar o risco de memorização.

Por que deduplicação muda o modelo, não apenas o arquivo

A aula usa um estudo de pesquisadores do Google Research sobre deduplicação de dados de treinamento para mostrar que isso não é apenas limpeza estética.

Ao analisar o corpus C4, os pesquisadores encontraram casos extremos de repetição, incluindo uma mesma frase aparecendo dezenas de milhares de vezes.

O resultado da deduplicação aparece no comportamento do modelo. O estudo citado mostra redução forte na emissão de texto memorizado palavra por palavra, necessidade de menos passos de treinamento para atingir desempenho equivalente ou melhor e redução de contaminação entre treino e teste.

Esse último ponto é especialmente importante. Quando exemplos iguais ou quase iguais aparecem nos dois conjuntos, a avaliação passa a medir parcialmente a memória, não generalização. A métrica parece melhor do que o modelo realmente é.

Portanto, deduplicar protege treinamento, custo e avaliação ao mesmo tempo.

Desbalanceamento de fonte

O segundo problema é a concentração de exemplos em poucas fontes.

Uma oficina ou clínica pode produzir mais documentos simplesmente porque possui maior volume de atendimento. Isso é uma característica operacional, mas não significa que ela deva receber o mesmo peso proporcional no dataset de fine-tuning.

Se uma fonte representa 60% do conjunto, ela também representa 60% das oportunidades de ajuste durante o treinamento. O modelo passa a receber muito mais evidência sobre aquele layout do que sobre os demais.

O risco aparece justamente em produção, quando chega uma fonte nova ou pouco frequente. O modelo foi treinado para um retrato enviesado da distribuição histórica e não para a variedade que eu gostaria que ele suportasse.

Padronização antes da comparação

Antes de procurar duplicatas, eu faço uma normalização básica do texto.

A comparação não deveria tratar letras maiúsculas, espaços duplicados ou sobras no início e no fim como diferença de conteúdo.

Por isso, o texto é padronizado com transformação para minúsculas, colapso de espaços e remoção de excesso nas extremidades.

Essa etapa não tenta reescrever semanticamente o documento. Ela apenas elimina diferenças cosméticas que poderiam fazer duas versões praticamente iguais parecerem mais distantes do que realmente são.

MinHash para aproximar similaridade

Comparar cada documento contra todos os outros funciona em conjuntos pequenos, mas cresce rapidamente com o número de exemplos.

A estratégia utilizada na aula é transformar cada texto em uma assinatura MinHash.

O texto é representado por pequenos fragmentos e a assinatura resume esse conjunto em uma quantidade fixa de valores. A fração de posições iguais entre duas assinaturas aproxima a similaridade de Jaccard sem exigir comparar todos os fragmentos diretamente.

Na demonstração, cada exemplo recebe uma assinatura com 32 valores.

MinHash não decide sozinho se dois documentos são duplicados. Ele cria uma representação barata para reduzir o espaço de comparação.

LSH para gerar candidatos

Depois do MinHash, eu aplico LSH, Locality Sensitive Hashing.

A assinatura é dividida em bandas. Na demonstração, são oito bandas com quatro valores em cada uma.

Se dois exemplos colidem em pelo menos uma banda, eles viram candidatos a quase-duplicata.

A vantagem é que eu não preciso calcular a similaridade exata para todos os pares possíveis. O LSH reduz drasticamente o número de comparações que chegam à etapa mais cara.

No conjunto da aula, 549 comparações possíveis por força bruta são reduzidas para apenas 20 candidatos. Isso representa uma redução de aproximadamente 96,4% no trabalho de comparação.

Mesmo assim, os três pares de quase-duplicatas plantados propositalmente são encontrados. Nenhum escapa.

Refino exato antes de remover

LSH serve para localizar candidatos, não para tomar a decisão final.

Os pares encontrados passam por uma comparação exata de similaridade antes de qualquer remoção.

Essa etapa evita tratar como duplicata dois documentos que utilizam o mesmo template, mas representam casos realmente diferentes.

A aula mostra justamente esse cenário com dois orçamentos da mesma oficina. A estrutura é parecida, mas seguro, valor e demais informações pertencem a sinistros distintos.

Mesmo template não é duplicata.

A combinação entre MinHash, LSH e refino exato permite reduzir custos sem transformar a proximidade visual em decisão automática.

Recall na deduplicação

Uma deduplicação eficiente não é apenas aquela que reduz comparações. Ela precisa continuar encontrando os duplicados relevantes.

Na demonstração, os três pares plantados são recuperados. Isso representa um recall perfeito para o conjunto de testes.

Esse indicador é importante porque uma otimização que economiza computação mas deixa muitas duplicatas escaparem não resolve o problema.

O objetivo é diminuir o número de pares analisados mantendo a capacidade de encontrar os casos que realmente precisam ser removidos.

Balanceamento por temperatura

Depois da deduplicação, eu ainda preciso corrigir a concentração por fonte.

A abordagem utilizada é a amostragem por temperatura. Em vez de cortar arbitrariamente qualquer fonte que passe de determinado percentual, eu atribuo pesos de acordo com a frequência original elevada a um expoente alfa.

Com alfa igual a 1, a distribuição permanece proporcional ao volume original. À medida que alfa diminui, a distribuição é suavizada em direção a uma participação mais uniforme entre as fontes.

Na demonstração, utilizo alfa igual a 0,3, valor também associado a estratégias de balanceamento em trabalhos como mT5.

O objetivo é reduzir a dominância das fontes grandes sem fingir que as fontes pequenas possuem exemplos que não existem.

Do peso contínuo à quantidade inteira

O cálculo por temperatura produz pesos contínuos. O dataset, porém, precisa terminar com uma quantidade inteira de exemplos por fonte.

Para fazer essa conversão, eu utilizo o método do maior resto, com uma restrição adicional de capacidade.

Nenhuma fonte pode receber uma quantidade maior do que o número de exemplos que realmente possui.

Essa restrição é fundamental. Balancear não significa duplicar artificialmente um exemplo minoritário apenas para preencher uma cota. Significa utilizar de forma mais equilibrada o dado disponível.

As fontes dominantes perdem participação relativa e as menores mantêm ou ganham espaço dentro do limite real de seus registros.

O efeito do balanceamento na Amplitude Auto e Saúde Empresarial

Antes do balanceamento, a Oficina Estrela representa aproximadamente 53,8% dos exemplos de Auto.

Depois da amostragem por temperatura, sua participação cai para 40%.

Em Saúde Empresarial, a Clínica Vitalis começa com aproximadamente 61,1% do conjunto e termina em 50%.

A intenção não é eliminar essas fontes. Elas continuam importantes porque representam volume real. O que eu evito é permitir que o volume de uma única origem substitua a diversidade do problema.

As fontes menores passam a ter participação suficiente para que seus formatos também influenciem o treinamento.

Entropia de Shannon como medida de diversidade

Dizer que o dataset ficou mais diverso ainda seria uma afirmação subjetiva se eu não tivesse uma medida.

Para isso, utilizo a entropia de Shannon sobre a distribuição das fontes.

Quanto mais concentrado o dataset em poucas fontes, menor a entropia. Quanto mais equilibrada a distribuição, maior o valor.

A aula também transforma essa entropia em número efetivo de fontes usando sua exponencial. Isso responde a uma pergunta mais intuitiva: a quantas fontes igualmente representadas essa distribuição equivale?

Em Auto, com quatro fontes reais, o número efetivo sobe de aproximadamente 3,279 para 3,742 depois do balanceamento.

Em Saúde Empresarial, com três fontes reais, o número efetivo sobe de aproximadamente 2.544 para 2.814.

Agora eu consigo demonstrar numericamente que o conjunto final está menos concentrado.

O dataset fica menor e melhor distribuído

A demonstração começa com 47 exemplos simulados, sendo 28 de Auto e 19 de Saúde Empresarial.

Depois da deduplicação, restam 44.

Após o balanceamento por temperatura, o dataset final fica com 34 exemplos.

O resultado é um arquivo menor, mas com menos repetição e distribuição mais equilibrada entre as fontes.

Esse é um exemplo concreto de por que remover exemplos pode melhorar um dataset. O objetivo não é maximizar a quantidade bruta. É maximizar informação útil e cobertura da tarefa.

Os testes automatizados do pipeline

Antes de confiar nessa transformação, o pipeline executa dezesseis testes automatizados.

Eles verificam a aproximação do MinHash em relação à similaridade exata, o recall do LSH, a detecção das duplicatas plantadas, a restrição de capacidade no balanceamento e o comportamento das métricas de diversidade.

Um dos princípios testados é que suavizar a distribuição não deve reduzir a diversidade medida.

Também verifico que nenhuma fonte recebe mais exemplos do que realmente possui.

Esses testes transformam limpeza e balanceamento em comportamento verificável, não em uma sequência de decisões manuais difíceis de reproduzir.

Por que implementar MinHash e LSH diretamente

A aula também registra uma decisão prática da implementação.

No momento da construção do material, não havia uma biblioteca JavaScript madura que resolvesse de forma satisfatória toda a combinação de MinHash e LSH Banding necessária para o exemplo.

Por isso, a dupla é implementada diretamente no projeto.

Isso não significa que a técnica seja didática ou exclusiva desta disciplina. MinHash e LSH aparecem em pipelines reais de deduplicação de grandes corpora.

O ponto é que a disponibilidade de biblioteca depende do ecossistema. Quando a abstração pronta não existe, eu ainda preciso compreender a técnica para implementar ou validar a solução.

Qualidade e volume precisam ser medidos juntos

O aprendizado deste módulo não é escolher qualidade no lugar de volume.

Fine-tuning continua precisando de cobertura suficiente. O que eu não posso fazer é contar exemplos repetidos ou concentrados como se fossem informação nova.

O material volta ao exemplo do trabalho LIMA para reforçar essa ideia: a curadoria pode permitir que um conjunto menor seja mais útil do que um conjunto muito maior e menos controlado.

MinHash, LSH, amostragem por temperatura e entropia fornecem mecanismos formais para transformar um conjunto bruto em um dataset mais representativo.

Eu continuo medindo volume, mas agora também meço redundância e diversidade.

Missão prática do módulo

A missão prática leva esse pipeline para o contexto de cada aluno.

Primeiro, eu aplico o gate de relevância do capítulo anterior a pelo menos quatro candidatos de fonte de dados, mantendo pelo menos um candidato rejeitado com justificativa.

Segundo, declaro um esquema canônico com instrução, entrada, saída e metadata, adaptando os campos de saída para a tarefa escolhida.

Terceiro, monto ou simulo de forma realista um dataset com pelo menos quinze exemplos vindos de pelo menos três fontes diferentes.

Depois, executo deduplicação com MinHash e LSH e balanceamento por temperatura, adaptando a ferramenta disponibilizada no módulo.

Não é necessário reimplementar tudo do zero. O importante é compreender o papel de cada etapa e ter código executável com pelo menos um teste automatizado provando que o comportamento esperado foi detectado.

Também é válido chegar à conclusão de que a principal fonte disponível ainda não possui volume suficiente para treinamento. O objetivo continua sendo decidir e preparar corretamente, não forçar fine-tuning em qualquer cenário.

Da preparação para o treinamento gerenciado

Com esta etapa, o módulo 2 fecha o pipeline de preparação.

No primeiro capítulo, eu garanti que cada documento relevante poderia virar um exemplo individual válido, rastreável e com tratamento de PII.

Agora eu garanto que o conjunto não esteja dominado por quase-duplicatas nem por poucas fontes.

A partir do próximo módulo, essa mesma lógica será escalada para o volume utilizado no treinamento real da Amplitude Seguros.

É então que entram conversão para o formato do provedor, upload, acompanhamento de jobs, hiperparâmetros e monitoramento em uma API comercial gerenciada.

O treinamento só começa depois que o dado passou por esse conjunto de gates. Primeiro eu provo que o exemplo é correto. Depois provo que o dataset é diverso o suficiente. Só então vale pagar para o modelo aprender com ele.

NA PRÁTICA

- Monte ou simule um dataset com exemplos provenientes de fontes diferentes.
- Aplique deduplicação com MinHash/LSH e depois balanceamento por temperatura, respeitando a capacidade real de cada fonte.
- Compare a distribuição antes e depois usando uma medida de diversidade, como a entropia de Shannon apresentada na aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Preparação de Datasets para Fine-Tuning (PT-2)”?
- Qual relação existe entre MinHash e LSH no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

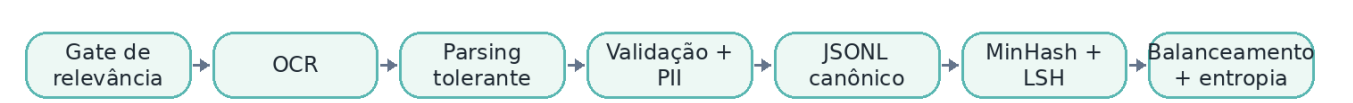
- No capítulo anterior, eu transformei documentos individuais em exemplos estruturados.
- Quando eu saio de quatro exemplos e começo a trabalhar com dezenas, centenas ou milhares de registros, surgem problemas que não aparecem na validação documento por documento.
- Um dataset pode ser formado apenas por exemplos individualmente corretos e, mesmo assim, induzir o modelo ao erro.

[↑ Voltar ao mapa da disciplina](#)

Revisão da Unidade 2

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de decisão, preparação ou operação.

FLUXO VISUAL DA UNIDADE



CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Preparação de Datasets para Fine-Tuning \(PT-1\)](#)
- [Revisar a Aula 2: Preparação de Datasets para Fine-Tuning \(PT-2\)](#)

[↑ Voltar ao mapa da disciplina](#)

Unidade 3

Fine-Tuning via API (upload e train)

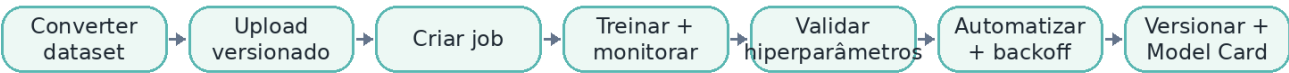
Treinamento gerenciado via API: conversão e upload de dados, criação e monitoramento de jobs, hiperparâmetros, automação segura, rastreabilidade, versionamento e Model Card.

GITHUB E PRÁTICA

- [Abrir pasta do Módulo 3](#)
- [Guia opcional de setup Google Cloud](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

FLUXO VISUAL DA UNIDADE



Aula	Tema
1	Fine-Tuning via API (upload e train) (PT-1)
2	Fine-Tuning via API (upload e train) (PT-2)
3	Fine-Tuning via API (upload e train) (PT-3)
4	Fine-Tuning via API (upload e train) (PT-4)
5	Fine-Tuning via API (upload e train) (PT-5)

Aula 1 — Fine-Tuning via API (upload e train) (PT-1)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-1)”.
- Relacionar Fine-tuning gerenciado e Vertex AI às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Guia opcional de setup Google Cloud](#)
- [Projeto do módulo: fine-tuning gerenciado](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Fine-tuning gerenciado	• Vertex AI
• Conversão de dataset	• Upload
• Job de tuning	• Monitoramento
• Artefato versionado	

Conteúdo da aula

No módulo anterior, eu fechei a preparação do dataset da Amplitude Seguros. Primeiro, filtrei quais documentos realmente mereciam virar exemplos de treino, passei por OCR, parsing tolerante, validação de esquema e PII scrubbing. Depois, olhando o conjunto como um todo, apliquei deduplicação com MinHash e LSH, balanceamento por temperatura e métricas de diversidade com entropia de Shannon. Nesse ponto, o dado já não é apenas um conjunto de documentos convertidos para JSONL. Ele passou por gates de qualidade e por uma curadoria formal antes de chegar ao treinamento.

Seria natural começar este capítulo enviando imediatamente esse dataset para algum provedor. Eu ainda não faço isso de imediato porque existe uma decisão arquitetural anterior: o provedor que parecia óbvio quando a disciplina foi planejada pode não continuar disponível quando o pipeline chegar à produção.

Essa mudança de perspectiva é importante porque o dataset pronto não significa infraestrutura de treinamento garantida. Fine-tuning via API comercial gerenciada depende de um terceiro, e esse terceiro pode alterar produto, modelo, política de acesso ou estratégia comercial independentemente da utilidade que aquela API tenha para o meu sistema.

Fine-tuning via API como segunda decisão

No módulo 1, o Decision Framework respondeu se fine-tuning valia a pena. Essa era a primeira decisão.

No módulo 2, eu tratei a condição que sustenta qualquer treinamento: dado relevante, estruturado, validado, higienizado, duplicado e suficientemente diverso.

Agora começa a segunda decisão: como treinar. Posso seguir por uma API gerenciada na nuvem ou por um caminho local com LoRA. O módulo 3 explora o primeiro caminho. O módulo 4 explora o segundo.

Essa separação precisa permanecer explícita. Aprovar fine-tuning não significa que API gerenciada seja automaticamente a melhor técnica. A escolha só fica completa quando eu consigo comparar custo, operação, tempo, controle e dependência de infraestrutura dos dois caminhos.

O risco de depender de um único provedor

Durante a preparação desta disciplina, dois provedores que eram escolhas naturais para fine-tuning via API mudaram de disponibilidade.

O material registra que o self-service de fine-tuning da OpenAI entrou em processo de descontinuação para novos acessos e para contas inativas, com desligamento completo previsto em etapas. Também registra que a API pública do Gemini para fine-tuning já havia sido encerrada anteriormente.

O ponto não é concluir que esses provedores são ruins. É exatamente o contrário: eram nomes centrais quando se falava em fine-tuning gerenciado. Se serviços tão conhecidos podem mudar em um intervalo relativamente curto, então a disponibilidade precisa fazer parte da arquitetura.

Um tutorial antigo pode continuar correto conceitualmente e ainda assim apontar para um fluxo que já não existe. Quem constrói produção precisa verificar a capacidade real do provedor na data em que vai usar, não apenas confiar em documentação histórica, post de blog ou exemplo de curso.

Diligência de provedor como parte da arquitetura

A Camila, no caso da Amplitude Seguros, não aprova um provedor apenas porque ele possui documentação e uma API funcional. A pergunta é de continuidade operacional.

Se o time precisar retreinar o modelo seis meses depois, o serviço continuará disponível? O modelo base ainda estará suportado? O acesso continuará self-service? O contrato e as permissões continuarão adequados?

Essa diligência evita que uma solução tecnicamente correta se torne operacionalmente frágil. Fine-tuning não é um evento isolado. O modelo será versionado, reavaliado e possivelmente retreinado. Se a infraestrutura usada no primeiro ciclo desaparecer, a manutenção futura fica comprometida.

A escolha de provedor precisa, portanto, considerar disponibilidade, integração, suporte, governança e estratégia de saída.

O provedor adotado na disciplina

Para os próximos capítulos, o caminho escolhido é o Vertex AI dentro do ecossistema do Google Cloud, apresentado no material sob a marca associada à plataforma empresarial do Gemini.

A escolha não acontece por falta de alternativas. O conteúdo também cita provedores como Azure OpenAI, Together AI, Fireworks e Mistral como opções que continuam relevantes em cenários de fine-tuning gerenciado.

O Vertex AI é escolhido principalmente por integração com o restante do stack utilizado na disciplina. Armazenamento, identidade, permissões, faturamento, cotas e operação de modelos ficam dentro do mesmo ecossistema de nuvem.

Essa integração reduz a quantidade de peças novas introduzidas apenas para treinar. Para a Amplitude Seguros, isso significa que o modelo ajustado pode ser tratado como mais um artefato governado dentro da infraestrutura de cloud do projeto.

O processo genérico de fine-tuning gerenciado

Mesmo com diferenças entre provedores, o processo de fine-tuning por API comercial costuma seguir cinco passos.

Primeiro, preparar o dataset no formato exato esperado pelo provedor. Segundo, enviar esse dataset para um armazenamento acessível ao serviço de treinamento. Terceiro, configurar o job escolhendo o modelo base e os hiperparâmetros. Quarto, executar o treinamento de forma gerenciada na nuvem. Quinto, monitorar o job até chegar a sucesso ou falha.

Essa sequência é suficientemente estável para ser tratada como arquitetura. O que muda de provedor para provedor é o contrato concreto: formato dos dados, SDK, nomes dos parâmetros, modelos disponíveis e superfície de monitoramento.

Passo 1: converter o dataset

O pipeline do módulo 2 já trabalha com um esquema canônico de quatro campos: instrução, entrada, saída e metadata.

Esse esquema canônico não precisa ter o mesmo formato consumido pelo provedor. Na verdade, cada serviço pode exigir uma estrutura específica, muitas vezes baseada em JSONL com papéis como system, user e model.

É justamente aqui que o investimento em um formato intermediário bem definido se paga. Eu não quero reescrever gate de relevância, OCR, PII scrubbing, deduplicação, balanceamento e validação toda vez que trocar de provedor.

Eu mantenho o pipeline de dados independente e adiciono apenas uma etapa de adaptação no final. Se o fornecedor mudar, substituo o conversor de saída, não o processo inteiro de preparação.

Passo 2: fazer upload

Depois de converter o dataset, preciso disponibilizá-lo para o serviço de treinamento.

Em um ambiente gerenciado, isso normalmente significa armazenar o arquivo em um bucket ou serviço equivalente na nuvem. O caminho precisa ser endereçável e o serviço de treinamento precisa possuir permissão de leitura.

Esse detalhe aparentemente simples já cria uma fronteira operacional importante. Um arquivo correto pode falhar no treinamento se o serviço não tiver acesso ao bucket, se a região estiver incompatível ou se a política de identidade estiver incorreta.

Por isso, upload não é apenas copiar um arquivo. Ele faz parte do contrato entre pipeline de dados, armazenamento e infraestrutura de treinamento.

Passo 3: configurar o job

Com o dataset disponível, eu configuro o job.

Essa etapa inclui a escolha do modelo base e dos hiperparâmetros, como número de épocas, taxa de aprendizado e tamanho de batch.

Cada provedor define faixas e nomes próprios, mas a função conceitual desses parâmetros continua a mesma. Eles controlam quanto e como o modelo será ajustado a partir dos exemplos.

Nos próximos capítulos, essa configuração ganha profundidade. Neste momento, o importante é entender que ela faz parte da declaração do job e não da lógica de preparação do dataset.

Passo 4: treinamento gerenciado

Depois da configuração, o provedor executa o treinamento.

Eu não administro diretamente a GPU, o processo distribuído ou o carregamento do modelo. A infraestrutura é responsabilidade do serviço gerenciado.

Isso reduz a complexidade operacional para quem quer treinar sem manter um ambiente próprio. Em troca, eu aceito depender de fila, disponibilidade, modelos suportados, limites e preços definidos pelo provedor.

Mesmo com datasets de centenas de exemplos, o treinamento não é instantâneo. O serviço precisa alocar recursos, carregar o modelo base e executar as épocas definidas. O tempo relevante fica concentrado nessa etapa.

Passo 5: monitorar o job

A última etapa é acompanhar o status.

Um job costuma passar por estados como pending, running e, por fim, succeeded ou failed.

Falha também é informação útil. O problema pode estar em formato inválido, hiperparâmetro rejeitado, cota, permissão ou outra validação feita pelo provedor.

O monitoramento pode acontecer por consultas periódicas, notificações ou pelo console do serviço. O ponto é que o pipeline de produção precisa saber identificar quando o job terminou e qual artefato foi produzido.

Escala de tempo do processo

Preparar e subir o dataset tende a ser rápido para a escala usada na Amplitude Seguros. Mesmo depois de deduplicar e balancear, estamos falando de centenas de exemplos, não de milhões.

Configurar o job também é uma operação curta. O tempo real está no treinamento.

Isso prepara uma comparação importante para o módulo 4. Tanto na nuvem quanto localmente, um dataset pequeno pode levar minutos. A diferença estrutural não é apenas a duração. É quem controla e paga por aquele tempo.

No serviço gerenciado, existe uma fila e uma infraestrutura compartilhada. No treinamento local, a máquina está sob meu controle, desde que eu tenha hardware suficiente. Essa comparação será feita com números mais adiante.

Cada treinamento gera um artefato versionado

Outro detalhe importante é que o resultado de um treinamento não deve ser tratado como um modelo que muda silenciosamente.

Cada novo ciclo com dataset ou hiperparâmetros diferentes gera um novo checkpoint ou endpoint versionado.

Isso significa que retreinar não deveria substituir o artefato anterior sem rastreabilidade. Produção precisa saber qual versão está atendendo, com qual dataset e com quais configurações.

Essa discussão se conecta ao módulo de versionamento e documentação de modelos, em que o modelo fine-tunado passa a ser tratado como artefato de software e de machine learning, não como um estado invisível do provedor.

Vertex AI dentro do ecossistema Google Cloud

O Vertex AI adotado nesta disciplina não deve ser confundido com a antiga API pública do Gemini que oferecia fine-tuning diretamente por API key.

Aqui eu trabalho com uma plataforma gerenciada de Google Cloud, integrada a IAM, armazenamento, cotas, faturamento e registro de modelos.

O fluxo passa por um dataset armazenado em bucket, um job supervisionado de treinamento e um endpoint publicado ao final.

Também existe um ponto importante sobre o ciclo de vida dos modelos. O próprio material registra que versões específicas do Gemini possuem datas de retirada anunciadas. Portanto, o nome exato do modelo usado em uma demonstração pode mudar quando o aluno executar o exercício.

O processo arquitetural continua válido: escolher uma versão vigente com suporte a fine-tuning supervisionado e verificar a documentação atual antes de montar o job.

Validação prática antes dos dados reais

Antes de levar o dataset real da Amplitude Seguros ao provedor, eu valido o caminho completo com dados sintéticos.

A demonstração utiliza 32 exemplos separados dos conjuntos empregados no módulo 2. A intenção não é avaliar a qualidade do modelo. É verificar a infraestrutura e compatibilidade do pipeline.

O job passa por pending, depois running e chega a succeeded em pouco menos de meia hora. O custo estimado fica abaixo de um centavo de dólar e um endpoint é publicado ao final.

Essa execução confirma quatro pontos que documentação nenhuma garante sozinha: a conversão do schema foi aceita, o upload para o bucket funcionou com as permissões corretas, os hiperparâmetros foram aceitos e o status reportado pela API correspondeu ao que aparecia no console do Google Cloud.

É esse tipo de teste ponta a ponta que eu quero antes de colocar dados reais no pipeline.

Governança precisa ser reavaliada ao trocar de provedor

A troca de provedor também reabre a questão de governança.

No módulo de preparação, eu já reduzi PII antes do treinamento e tratei Saúde Empresarial com atenção adicional por envolver dado sensível. Mesmo assim, uma base legal e um DPA aprovados para um serviço não se transferem automaticamente para outro.

Cada provedor possui contrato, condições de processamento e responsabilidades próprias. Se o motivo da troca foi descontinuação de mercado, a revisão ainda precisa acontecer.

Esse ponto reconecta o módulo 3 ao gate apresentado no módulo 1 e ao tratamento de dados do módulo 2. Governança não é uma etapa que eu marco uma vez e esqueço. Ela acompanha mudanças de infraestrutura e de fluxo de dados.

Do dataset preparado ao treinamento gerenciado

Com o dataset já preparado e o provedor validado, o próximo passo é juntar as duas partes do pipeline.

O mesmo conjunto que passou por relevância, extração, validação, higienização, deduplicação e balanceamento será convertido para o formato esperado pelo Vertex AI, enviado para o armazenamento e utilizado na criação de um job real.

Os próximos capítulos entram em upload, acompanhamento, hiperparâmetros e monitoramento com chamadas reais contra o provedor.

A partir daqui, fine-tuning deixa de ser apenas uma decisão e também deixa de ser apenas um problema de preparação de dados. Ele passa a ser um pipeline operacional completo, no qual qualidade do dataset, contrato com o provedor, versionamento e monitoramento precisam funcionar juntos.

NA PRÁTICA

- Represente o processo gerenciado em cinco passos: converter, fazer upload, configurar o job, treinar e monitorar.
- Registre quais verificações podem ser feitas localmente antes de consumir recursos do provedor.
- Inclua a diligência de provedor e a reavaliação de governança como parte da arquitetura do treinamento.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-1)”?
- Qual relação existe entre Fine-tuning gerenciado e Vertex AI no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No módulo anterior, eu fechei a preparação do dataset da Amplitude Seguros.
- Seria natural começar este capítulo enviando imediatamente esse dataset para algum provedor.
- Essa mudança de perspectiva é importante porque o dataset pronto não significa infraestrutura de treinamento garantida.

[↑ Voltar ao mapa da disciplina](#)

Aula 2 — Fine-Tuning via API (upload e train) (PT-2)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-2)”.
- Relacionar Esquema canônico e Gate de confiança de OCR às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Projeto do módulo: conversão, upload e job real](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Esquema canônico	• Gate de confiança de OCR
• Upload versionado	• Pending / Running / Succeeded
• Tokens e custo	• Orquestração de nuvem

Conteúdo da aula

No capítulo anterior, eu apresentei o caminho de fine-tuning gerenciado que passa a ser utilizado nesta disciplina. Depois de discutir a mudança de disponibilidade de alguns provedores self-service e de validar o Vertex AI ponta a ponta com um pequeno conjunto sintético, chegou a hora de trabalhar com os dados que representam os casos da Amplitude Seguros em uma escala de treinamento mais próxima da prática.

Até aqui, nenhuma execução de fine-tuning havia utilizado o dataset real dos casos de Auto e Saúde Empresarial. Agora eu avanço sobre os dois primeiros passos do processo genérico que defini no módulo 3.1: preparar o dataset no formato exato esperado pelo provedor e fazer o upload para a nuvem, acompanhando o job até o final.

A demonstração não fica restrita à criação do job. Eu também valido a conversão do esquema canônico, aplico o gate de confiança de OCR, escalo o pipeline formal do módulo 2, reavalio Saúde Empresarial com o mesmo Decision Framework utilizado anteriormente e consulto o estado de um job real diretamente na API do Google Cloud.

Por que o job já existe antes da gravação

O job apresentado nesta aula foi criado antes da gravação. Isso não significa que o resultado é simulado. A razão é operacional: um fine-tuning real leva dezenas de minutos e não faria sentido manter uma aula inteira parada esperando a infraestrutura concluir o treinamento.

O que eu executo ao vivo é uma consulta de status real contra a API do provedor. Essa consulta utiliza um token de acesso gerado pelo gcloud na máquina local para autenticar a chamada e provar que a conta possui autorização para acessar o projeto.

A distinção é importante. O treinamento foi real, com dataset real do exercício, e a API consultada durante a aula também é real. Apenas a espera entre a criação do job e seu término foi retirada da gravação.

Do esquema canônico ao formato do Vertex AI

O módulo 2 definiu deliberadamente um esquema canônico genérico com instrução, entrada, saída e metadata. O objetivo era separar a preparação de dados da formatação específica de um fornecedor.

Agora esse desenho começa a pagar o investimento. Para o formato consumido pelo Vertex AI, instrução e entrada são combinadas no turno do usuário. A saída esperada vira o turno do modelo, preservada como o JSON exato que o extrator precisa aprender a devolver.

Cada exemplo passa, portanto, a ter dois turnos: um de usuário e outro de modelo, cada um contendo suas partes de conteúdo.

Essa camada de adaptação é pequena porque limpeza, gate de relevância, deduplicação, balanceamento e demais decisões de preparação continuam independentes do provedor. Se o mercado mudar novamente, eu altero o conversor final e não o pipeline inteiro.

Onde começa a orquestração real de nuvem

Existe uma fronteira clara entre o que ainda roda localmente e o que começa a consumir a infraestrutura do Google Cloud.

A conversão do dataset e o gate de OCR acontecem localmente. Só depois, quando obtenho credenciais e começo a interagir com os serviços de nuvem, entro de fato na orquestração do Vertex AI.

Para quem quiser configurar o próprio projeto, o material disponibiliza um guia opcional de setup do GCP. Esse tipo de separação é útil porque permite testar conversão e regras de qualidade sem depender de credenciais, cota ou custo de nuvem.

Testando a conversão antes do upload

O primeiro bloco da execução roda treze testes automatizados.

Esses testes confirmam que cada exemplo convertido sempre possui dois turnos, que o turno do usuário carrega instrução e entrada completas e que o turno do modelo contém exatamente o JSON esperado.

Também verifico os casos inválidos. Exemplos sem instrução, entrada ou saída são rejeitados antes de qualquer upload.

Isso impede que um erro simples de preparação chegue até o job gerenciado e só seja descoberto depois de alocar recurso de treinamento.

Além disso, a suíte testa o gate de confiança de OCR para garantir que os exemplos sejam aprovados, encaminhados para revisão humana ou ignorados pelo motivo correto.

Convertendo exemplos reais de Auto e Saúde Empresarial

Depois dos testes, eu converto dois exemplos reais do dataset, um de Amplitude Auto e outro de Saúde Empresarial.

No turno do usuário, entra literalmente a instrução combinada com o texto bruto do documento. Eu não faço uma edição manual específica para deixar o exemplo mais fácil para o modelo.

No turno do modelo, a saída é o contrato estruturado esperado para cada caso.

Em Auto, os campos são segurado, placa e valor. Em Saúde Empresarial, os campos são beneficiário, procedimento e valor.

Esse teste visual da conversão ajuda a verificar se o esquema canônico está realmente chegando ao formato do provedor sem perder informação ou alterar o contrato que o modelo precisa aprender.

O gate de confiança de OCR

O campo de confiança de OCR que havia sido preparado anteriormente passa agora a ter uma função concreta no pipeline.

Nos documentos reais escaneados pelo Tesseract no módulo 2.1, a confiança fica aproximadamente entre 94% e 96%. Todos esses exemplos passam pelo gate.

Quando eu simulo um documento mais degradado, com confiança de 62%, o pipeline não o envia automaticamente para treinamento. Ele sinaliza o exemplo para revisão humana.

A regra evita transformar erro de OCR em comportamento aprendido pelo modelo. Se a leitura do documento já é duvidosa, eu não deveria tratá-la como gabarito confiável.

Os 200 exemplos utilizados no job de treinamento desta aula não passam por esse gate porque foram gerados como texto sintético. Eles não carregam uma confiança de OCR real. Nesse caso, a ausência da métrica é interpretada como não aplicável, e não como baixa qualidade.

O gate continua disponível para futuras ingestões de documentos reais.

Escalando o pipeline do módulo 2

Os 34 exemplos didáticos utilizados anteriormente eram suficientes para provar que o pipeline de limpeza funcionava, mas não representavam o volume desejado para um job de treinamento real.

A resposta não foi inventar outro método. Eu escalo exatamente o mesmo pipeline formal.

Continuo utilizando MinHash com LSH para deduplicação, amostragem por temperatura para balancear fontes e entropia de Shannon para medir diversidade.

A execução começa com 305 exemplos brutos, distribuídos entre seis oficinas no caso de Auto e cinco clínicas no caso de Saúde Empresarial. Duas fontes novas são acrescentadas em cada cenário para ampliar a cobertura.

Depois da deduplicação, restam 300 exemplos. O balanceamento por temperatura reduz o conjunto para 200 exemplos finais, sendo 120 de Auto e 80 de Saúde Empresarial.

A diversidade efetiva de fontes também melhora. Em Auto, o número efetivo sobe de aproximadamente 5,516 para 5,723. Em Saúde Empresarial, passa de aproximadamente 4,140 para 4,706.

É a mesma evidência formal utilizada no módulo 2.2, agora aplicada a um conjunto aproximadamente seis vezes maior.

Reavaliando Saúde Empresarial nove meses depois

Existe uma aparente contradição que eu preciso resolver antes de treinar.

No módulo 1.2, Saúde Empresarial foi reprovado por falta de dados suficientes. No módulo 1.3, o Decision Framework recomendou esperar nove meses. Se eu simplesmente incluísse o caso no treinamento agora, estaria ignorando a própria decisão ensinada anteriormente.

Por isso, eu rodo novamente o mesmo framework.

A reavaliação não cria uma lógica nova. O script importa diretamente o Decision Framework já existente e mantém os mesmos pesos, limiar e três perguntas que já estavam verdes. A única dimensão atualizada é a pergunta 3, sobre suficiência dos dados.

O score anterior era 0,35. A projeção usada anteriormente considerava crescimento de 0,03 por mês. Nove meses depois, o cálculo chega a 0,62, acima do limiar de 0,6.

O volume mensal também cresce de 1.200 para aproximadamente 1.862 casos, seguindo a mesma taxa usada na análise anterior.

Dessa vez, nenhuma pergunta permanece vermelha. O caso entra no treinamento porque o mesmo critério que o reprovou antes agora o aprova com dados atualizados. A régua não foi reduzida para caber na demonstração.

Um bug real no gerador sintético

Ao aumentar o volume do dataset sintético, aparece um problema que vale ser tratado como parte da aula.

O gerador original utilizava listas de nomes, placas e valores com o mesmo tamanho, quarenta itens. Em uma fonte com mais de quarenta exemplos, os índices começavam a ciclar juntos.

O exemplo de posição 41 podia repetir nome, placa e valor do exemplo 1, mudando apenas a data. Isso criava colisões artificiais que o detector de duplicidade interpretava como duplicatas.

A correção foi aumentar o período antes da repetição. Os nomes passaram a ser formados por combinações de primeiro nome e sobrenome vindos de pools com tamanhos diferentes, produzindo mais de 1.500 combinações antes de repetir.

Depois da correção, os falsos positivos desaparecem e apenas os cinco pares de duplicatas plantados propositalmente são detectados.

A lição é generalizável. Um gerador sintético com listas curtas pode contaminar a métrica que deveria provar a qualidade do pipeline. Por isso, eu sempre valido as duplicatas do conjunto bruto antes de confiar no resultado.

Upload versionado para o bucket

Com o dataset convertido e salvo em JSONL, o upload para o armazenamento do Google Cloud é simples.

O arquivo é enviado para um bucket específico do projeto e depois referenciado na criação do job de tuning junto com o modelo base e os hiperparâmetros.

Na execução utilizada para esta disciplina, o job foi configurado com Gemini 2.5 Flash, três épocas e um nome de exibição associado ao conjunto de 200 exemplos de Auto e Saúde Empresarial.

O bucket já existia de testes anteriores, mas o dataset atual foi enviado como um novo objeto.

Essa escolha é deliberada. Cada versão de dataset precisa manter seu próprio caminho. Eu não sobrescrevo silenciosamente o arquivo anterior, porque um job que falha ou produz um resultado inesperado precisa continuar apontando para exatamente os dados usados naquele treinamento.

Criando e identificando o job

Depois do upload, o job é criado no serviço de tuning do Vertex AI.

A execução real recebe um identificador próprio e entra inicialmente no estado Pending.

A partir desse momento, o trabalho pesado roda na infraestrutura do provedor. Minha máquina não precisa permanecer responsável pelo treinamento.

O que eu preciso manter é a capacidade de consultar o job, registrar seu identificador e acompanhar as transições de estado até chegar ao resultado final.

Interpretando Pending, Running e Succeeded

Os estados do job deixam de ser abstrações e passam a ter significado operacional.

Pending representa a fila. O provedor já recebeu a solicitação, mas ainda não alocou os recursos necessários para iniciar o treinamento.

Running significa que o treinamento está efetivamente em execução. As épocas estão sendo processadas e os pesos estão sendo atualizados conforme os lotes de exemplos.

Succeeded indica que todas as etapas terminaram sem erro e que o artefato resultante foi publicado.

Existe também o caminho de falha. Se o job chegar a Failed, a resposta da API pode trazer informações como formato inválido do dataset, cota excedida ou hiperparâmetro fora da faixa aceita.

Em produção, esses estados determinam se o sistema apenas espera ou se alguém precisa agir.

Consultando o job real pela API

A parte central da demonstração é uma consulta real ao endpoint REST do serviço de AI Platform do Google Cloud.

A chamada autenticada devolve o estado do job, o modelo base, o nome de exibição, os timestamps de início e fim e os endpoints gerados.

O job apresentado chega a Succeeded.

A duração total foi de aproximadamente 45 minutos e 42 segundos. O tempo é superior ao piloto anterior, que utilizou 32 exemplos e levou cerca de 28 minutos e 32 segundos, porque agora o conjunto de treinamento é aproximadamente seis vezes maior.

Essa comparação também mostra que o custo de um job gerenciado não deve ser estimado apenas pela quantidade de minutos visíveis. O Vertex AI utilizado na demonstração contabiliza o treinamento pelos tokens processados.

Tokens, custo e artefatos resultantes

O job processa aproximadamente 27.353 tokens de treinamento.

O custo registrado para essa execução fica na faixa de cinco a onze centavos de dólar.

Ao final, existem dois artefatos importantes: o modelo ajustado e o endpoint publicado para inferência.

Esses artefatos fecham o ciclo de upload e treinamento. O próximo problema passa a ser acompanhar métricas e configurar hiperparâmetros com mais profundidade.

A consulta de status realizada manualmente nesta aula será automatizada posteriormente, e o endpoint publicado será tratado como artefato versionado e documentado nos capítulos seguintes.

Da validação do job aos hiperparâmetros

O ponto deste capítulo não é apenas mostrar que um job chegou a Succeeded.

Eu valido que o esquema canônico pode ser convertido sem reescrever o pipeline, que exemplos incompletos são bloqueados, que OCR de baixa confiança pode exigir revisão humana e que o mesmo mecanismo de deduplicação e balanceamento escala para um conjunto maior.

Também mostro que uma decisão anterior pode ser reavaliada sem alterar a lógica do framework. Saúde Empresarial só passa a ser treinado porque o score de dados realmente cruza o limiar depois do período de espera projetado.

Por fim, a execução conecta dataset, armazenamento, job gerenciado, estados operacionais e artefatos publicados numa única trilha reproduzível.

No próximo capítulo, eu entro nos hiperparâmetros. O foco passa a ser entender o que cada configuração controla, como escolher épocas e taxa de aprendizado para o volume disponível e o que as estatísticas de monitoramento do provedor conseguem, e não conseguem, revelar sobre o treinamento.

NA PRÁTICA

- Converta exemplos do esquema canônico para o formato esperado pelo provedor escolhido no conteúdo.
- Defina um gate de confiança de OCR antes do upload e registre o motivo de uma eventual rejeição.
- Acompanhe o ciclo de estados do job e registre identificador, tokens, custo e artefatos resultantes quando disponíveis.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-2)”?
- Qual relação existe entre Esquema canônico e Gate de confiança de OCR no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No capítulo anterior, eu apresentei o caminho de fine-tuning gerenciado que passa a ser utilizado nesta disciplina.
- Até aqui, nenhuma execução de fine-tuning havia utilizado o dataset real dos casos de Auto e Saúde Empresarial.
- A demonstração não fica restrita à criação do job.

[↑ Voltar ao mapa da disciplina](#)

Aula 3 — Fine-Tuning via API (upload e train) (PT-3)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-3)”.
- Relacionar Épocas e Underfitting e overfitting às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Projeto do módulo: hiperparâmetros e monitoramento](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Épocas	• Underfitting e overfitting
• Learning rate multiplier	• Adapter size
• Validação pré-rede	• Pedido versus aplicado

Conteúdo da aula

No capítulo anterior, eu executei o pipeline de upload e acompanhamento do fine-tuning gerenciado sobre 200 exemplos da Amplitude Auto e da Amplitude Saúde Empresarial. O job chegou ao estado Succeeded depois de aproximadamente 45 minutos e 42 segundos. Isso prova que o processo funciona operacionalmente, mas ainda falta responder uma pergunta mais importante: ele foi configurado corretamente para o volume e o tipo de dado que eu tinha?

Agora eu quero entender o que cada hiperparâmetro controla, por que um valor pode fazer sentido para um dataset pequeno e deixar de fazer sentido quando a escala muda, e quais mecanismos preciso colocar ao redor da API para não depender cegamente da validação do provedor.

O foco passa de conseguir criar um job para conseguir operá-lo com critérios. Isso inclui escolher hiperparâmetros, validar a configuração antes de gastar recurso, confirmar o que o provedor realmente aplicou e interpretar as estatísticas devolvidas pela plataforma.

Os três hiperparâmetros centrais

A configuração deste módulo trabalha com três hiperparâmetros principais.

O primeiro é o epoch count, que define quantas vezes o modelo percorre o dataset completo. Três épocas significam que cada exemplo participa do processo de ajuste três vezes.

O segundo é o learning rate multiplier. Ele controla a intensidade com que cada exemplo influencia o ajuste. No serviço gerenciado, eu não defino uma taxa absoluta do zero, mas multiplico uma taxa base interna calibrada pelo provedor.

O terceiro é o adapter size, ligado à capacidade da camada adaptativa treinada. Como o fine-tuning eficiente não precisa alterar todos os parâmetros do modelo base, essa camada aprende a correção necessária para o comportamento específico que quero especializar.

Épocas e o equilíbrio entre underfitting e overfitting

O número de épocas precisa equilibrar dois riscos.

Com poucas passagens pelo dataset, o modelo pode não aprender suficientemente o padrão. Esse é um cenário de underfitting.

Com épocas demais, principalmente em datasets pequenos, o modelo pode memorizar exemplos específicos em vez de aprender uma estrutura geral. Esse é o overfitting.

As três épocas utilizadas com os 200 exemplos procuram um ponto intermediário. Não existe uma regra universal dizendo que três é sempre o valor correto. É uma escolha coerente com o tamanho atual do dataset e com o comportamento relativamente estreito que estou treinando.

Relação entre tamanho do dataset e número de épocas

Uma heurística útil é observar a relação entre volume e repetição.

Quando o dataset é menor, mais épocas podem ajudar até certo limite porque cada padrão aparece poucas vezes. Quando o dataset cresce, os padrões se repetem entre exemplos diferentes e muitas épocas podem se tornar desnecessárias.

Por isso, 200 exemplos com três épocas formam um ponto intermediário. Com um conjunto muito maior, uma ou duas épocas poderiam bastar. Com poucos exemplos, eu poderia testar mais passagens, sempre acompanhado por avaliação adequada.

Essa relação é um princípio de partida, não uma fórmula. Em comparações experimentais, às vezes faz sentido manter o número de épocas fixo para isolar outra variável.

Learning rate multiplier

O learning rate multiplier pode ser confundido com uma taxa de aprendizado tradicional.

No Vertex AI, ele atua sobre uma taxa base interna. Um valor próximo de 1 preserva um ritmo próximo do calibrado pelo provedor. Um valor maior intensifica o ajuste a cada exemplo.

Na configuração usada na disciplina, o multiplicador é 5. Com poucas centenas de exemplos e três épocas, um multiplicador baixo poderia produzir alteração pouco perceptível, deixando o modelo fine-tunado muito parecido com o modelo base.

Por outro lado, um valor alto demais pode desestabilizar o treinamento. A decisão precisa considerar volume de dados, objetivo do fine-tuning e quantidade de passagens pelo conjunto.

Adapter size e capacidade da camada treinável

O adapter size representa outra dimensão da decisão.

Um rank baixo restringe a camada adaptativa a menos graus de liberdade e consegue aprender um padrão estreito com menor custo de memória e treinamento.

No caso da Amplitude Seguros, o comportamento é específico: receber um documento e produzir uma saída estruturada curta com três campos principais. Por isso, a configuração utilizada trabalha com adapter size 4.

Ranks maiores, como 8 ou 16, podem capturar comportamentos mais complexos, mas custam mais parâmetros treináveis, memória e tempo. No módulo de treinamento local, essa relação volta com mais profundidade porque o rank passa a afetar diretamente a memória disponível na máquina.

O problema real encontrado no Vertex AI

A parte mais importante da aula surge de um teste deliberadamente inválido.

Antes de gravar, eu quis verificar se o Vertex AI rejeitava rapidamente um hiperparâmetro impossível. Para isso, tentei criar um job com epoch count igual a zero.

A expectativa seria receber a falha de validação antes de qualquer recurso ser alocado. Não foi o que aconteceu.

O provedor aceitou o job, levou a execução para Running e o campo inválido acabou ausente na configuração aplicada. Em vez de recusar explicitamente o valor, a plataforma seguiu sem deixar o problema evidente na criação.

Assim que identifiquei a divergência, cancelei o treinamento. O job ficou ativo por cerca de quarenta segundos. O custo foi pequeno, mas real.

Silêncio da API não significa configuração correta

Se uma API aceita uma chamada sem erro, isso não prova que todos os parâmetros foram aplicados da forma esperada.

Em uma automação semanal de retreino, um bug simples poderia zerar o epoch count e continuar criando jobs durante semanas. O primeiro sintoma talvez surgisse apenas no comportamento ruim do modelo publicado.

Por isso, ausência de reclamação do provedor não pode ser tratada como confirmação. Configuração precisa ser validada por quem chama a API e auditada depois que o job existe.

Primeira camada de proteção: validar antes da rede

A primeira proteção é uma validação local de hiperparâmetros, executada antes de qualquer chamada de rede e antes de qualquer possibilidade de custo.

Na implementação demonstrada, o epoch count precisa estar entre 1 e 20 e ser inteiro. Valores zero, negativos ou fracionários são rejeitados. O learning rate multiplier também recebe uma faixa aceitável, de 0,1 a 10.

Essa validação transforma uma falha que poderia consumir infraestrutura em um erro local imediato e barato. O epoch count zero que escapou da API real passa a ser bloqueado antes da requisição.

Segunda camada de proteção: comparar pedido e aplicado

A segunda proteção acontece depois da criação do job.

Eu comparo os hiperparâmetros solicitados com aquilo que o próprio objeto do job informa ter aplicado.

Essa verificação existe porque nenhuma validação local prevê perfeitamente todas as regras internas do provedor. Se um campo desaparecer, for alterado ou receber outro valor, a divergência precisa aparecer.

As duas camadas não são redundantes. Uma evita erros antes de gastar. A outra protege contra comportamento inesperado depois que o provedor recebeu o pedido.

Comparação numérica e uma pegadinha de API

Algumas APIs do Google devolvem determinados números como texto. Se eu comparar diretamente o número 3 com o texto "3" usando igualdade estrita, o resultado será falso, embora os valores sejam equivalentes.

Por isso, a função de comparação tenta converter os dois lados para número antes de decidir se existe divergência. Quando isso não faz sentido, volta para comparação direta.

Sem esse cuidado, o monitor poderia acusar erro em jobs corretos e levar a cancelamentos desnecessários.

Checklist antes da criação do job

Antes de criar um treinamento real, eu consigo resumir a análise em três perguntas.

O número de épocas é compatível com o tamanho do dataset ou foi apenas copiado de algum tutorial? O learning rate multiplier faz sentido para o volume e a intensidade de ajuste desejada? O adapter size é o menor que representa o comportamento necessário?

Essas perguntas levam pouco tempo, mas evitam que uma configuração ruim entre em uma fila de treinamento apenas porque a API aceitou o payload.

Onde começa a chamada real ao Google Cloud

Assim como na aula anterior, existe uma fronteira entre lógica local e orquestração real de nuvem.

Testes, validações e comparação de configuração podem rodar localmente sem custo. A partir do momento em que o script obtém o token de acesso e consulta o Google Cloud, as chamadas dependem da autenticação real do projeto.

O token é obtido pelo gcloud autenticado na máquina. Se a ferramenta não estiver conectada ao projeto correto, essa parte da demonstração não executa.

Os testes automatizados desta etapa

A ferramenta de hiperparâmetros e monitoramento começa com nove testes automatizados.

Eles confirmam que a configuração real do módulo 3.2 passa pela validação, que epoch count igual a zero é rejeitado, que valores negativos ou não inteiros falham e que a comparação entre solicitado e aplicado detecta divergência de valor e campo ausente.

Outro teste cobre a equivalência entre texto e número, evitando que uma representação diferente da API seja interpretada como alteração real.

Em seguida, o script reproduz o bloqueio do epoch count zero localmente, mostrando o erro antes de qualquer acesso à rede.

Consultando os hiperparâmetros realmente aplicados

Depois dos testes locais, eu consulto novamente o job real do módulo 3.2.

A resposta confirma os valores aplicados. O learning rate multiplier está em 5, o adapter size em 4 e o epoch count corresponde ao valor esperado.

Pedido e aplicado batem.

Esse resultado é tão importante quanto observar o caso inválido. A finalidade da comparação é produzir evidência de que a configuração executada corresponde ao que foi aprovado antes da criação do job. Os hiperparâmetros passam a fazer parte da trilha auditável do treinamento.

Monitoramento sem curva de loss

Quem está acostumado a treinamento local pode esperar que a API devolva uma curva de loss durante o job.

Nesse fluxo do Vertex AI, essa curva não está disponível pela API usada na demonstração. O provedor expõe estatísticas do dataset calculadas durante a criação do job.

A API também aceita um dataset de validação separado, capaz de produzir estatísticas de validação durante o treinamento. Eu deixo esse recurso de fora de propósito porque a avaliação rigorosa do modelo fine-tunado fica para o módulo 5.

Aqui eu quero aprender a operar e monitorar o job sem misturar ainda a discussão completa de baseline e qualidade final.

Estatísticas reais do dataset de treinamento

A consulta do job devolve aproximadamente 27.353 tokens cobráveis para os 200 exemplos.

Também consigo observar estatísticas de tamanho de entrada e saída. As entradas ficam na ordem de uma centena de tokens, por exemplo, enquanto as saídas ficam perto de 27 tokens em média, coerentes com um JSON pequeno de três campos.

Essas estatísticas não substituem a curva de loss. Elas ajudam a entender o volume efetivamente processado, a relação entre entrada e saída e a base sobre a qual o provedor calcula a cobrança.

Também funcionam como verificação indireta. Se uma tarefa que deveria produzir respostas curtas aparecesse com saídas muito grandes, eu teria motivo para voltar ao dataset.

A configuração é contextual, não definitiva

Três épocas, learning rate multiplier 5 e adapter size 4 fazem sentido para o dataset e para a tarefa que estou treinando agora. Eles não se tornam defaults universais.

Com um dataset significativamente maior, eu provavelmente reduziria o número de épocas. Com uma tarefa mais ampla, talvez o rank do adaptador precisasse crescer.

Em um experimento comparativo, eu também poderia manter determinados valores fixos para observar o efeito de apenas uma variável.

O princípio é sempre o mesmo: hiperparâmetro precisa ser uma decisão registrada e relacionada ao experimento, não um número copiado sem contexto.

Da operação manual para automação

A próxima evolução é automatizar o fluxo que agora já possui as proteções necessárias.

O script poderá preparar e fazer upload do dataset, validar hiperparâmetros antes de enviar, criar o job, registrar os valores pedidos, consultar o que foi aplicado e monitorar sozinho as mudanças de estado até sucesso ou falha.

Essa automação só é segura porque as validações foram construídas antes. Automatizar um pipeline que confia cegamente na API apenas faria o erro silencioso acontecer com mais eficiência.

No próximo capítulo, o foco passa a ser transformar essas chamadas isoladas em um fluxo JavaScript que cria e acompanha o job sem exigir consultas manuais.

NA PRÁTICA

- Defina valores pretendidos para épocas, learning rate multiplier e adapter size, justificando a escolha.
- Valide a configuração antes da chamada de rede e compare posteriormente os valores pedidos com os efetivamente aplicados.
- Registre qualquer parâmetro silenciosamente ignorado pela API como falha de configuração a ser tratada.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-3)”?
- Qual relação existe entre Épocas e Underfitting e overfitting no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No capítulo anterior, eu executei o pipeline de upload e acompanhamento do fine-tuning gerenciado sobre 200 exemplos da Amplitude Auto e da Amplitude Saúde Empresarial.
- Agora eu quero entender o que cada hiperparâmetro controla, por que um valor pode fazer sentido para um dataset pequeno e deixar de fazer sentido quando a escala muda, e quais mecanismos preciso colocar ao redor da API para não depender cegamente da validação do provedor.
- O foco passa de conseguir criar um job para conseguir operá-lo com critérios.

[↑ Voltar ao mapa da disciplina](#)

Aula 4 — Fine-Tuning via API (upload e train) (PT-4)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-4)”.
- Relacionar Automação e Confirmação às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Projeto do módulo: automação do pipeline](#)
- [Dataset real alternativo \(Dolly-15k\)](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Automação	• Confirmação
• Idempotência	• Polling
• Backoff exponencial	• Callback
• Injeção de dependência	

Conteúdo da aula

No capítulo anterior, eu deixei prontas duas proteções importantes para o fine-tuning gerenciado. Antes de criar qualquer job, os hiperparâmetros passam por validação local. Depois que o job existe, eu comparo aquilo que foi solicitado com o que o provedor realmente aplicou. Também passei a consultar estatísticas reais do treinamento em vez de assumir que a ausência de erro significava que tudo estava correto.

Até aqui, porém, cada etapa foi executada de forma isolada, sempre com um comando específico. Agora eu junto essas peças em uma única automação em JavaScript. O objetivo é transformar conversão, validação, upload, criação de job e acompanhamento em um fluxo sequencial que consiga operar sozinho sem perder os gates de segurança que construímos até aqui.

Automatizar não significa remover todo controle humano. Pelo contrário. Quanto mais etapas um script consegue executar sem supervisão, mais importante fica decidir quais ações são seguras para rodar sozinhas e quais precisam de uma confirmação explícita antes de provocar custo ou criar um recurso real.

Os cinco passos da automação

A função de orquestração encadeia cinco passos.

Primeiro, converto os exemplos para o formato esperado pelo Gemini. Segundo, valido os hiperparâmetros. Terceiro, envio o dataset para o bucket. Quarto, crio o job de fine-tuning. Quinto, acompanho o job até que ele chegue a um estado terminal.

Os dois primeiros passos reutilizam a mesma lógica apresentada nos capítulos anteriores, agora incorporada ao script de automação. A intenção não é mudar o comportamento, mas provar que as peças continuam funcionando quando deixam de ser chamadas manualmente e passam a fazer parte de uma cadeia única.

A ordem também importa. Eu não posso criar um job sem que o dataset tenha sido enviado. Não posso fazer upload antes de converter o formato. E não faz sentido acompanhar um job que ainda não existe.

Sequência simples e falha rápida

A orquestração é propositalmente sequencial. Cada etapa espera a anterior terminar antes de continuar.

Essa estrutura já fornece uma garantia importante. Se o upload falhar por permissão incorreta, indisponibilidade de rede ou destino inválido, a execução é interrompida ali. O código não avança para a criação do job com um estado incompleto.

O mesmo vale para a validação. Se um exemplo estiver incompleto, o processo para antes dos hiperparâmetros. Se um hiperparâmetro estiver inválido, o processo para antes do upload.

Esse comportamento de fail fast evita que uma falha pequena no início do pipeline se transforme em um recurso criado de forma incorreta no final. Muitas vezes, uma função assíncrona simples e sequencial fornece uma garantia mais clara do que uma orquestração excessivamente paralela para um fluxo em que cada etapa depende da anterior.

A etapa que não deve rodar sem confirmação

Entre os cinco passos, a criação do job recebe tratamento diferente.

Converter dados, validar parâmetros, consultar estado e até repetir um upload para o mesmo destino são operações que podem ser automatizadas com risco relativamente controlado. Criar um job de fine-tuning, porém, gera um recurso cobrável.

Por isso, a função de criação só executa a chamada de rede quando recebe uma confirmação explícita igual a true nas opções.

Sem essa confirmação, o erro é lançado localmente antes de qualquer chamada ao provedor. A mensagem lembra inclusive o incidente do módulo anterior, quando um epoch count inválido foi aceito pela API e chegou a iniciar um treinamento real.

A regra é deliberada: automação de verdade não é ausência de supervisão. É colocar supervisão exatamente onde o impacto justifica.

O padrão de confirmação antes de uma ação cara

A trava utilizada aqui segue um princípio que aparece em outras ferramentas de infraestrutura.

Um exemplo conhecido é a separação entre planejar e aplicar uma mudança em ferramentas de infraestrutura como código. Primeiro eu vejo o que será feito. Depois confirmo a execução que realmente altera recursos.

Em diferentes serviços de cloud também existem modos de simulação ou dry run pelo mesmo motivo. A confirmação explícita no script de fine-tuning é uma versão mínima desse padrão aplicada à criação de um job que custa dinheiro.

Essa ideia também se conecta aos gates dos módulos anteriores. No módulo 2.1, um documento precisava provar relevância antes de entrar no dataset. No módulo 2.2, o balanceamento nunca podia alocar mais exemplos do que uma fonte possuía. Quando uma ação pode causar dano real, eu valido antes, não depois.

Idempotência parcial do pipeline

A automação também precisa considerar o que acontece quando o script é executado duas vezes.

No passo de upload, repetir a operação para o mesmo caminho substitui o objeto no destino e mantém o mesmo resultado final. Nesse sentido, o upload é idempotente para o caso apresentado.

A criação do job não possui essa mesma propriedade. Cada chamada bem-sucedida cria um novo treinamento e pode gerar nova cobrança.

É por isso que a confirmação explícita fica justamente nesse ponto. Não faria sentido exigir intervenção humana para toda etapa segura, mas seria perigoso permitir que uma nova execução acidental criasse outro job silenciosamente.

O problema do polling constante

Depois de criar o job, a automação precisa acompanhar o status até o final.

Consultar a API a cada segundo seria simples, mas ineficiente. Na maior parte do tempo, um job continua no mesmo estado e não existe informação nova para recuperar.

Eu preciso de uma estratégia que consulte com frequência suficiente para continuar responsiva, mas sem martelar o endpoint de status desnecessariamente.

A solução utilizada é o backoff exponencial.

Backoff exponencial com teto

A primeira consulta acontece imediatamente depois da criação do job. Se o estado ainda não for terminal, o script passa a esperar entre as próximas consultas.

O primeiro intervalo é de cinco segundos. Depois, cada espera é multiplicada por 1,5 até chegar a um teto de sessenta segundos.

A sequência cresce aproximadamente de 5 para 7,5, depois 11,25, 16,875 e continua aumentando até estabilizar em sessenta segundos.

O crescimento evita chamadas excessivas. O teto evita o problema contrário. Sem um limite, um job que demorasse horas poderia fazer o intervalo crescer tanto que a automação passaria muitos minutos sem perceber que o treinamento já havia terminado.

Com o teto de sessenta segundos, mesmo em um job longo, o atraso máximo para detectar o estado final fica limitado a aproximadamente um minuto.

Por que usar fator 1,5

O fator 1,5 não é o único possível.

Se eu dobrasse o intervalo em cada rodada, chegaria ao teto mais rápido e faria menos chamadas. Em compensação, ficaria mais tempo sem consultar justamente na faixa intermediária, onde um job de duração média pode terminar.

O crescimento por 1,5 é mais gradual. Ele consome algumas consultas adicionais no início, mas mantém maior responsividade até alcançar o platô de sessenta segundos.

Para os jobs desta disciplina, que levam dezenas de minutos, a maior parte do tempo de acompanhamento acaba acontecendo já nesse teto. O backoff reduz o ruído no início e depois mantém uma frequência previsível até o término.

Callback para tornar o acompanhamento observável

O loop de monitoramento utiliza também um callback.

Callback é uma função fornecida ao processo para ser chamada automaticamente quando um evento acontece. Neste caso, ela recebe atualizações conforme o script consulta o estado do job.

Isso permite registrar Pending, Running e Succeeded sem que alguém precise consultar manualmente a API.

Sem callback, o acompanhamento poderia funcionar internamente e ainda parecer travado para quem observa o terminal. A automação estaria viva, mas muda por fora.

Com atualizações explícitas, eu consigo enxergar que o processo continua consultando, qual foi o último estado recebido e quando houve uma transição relevante. Observabilidade também faz parte de uma automação confiável.

A fronteira entre execução local e Google Cloud

O script mantém uma separação clara entre a parte local e a parte que realmente toca a nuvem.

Conversão e validação rodam sem custo e sem dependência de credenciais externas.

A partir do upload, criação e acompanhamento, o código passa a interagir com o Google Cloud de verdade. É nessa região que entram autenticação, permissões, bucket, API e recursos cobráveis.

Manter essa fronteira explícita facilita testes e reduz o risco de uma alteração em lógica local disparar acidentalmente alguma ação remota.

Os dezoito testes automatizados

A demonstração executa dezoito testes automatizados divididos em grupos.

O grupo de upload verifica se o comando de cópia para o bucket é montado corretamente, se um arquivo que não é JSONL é rejeitado e se o destino precisa começar com o esquema esperado do Google Cloud Storage.

Outro grupo testa a trava de confirmação. A criação é bloqueada quando não existe confirmação, continua bloqueada com opções vazias e só é liberada quando a confirmação é explicitamente verdadeira.

Também são repetidos testes de conversão para o formato Gemini e validação de hiperparâmetros, garantindo que incorporar a mesma lógica em uma automação maior não alterou o comportamento validado anteriormente.

Testando a orquestração de ponta a ponta

Um grupo novo testa a função que coordena os cinco passos.

O teste confirma que conversão, validação, upload, criação e acompanhamento acontecem na ordem correta e que o resultado de cada etapa é devolvido pela orquestração.

Também verifica os caminhos de falha. Um exemplo incompleto interrompe o fluxo antes da validação de hiperparâmetros. Um hiperparâmetro inválido interrompe antes do upload. A confirmação continua sendo exigida na etapa de criação.

Esses testes são importantes porque uma coleção de funções corretas não garante automaticamente que a composição delas também esteja correta. O comportamento da cadeia precisa ser testado como uma unidade.

Testando o backoff

O cálculo do backoff também recebe testes próprios.

Eles confirmam que o fator de crescimento é aplicado corretamente, que o teto de sessenta segundos nunca é ultrapassado e que, depois de várias rodadas, o intervalo realmente satura no limite esperado.

Isso evita que uma mudança aparentemente pequena no cálculo transforme o monitor em um cliente agressivo demais ou, no extremo contrário, em um processo que demora muito para perceber o fim do job.

Testando lógica assíncrona sem rede nem espera real

O grupo mais importante testa o loop assíncrono de acompanhamento sem utilizar a rede e sem esperar segundos de verdade.

Para isso, a função de consulta e a função de espera são injetáveis. Durante o teste, eu substituo ambas por versões falsas e controláveis.

Um teste simula um job que já está em estado terminal na primeira consulta. Nesse caso, nenhuma espera deve acontecer.

Outro simula a sequência Pending, Running, Running e Succeeded. O loop consulta quatro vezes e espera exatamente três vezes, uma entre cada estado não terminal.

Um terceiro verifica que o callback recebe todos os estados intermediários na ordem correta, e não apenas o resultado final.

Injeção de dependência

A técnica que permite esses testes recebe aqui um nome formal: injeção de dependência.

Em vez de a função de acompanhamento estar presa a uma implementação fixa de consulta de rede e a uma espera real, essas operações entram como dependências substituíveis.

Em produção, eu forneço a função que consulta a API e a função que realmente espera. No teste, forneço versões falsas que retornam exatamente os estados desejados e concluem em milissegundos.

Sem essa separação, testar o loop exigiria conexão real com o provedor ou minutos de espera entre consultas. A suíte ficaria lenta, cara e sujeita a falhas externas.

Com a injeção de dependência, a lógica assíncrona continua sendo a mesma, mas a infraestrutura é trocada por uma versão controlável. O teste se torna rápido e determinístico.

Acompanhando o job real

Depois da suíte, a demonstração volta ao job real criado no módulo 3.2.

Como o treinamento já terminou, a primeira consulta encontra diretamente o estado Succeeded. O loop identifica que o estado é terminal e devolve o endpoint publicado sem executar qualquer espera adicional.

Se o job ainda estivesse em andamento, o mesmo código continuaria sozinho, consultando com backoff e chamando o callback até chegar ao estado final.

A diferença é que agora não preciso digitar novos comandos para cada consulta. O acompanhamento deixou de ser uma sequência manual e virou comportamento do próprio pipeline.

Automação segura antes de versionar o modelo

Ao final desta etapa, eu tenho um pipeline que converte o dataset, valida configuração, prepara o upload, cria o job somente com confirmação explícita e acompanha o treinamento até obter o endpoint final.

Ele também possui testes para caminhos felizes e de falha, proteção contra criação acidental de recursos e uma estratégia de polling que respeita a API sem perder responsividade.

O mesmo fluxo está disponível em JavaScript e também em Python com a lógica de dependências substituíveis para teste.

O endpoint produzido pelo job do módulo 3.2 agora é o artefato que segue para o próximo passo. No capítulo seguinte, ele será versionado e documentado, fechando o ciclo do fine-tuning via API e preparando a missão prática em que esse pipeline poderá ser adaptado para outro dataset e outro contexto de negócio.

NA PRÁTICA

- Modele os cinco passos da automação com falha rápida e confirmação explícita antes da ação cara.
- Implemente ou descreva polling com backoff exponencial e um callback para tornar o acompanhamento observável.
- Separe dependências externas para que a lógica assíncrona possa ser testada sem rede e sem espera real.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-4)”?
- Qual relação existe entre Automação e Confirmação no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

- No capítulo anterior, eu deixei prontas duas proteções importantes para o fine-tuning gerenciado.
- Até aqui, porém, cada etapa foi executada de forma isolada, sempre com um comando específico.
- Automatizar não significa remover todo controle humano.

[↑ Voltar ao mapa da disciplina](#)

Aula 5 — Fine-Tuning via API (upload e train) (PT-5)

Unidade 3 • Fine-Tuning via API (upload e train)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “Fine-Tuning via API (upload e train) (PT-5)”.
- Relacionar Linhagem e SHA-256 às decisões e ao fluxo apresentados na aula.
- Identificar critérios, gates, métricas ou evidências utilizados no conteúdo para validar as decisões apresentadas.

GITHUB E PRÁTICA

- [Model Cards e artefatos de versionamento do módulo](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Linhagem	• SHA-256
• Model Card	• Model Registry
• Versionamento	• Rastreabilidade
• Preference Tuning	

Conteúdo da aula

No capítulo anterior, eu fechei a automação de ponta a ponta do fine-tuning via API. O pipeline já consegue converter o dataset, validar hiperparâmetros, fazer upload, proteger a criação do job com uma confirmação explícita e acompanhar o treinamento até o endpoint ser publicado. Só que um endpoint isolado ainda diz muito pouco sobre o modelo que está em produção.

Daqui a alguns meses, alguém pode perguntar qual dataset treinou aquele endpoint, quais hiperparâmetros foram realmente aplicados, qual modelo base serviu de ponto de partida e em que momento aquele artefato passou a existir. Se eu não tiver essas respostas registradas, o modelo está funcionando, mas a operação perdeu a linhagem.

Por isso, o último passo deste módulo é versionar e documentar. Eu quero transformar um endpoint publicado em um artefato rastreável, ligado ao conteúdo exato do dataset, à configuração aplicada, ao job que gerou o modelo e à linha do tempo em que tudo aconteceu.

Por que um endpoint não é documentação

Um endpoint é apenas um identificador operacional. Ele permite chamar o modelo, mas não conta a história que levou até aquele resultado.

Em produção, isso se torna um risco. O mesmo projeto pode retreinar várias vezes com versões diferentes do dataset. Pode trocar hiperparâmetros, mudar o modelo base ou publicar um novo endpoint. Se essas relações não estiverem registradas, fica difícil explicar por que o comportamento mudou ou reproduzir um treinamento anterior.

Essa dor é uma das razões pelas quais MLOps se tornou uma disciplina própria dentro da engenharia de machine learning. Versionar código com Git é um problema conhecido há décadas. Versionar código, dataset, experimento, modelo e publicação como uma única linhagem ainda exige disciplina específica.

O que precisa ser versionado

- Para qualquer modelo fine-tunado que chegue à produção, eu quero amarrar pelo menos quatro elementos.
 - O primeiro é o dataset de treinamento, identificado pelo conteúdo e não apenas pelo nome do arquivo.
 - O segundo são os hiperparâmetros realmente aplicados. Essa distinção importa porque o módulo 3.3 já mostrou que aquilo que eu peço para a API e aquilo que o provedor registra como aplicado podem divergir.
 - O terceiro é o modelo base, ou seja, o checkpoint do provedor utilizado como ponto de partida.
 - O quarto é o resultado do treinamento: modelo ajustado, endpoint ou endpoints publicados e identificadores reais do job.
- Esses elementos já existiam espalhados pelos capítulos anteriores. Nesta etapa, eu reúno todos em um artefato único e verificável.

O modelo base também precisa de linhagem

- Registrar o modelo base é mais importante do que parece.
- Provedores aposentam versões. Um modelo que hoje aceita novos jobs pode deixar de aceitar treinamento no futuro.

O material utiliza o Gemini 2.5 Flash como exemplo do modelo base empregado ao longo do módulo e registra uma data de retirement anunciada para essa versão.

Mesmo que o modelo ajustado continue publicado, o time precisa saber qual base o originou. Isso afeta reprodutibilidade, retreino e comparação com versões futuras.

Sem esse campo, um modelo fine-tunado pode continuar funcionando enquanto a sua origem já não está mais disponível para um novo ciclo equivalente.

Nome de arquivo não é versão

O ponto mais importante desta aula está no dataset.

Um nome como dataset-final-v2.jsonl não prova o conteúdo daquele arquivo. O arquivo pode ser renomeado, movido ou sobrescrito mantendo exatamente o mesmo nome.

Se alguém corrigir um exemplo e salvar por cima do arquivo original, o caminho pode continuar idêntico enquanto o conteúdo que existe hoje já não é o mesmo que treinou o modelo publicado.

Nesse cenário, documentar apenas o nome do arquivo cria uma falsa sensação de versionamento. Eu sei onde o arquivo está, mas não sei se ele ainda representa o conteúdo usado no treinamento.

Hash SHA-256 como identificador de conteúdo

A solução utilizada é calcular um hash SHA-256 do conteúdo do dataset.

Para um mesmo arquivo, byte por byte, o resultado é sempre o mesmo valor hexadecimal de 64 caracteres. Se qualquer parte do conteúdo mudar, mesmo uma alteração pequena, o hash muda completamente.

Isso transforma o conteúdo em identificador de versão.

O princípio é semelhante ao versionamento por conteúdo utilizado em ferramentas como Git e em imagens de container. O que define a identidade não é o nome amigável, mas uma assinatura derivada do conteúdo real.

Para o dataset de 200 exemplos utilizado nos módulos anteriores, o hash fica registrado no artefato de versionamento. Enquanto aquele arquivo permanecer exatamente igual, o identificador permanece o mesmo.

Detectando alterações silenciosas no dataset

O hash resolve um cenário comum em produção.

Imagine que alguém descubra um erro de digitação em um exemplo, corrija o arquivo e faça upload novamente para o mesmo caminho do bucket.

Sem hash, um Model Card antigo pode continuar apontando para o mesmo nome de dataset e dar a impressão de que aquele conteúdo foi utilizado no treinamento original.

Com hash, a comparação é objetiva. O Model Card guarda o identificador calculado no momento do treinamento. Se o arquivo atual tiver outro hash, eu sei imediatamente que o conteúdo mudou.

Não preciso depender da memória de quem fez a alteração ou de uma convenção de nomes aplicada corretamente.

Model Card como documentação estruturada

A documentação final é organizada em um Model Card.

O termo não nasce nesta disciplina. O material faz referência ao trabalho Model Cards for Model Reporting, publicado por pesquisadores do Google, que formalizou a ideia de documentar modelos de maneira estruturada para apoiar entendimento e uso responsável.

Aqui eu uso uma versão mais enxuta e focada no pipeline que construímos.

O Model Card inclui identificação do job, modelo ajustado, endpoint, estado final, modelo base, dataset e hash, hiperparâmetros aplicados, estatísticas do dataset e linha do tempo de criação e conclusão.

A função desse documento é deixar de falar de um modelo genérico e passar a falar daquele modelo específico, treinado naquele conteúdo, daquela forma e naquele momento.

Do arquivo Markdown ao Model Registry

Para a escala desta disciplina, um arquivo Markdown é suficiente para registrar a linhagem.

Em um ambiente maior, o mesmo princípio costuma ser formalizado em um registro de modelos, como Vertex AI Model Registry, MLflow ou outra solução equivalente.

O registro automatiza parte do trabalho. Cada novo treino pode gerar uma versão, associar métricas, armazenar artefatos e apoiar promoção ou rollback entre versões publicadas.

A diferença é principalmente de automação e escala. Conceitualmente, os campos que eu estou registrando agora são os mesmos que um model registry precisa conhecer para manter a proveniência do modelo.

Rastreabilidade do dado e rastreabilidade do modelo

Esse comportamento conecta o módulo 3 ao módulo 2.

Durante a preparação de dados, eu preservei o texto OCR bruto e os metadados para conseguir rastrear a origem de um exemplo estranho.

Agora eu aplico a mesma disciplina ao modelo inteiro.

Rastrear de onde veio o exemplo e rastrear de onde veio o modelo são dois níveis da mesma prática. Se o resultado em produção estiver errado, eu preciso conseguir descer da versão publicada para o job, do job para o dataset e do dataset para os exemplos que o compõem.

O que permanece local e o que consulta a nuvem

Quase toda a lógica desta etapa pode rodar localmente.

Calcular hash, duração, custo estimado, montar a ficha de versionamento e gerar o Model Card não exige uma nova chamada ao provedor.

A única parte que toca de fato o Google Cloud é a consulta ao job já existente para recuperar os identificadores e os valores registrados pela plataforma.

Essa separação mantém o processo barato e testável. O artefato de documentação pode ser regenerado ou validado sem criar nenhum novo recurso de treinamento.

Os treze testes automatizados

A ferramenta de versionamento executa treze testes divididos em três grupos.

O primeiro grupo valida o hash de conteúdo. O mesmo arquivo calculado duas vezes precisa produzir exatamente o mesmo resultado. O valor precisa ter 64 caracteres hexadecimais e conteúdos diferentes precisam produzir hashes diferentes.

Para provar a última condição, o teste cria um arquivo temporário com conteúdo diferente e compara as assinaturas.

O segundo grupo testa a ficha de versionamento. Um job completo deve gerar uma ficha válida. Um job sem identificador obrigatório precisa ser rejeitado. A validação de completude também precisa falhar quando um campo essencial, como o endpoint, é removido propositalmente.

O terceiro grupo cobre custo e Model Card, verificando duração, presença do hash, identificadores do modelo ajustado, endpoint e dados financeiros registrados.

A ficha gerada a partir do job real

Depois dos testes, a ferramenta calcula o hash real do dataset de 200 exemplos utilizado no treinamento dos módulos 3.2 a 3.4.

Em seguida, consulta o job real e monta a ficha completa.

Ela registra o modelo base, o dataset e seu hash, epoch count igual a 3, learning rate multiplier igual a 5 e adapter size igual a 4. São os mesmos valores que o módulo 3.3 confirmou como efetivamente aplicados.

Também entram as estatísticas do treinamento, incluindo os 200 exemplos e aproximadamente 27.353 tokens cobráveis, além dos identificadores do modelo ajustado e do endpoint publicado.

A ficha não reconstrói o passado por suposição. Cada campo vem de uma decisão ou de um resultado observado nos capítulos anteriores.

Por que registrar criação e conclusão

O versionamento guarda os horários de criação e conclusão, não apenas uma duração total.

Essa diferença ajuda na investigação de incidentes.

Se o endpoint começar a se comportar de forma inesperada em determinada data, eu posso cruzar o momento em que o modelo mudou com deploys, alterações de configuração ou incidentes reportados no mesmo intervalo.

Sem timestamps absolutos, eu sei que um job durou quarenta e poucos minutos, mas não consigo posicionar essa mudança na linha do tempo operacional do sistema.

Custo estimado e custo real

A ficha também separa custo estimado de custo real.

O custo estimado utiliza uma faixa de referência associada a GPU para permitir comparação com o caminho local que será explorado no módulo 4.

Ele não representa a fatura do Vertex AI, porque o serviço gerenciado utilizado aqui cobra o treinamento por tokens e não simplesmente por hora de GPU.

Por isso, a ficha também registra o custo efetivamente verificado no billing do Google Cloud para o job, no exemplo apresentado, R\$2,39.

Manter os dois valores lado a lado ajuda a comparar alternativas sem confundir estimativa de infraestrutura própria com cobrança real do serviço gerenciado.

O Model Card como artefato do pipeline

Ao final da execução, o Model Card é gerado como um arquivo real.

Ele pode acompanhar o modelo no repositório, na documentação da equipe ou ser usado como base para um registro de modelos mais formal.

O mesmo dataset produz o mesmo hash independentemente de eu calcular a assinatura em JavaScript ou Python. O versionamento por conteúdo depende do arquivo, não da linguagem usada para processá-lo.

Isso permite transportar a prática para qualquer ambiente de treinamento sem alterar o princípio.

A missão prática que fecha o módulo 3

A missão prática deste módulo reúne tudo o que foi construído até aqui.

Primeiro, eu converto o dataset da missão anterior, ou outro dataset do meu contexto, para o formato esperado pelo provedor escolhido.

Segundo, valido os hiperparâmetros antes da chamada e comparo o que foi solicitado com o que realmente foi aplicado depois da criação do job.

Terceiro, implemento acompanhamento com backoff exponencial em JavaScript e incluo pelo menos um teste automatizado que não dependa de rede, utilizando injeção de dependência.

Quarto, gero uma ficha de versionamento com hash do dataset e os campos mínimos de um Model Card.

A entrega não exige obrigatoriamente gastar dinheiro com um job real. Se o orçamento não permitir, é válido simular a criação e documentar exatamente o que teria sido enviado, incluindo hiperparâmetros, hash e custo estimado.

Também é possível analisar por escrito jobs já publicados na disciplina, comparando configurações e resultados sem iniciar novo treinamento.

Rastreabilidade antes mesmo de gastar

Documentar uma decisão de não executar também faz parte da rastreabilidade.

Se eu registro qual dataset seria utilizado, qual é o hash, quais hiperparâmetros seriam enviados e qual custo estimo, outra pessoa consegue entender posteriormente qual decisão foi considerada e por que ela não avançou.

Rastreabilidade não serve apenas para explicar o que já aconteceu. Ela também serve para registrar de forma verificável o que estava planejado antes de uma ação cara ou irreversível.

O que continua valendo no treinamento local

No próximo módulo, a infraestrutura muda. O treino sai da API gerenciada e entra na máquina local com técnicas de Parameter Efficient Fine-Tuning.

O rigor de engenharia, porém, não muda.

O esquema canônico do módulo 2 continua útil. As validações antes de agir continuam úteis. Os testes automatizados continuam úteis. E o versionamento por conteúdo continua funcionando da mesma forma.

Quando o treinamento local gerar um adaptador, como um arquivo SafeTensor, esse artefato também pode receber um hash SHA-256. O princípio não depende de nuvem. Ele depende apenas do conteúdo que eu quero identificar de maneira verificável.

Um segundo caminho: Preference Tuning

Antes de fechar o módulo, a aula apresenta um segundo caminho disponível no Vertex AI além do Supervised Fine-Tuning utilizado até aqui: Preference Tuning.

No treinamento supervisionado, cada entrada possui uma resposta correta que funciona como gabarito. É exatamente o cenário da extração estruturada da Amplitude, em que eu quero um conjunto de campos específicos.

No Preference Tuning, a estrutura muda. Para cada prompt, eu forneço duas respostas: uma preferida e uma rejeitada. O modelo aprende a aproximar o comportamento da resposta preferida e a se afastar da alternativa rejeitada.

O material relaciona esse caminho a DPO, Direct Preference Optimization. Diferentemente do RLHF clássico, que envolve um modelo de recompensa separado e depois uma etapa de reinforcement learning, DPO otimiza diretamente a preferência entre pares de respostas.

O teste real de Preference Tuning

A técnica também é testada de verdade na disciplina.

Quarenta exemplos do módulo 3.2 são convertidos em pares de preferência. De um lado fica a extração correta. Do outro, uma resposta gerada a partir de um prompt mais fraco, sem a exigência rígida de JSON.

O job de Preference Tuning é criado no Vertex AI usando o Gemini 2.5 Flash e termina com sucesso em aproximadamente 17 minutos e 32 segundos, publicando modelo ajustado e endpoint.

O tempo menor não deve ser interpretado isoladamente como prova de superioridade. O dataset desse experimento é muito menor, com 40 exemplos, enquanto o treinamento supervisionado principal utilizou 200.

Onde Preference Tuning faz mais sentido

A própria natureza do caso ajuda a decidir qual técnica é mais adequada.

Extração de campos estruturados possui uma resposta objetiva. Para esse tipo de tarefa, Supervised Fine-Tuning é o habitat natural.

Preference Tuning ganha relevância quando qualidade é subjetiva e existe mais de uma resposta plausível, mas uma delas é preferível.

A aula cita situações como tom, voz de marca, linguagem de compliance e comunicação com cliente.

Em uma seguradora, isso poderia aparecer em comunicado de sinistro, resposta de atendimento ou linguagem utilizada em uma negativa de cobertura. Nesses casos, o jeito de responder pode importar tanto quanto o conteúdo factual.

O módulo termina, portanto, com duas ideias conectadas. Primeiro, qualquer modelo treinado precisa carregar linhagem suficiente para ser reproduzido e auditado. Segundo, escolher fine-tuning não encerra a decisão técnica. O tipo de ajuste continua dependente da natureza da tarefa e do tipo de supervisão disponível.

NA PRÁTICA

- Calcule um hash SHA-256 do conteúdo do dataset e use-o como identificador de versão.
- Monte um Model Card com linhagem do modelo base, dataset, configuração, datas, custo e resultado do treinamento.
- Explique como o artefato poderá ser registrado em um Model Registry sem depender apenas de nomes de arquivo.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “Fine-Tuning via API (upload e train) (PT-5)”?
- Qual relação existe entre Linhagem e SHA-256 no conteúdo apresentado?
- Que evidência, gate, métrica ou registro você verificaria para confirmar que a decisão ou o pipeline descrito na aula está funcionando como esperado?

RESUMO DA AULA

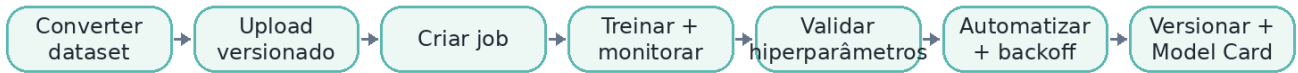
- No capítulo anterior, eu fechei a automação de ponta a ponta do fine-tuning via API.
- Daqui a alguns meses, alguém pode perguntar qual dataset treinou aquele endpoint, quais hiperparâmetros foram realmente aplicados, qual modelo base serviu de ponto de partida e em que momento aquele artefato passou a existir.
- Por isso, o último passo deste módulo é versionar e documentar.

[↑ Voltar ao mapa da disciplina](#)

Revisão da Unidade 3

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de decisão, preparação ou operação.

FLUXO VISUAL DA UNIDADE



CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Fine-Tuning via API \(upload e train\) \(PT-1\)](#)
- [Revisar a Aula 2: Fine-Tuning via API \(upload e train\) \(PT-2\)](#)
- [Revisar a Aula 3: Fine-Tuning via API \(upload e train\) \(PT-3\)](#)
- [Revisar a Aula 4: Fine-Tuning via API \(upload e train\) \(PT-4\)](#)
- [Revisar a Aula 5: Fine-Tuning via API \(upload e train\) \(PT-5\)](#)

[↑ Voltar ao mapa da disciplina](#)

Unidade 4

LoRA e PEFT (Conceitos Práticos)

Ajuste eficiente após a aprovação do fine-tuning, comparando custo, treinamento local, rank, qualidade, memória e Full Fine-Tuning com base em medições apresentadas nas aulas.

GITHUB E PRÁTICA

- [Abrir pasta do Módulo 4 — LoRA e PEFT](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

FLUXO VISUAL DA UNIDADE



Aula	Tema
1	LoRA e PEFT (Conceitos Práticos) (PT-1)
2	LoRA e PEFT (Conceitos Práticos) (PT-2)
3	LoRA e PEFT (Conceitos Práticos) (PT-3)
4	LoRA e PEFT (Conceitos Práticos) (PT-4)

Aula 1 — LoRA e PEFT (Conceitos Práticos) (PT-1)

Unidade 4 • LoRA e PEFT (Conceitos Práticos)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “LoRA e PEFT (Conceitos Práticos) (PT-1)”.
- Relacionar LoRA e custo fixo às decisões, comparações e evidências apresentadas na aula.
- Identificar métricas, custos, hiperparâmetros ou resultados utilizados no conteúdo para sustentar a decisão técnica.

GITHUB E PRÁTICA

• [Projeto do módulo: fundamentos de LoRA e PEFT](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• LoRA	• PEFT
• Low-Rank Adaptation	• NPV
• Custo fixo	• Adapters

Conteúdo da aula

No módulo anterior, nós fechamos o fluxo de fine-tuning com um endpoint real, publicado e versionado. Na demonstração, o ajuste foi feito com o Gemini 2.5 Flash, utilizando 200 exemplos dos contextos da Amplitude Auto e da Amplitude Saúde Empresarial, com execução na nuvem por meio do Vertex AI. O resultado funcionou bem e o módulo terminou mostrando também como esse artefato deveria ser versionado.

Só que ficou uma pergunta em aberto. Nem todo caso real cabe bem em um job gerenciado na nuvem. O ferramental financeiro que já construímos no módulo 1.3 permite enxergar exatamente esse limite. Então, agora eu quero discutir o que acontece quando o caso de negócio continua sendo um bom candidato a fine-tuning, mas o volume é pequeno demais para justificar um custo fixo elevado de treinamento.

É nesse ponto que entram LoRA e PEFT.

O problema não é decidir novamente se vale fazer fine-tuning

É importante separar duas decisões.

A primeira decisão já foi resolvida pelo nosso GATE. Nós avaliamos se a tarefa possui formato estável, se existe padrão que pode ser aprendido, se temos dados adequados e se fine-tuning realmente é a opção correta.

LoRA não substitui esse processo.

Se o caso fosse semelhante ao exemplo de atendimento ao cliente que reprovamos no módulo 1.3 por ser uma tarefa aberta e instável, fazer LoRA local não resolveria nada. O problema ali não era o custo de treinamento. O problema era a própria tarefa não apresentar um formato suficientemente fixo para ser aprendido.

LoRA entra depois que o GATE já disse sim.

A pergunta passa a ser outra: qual é a forma economicamente mais adequada de executar esse ajuste?

O custo fixo começa a pesar em operações menores

No caso nacional da Amplitude, o volume era grande. O exemplo trabalhava com algo na ordem de milhares de sinistros por mês. Nesse cenário, um custo fixo de treinamento se dilui ao longo de muitas chamadas e de uma economia recorrente relevante.

Agora imagine a expansão para parcerias regionais, corretoras menores no Sul e no Nordeste, cada uma gerando seus próprios orçamentos de oficina.

A tarefa continua sendo estruturalmente a mesma: extrair segurado, placa e valor.

O que muda é o volume.

Em vez de milhares de documentos por mês, algumas parcerias geram apenas algumas centenas.

Nesse cenário, pagar um custo fixo de GPU na nuvem para cada parceria pode destruir a viabilidade econômica do projeto.

Reutilizando o mesmo framework financeiro

A aula retoma exatamente o ferramental financeiro do módulo 1.3.

A lógica continua sendo aquela das quatro perguntas, dos pesos obtidos via AHP e do cálculo real de NPV para verificar se o investimento se paga.

Não estamos inventando uma nova ferramenta financeira.

A demonstração reaproveita o mesmo cálculo de NPV e compara dois caminhos para uma parceria regional de 400 documentos por mês.

No primeiro caminho, considera-se uma GPU alugada na nuvem com custo fixo ilustrativo de R\$2.400 por treinamento.

Nesse cenário, o NPV em 24 meses fica em aproximadamente menos R\$2.041 e o break-even não é atingido dentro do horizonte.

O volume é pequeno demais para diluir o custo fixo.

A demonstração compara os dois caminhos com a mesma lógica

A demonstração usa um script de comparação entre LoRA regional e treinamento em nuvem e executa cinco testes automatizados.

O ponto técnico mais importante é que ele não duplica a lógica financeira. O cálculo de NPV é reaproveitado diretamente da ferramenta de Decision Framework construída anteriormente, mantendo o mesmo custo por chamada e alterando apenas o formato do custo fixo de treinamento.

Isso é importante porque garante uma comparação justa. Não estamos mudando as premissas para favorecer a LoRA. Estamos submetendo o mesmo caso regional às duas alternativas de infraestrutura.

LoRA local muda a estrutura do custo

No segundo caminho, o treinamento é feito localmente com LoRA.

Se o hardware já existe, o custo marginal de treino fica muito próximo do tempo de máquina.

No exemplo da aula, o mesmo caso regional passa a apresentar NPV positivo de aproximadamente R\$359, com break-even já no primeiro mês.

O caso de negócio não mudou.

O custo por chamada não mudou.

O volume não mudou.

O que mudou foi a estrutura do custo fixo da unidade de treinamento.

Essa diferença fica ainda mais forte quando pensamos em várias parcerias.

Cinco parcerias regionais com um custo fixo de R\$2.400 cada significariam R\$12 mil em treinamento. Individualmente, nenhuma chegaria ao break-even no horizonte considerado.

Com adaptadores LoRA locais, o custo fixo adicional fica próximo de zero e cada parceria pode ter retorno positivo desde o início.

O valor de R\$ 2.400 é um exemplo do case

A aula faz uma ressalva importante sobre esse número.

Os R\$2.400 não representam uma cotação universal de mercado.

São um valor ilustrativo utilizado no case da Amplitude Seguros.

Para uma análise real, a conta deveria usar o preço atual da GPU multiplicado pela quantidade real de horas de treinamento.

O material cita, como referência temporal de setembro de 2026, valores de GPU de perfil consumidor em serviços de aluguel e reforça que a decisão correta é substituir o custo hipotético pelo preço efetivamente encontrado no momento do projeto.

O ponto do exercício não é decorar R\$2.400.

É entender como um custo fixo por job altera o NPV quando o volume é pequeno.

O que LoRA realmente faz

LoRA significa Low-Rank Adaptation.

A ideia central é não atualizar a matriz de pesos original do modelo durante o fine-tuning.

Em um fine-tuning completo, cada camada possui uma matriz de pesos e os gradientes atualizam uma quantidade enorme de parâmetros.

Em modelos com bilhões de parâmetros, isso exige muita memória, tempo de processamento e armazenamento.

Além disso, cada checkpoint ajustado tende a ter novamente o tamanho do modelo inteiro.

LoRA muda esse processo.

Os pesos originais permanecem congelados e o treinamento acontece apenas em pequenas matrizes adicionais.

O job gerenciado anterior já utilizava uma lógica de adapter

A aula também esclarece um detalhe importante sobre o módulo anterior.

O job gerenciado executado no Vertex AI não correspondia a um Full Fine Tuning clássico de todos os pesos.

Ele já utilizava uma estratégia baseada em adapter.

O campo `adapterSize` que aparecia no Model Card estava relacionado justamente a esse tipo de ajuste eficiente.

O Full Fine Tuning de verdade será tratado depois, quando o curso comparar diretamente o custo e os limites de memória de atualizar todo o modelo.

Essa distinção ajuda a perceber que a ideia de ajustar poucos parâmetros não está restrita ao treinamento local.

Ela também aparece por baixo de ferramentas gerenciadas.

A intuição do posto baixo

LoRA parte de uma observação apresentada por Hu e colegas, inicialmente em 2021 e depois formalizada no contexto do ICLR de 2022.

A ideia é que a mudança necessária nos pesos para adaptar um grande modelo a uma tarefa específica pode ser representada por uma estrutura de posto baixo.

Para entender isso, pense em uma matriz em que uma linha seja apenas uma multiplicação da outra.

Se a primeira linha possui os valores 1, 2 e 3, a segunda é 10, 20 e 30 e a terceira é 100, 200 e 300.

Apesar de existirem três linhas, elas não carregam três informações independentes.

Existe essencialmente uma única direção de informação, repetida em escalas diferentes.

Essa matriz possui posto 1.

A atualização não precisa usar toda a liberdade da matriz

Matrizes reais não são necessariamente tão redundantes quanto esse exemplo.

O ponto observado no trabalho de LoRA é que a atualização necessária durante o fine-tuning tende a possuir muito mais redundância do que uma matriz arbitrária.

Isso significa que não precisamos aprender livremente uma atualização completa para cada elemento da matriz original.

Podemos aproximar essa mudança utilizando duas matrizes menores.

Em vez de mexer diretamente em todos os pesos de uma camada, treinamos apenas essas estruturas reduzidas.

A matemática conceitual de LoRA

Considere uma matriz original de pesos W .

Durante LoRA, W permanece congelada.

A atualização é representada pelo produto de duas matrizes menores, normalmente chamadas A e B , com um posto R muito menor do que as dimensões completas de W .

A saída da camada passa a considerar o comportamento original de W somado à contribuição produzida por essas matrizes menores, com um fator de escala relacionado a α e R .

Durante o treinamento, o gradiente flui apenas para A e B .

Todo o conhecimento que já estava armazenado em W permanece intocado.

O treinamento aprende somente o pequeno ajuste necessário para especializar o comportamento.

Um exemplo numérico da economia de parâmetros

Imagine que W tenha 1.000 linhas e 1.000 colunas.

Essa matriz possui 1 milhão de números.

Se utilizarmos o posto 4, a atualização pode ser representada por uma matriz de 1.000 por 4 e outra de 4 por 1.000.

No total, temos 8 mil números treináveis em vez de 1 milhão.

É menos de 1% do tamanho da matriz original.

A intuição é que a tarefa específica não precisa reescrever todo o conhecimento de mundo do modelo.

Extrair seguro, placa e valor de um orçamento de oficina exige um ajuste direcionado de comportamento, não uma reconstrução completa de tudo que o modelo sabe.

O piloto real da disciplina

A aula antecipa os números do piloto que será executado no próximo vídeo.

Nesse experimento local com o modelo Gemma utilizado na disciplina, o adaptador LoRA teve aproximadamente 6,8 milhões de parâmetros treináveis diante de cerca de 4,6286 bilhões de parâmetros totais.

Isso representa aproximadamente 0,147% do modelo.

Menos de 0,2% dos pesos participam efetivamente do treinamento.

O restante permanece congelado, exatamente como veio do modelo base.

Essa diferença é o que torna possível treinar em hardware de consumidor, em poucos minutos, usando muito menos memória do que um Full Fine Tuning.

O ganho também aparece no armazenamento

A diferença não está apenas no custo de computação.

Existe também armazenamento.

Em Full Fine Tuning, cada nova tarefa ou parceria gera um checkpoint completo do modelo.

Se tivermos cinco parcerias regionais, poderíamos precisar armazenar cinco cópias completas.

Com LoRA, o modelo base é compartilhado.

Cada parceria armazena apenas seu adaptador.

A aula trabalha com um modelo base de aproximadamente 10,24 GB. Cinco cópias completas ultrapassariam 51 GB apenas em checkpoints, enquanto com LoRA teríamos uma única cópia do modelo base somada a cinco adaptadores de poucas dezenas de megabytes.

Quanto maior o número de especializações, maior a diferença.

LoRA faz parte de uma família maior chamada PEFT

LoRA não é uma categoria isolada.

Ela pertence a uma família chamada PEFT, Parameter Efficient Fine Tuning.

PEFT reúne técnicas que ajustam modelos grandes treinando apenas uma pequena fração de parâmetros novos ou existentes, em vez de atualizar o modelo inteiro.

A aula organiza essas abordagens em quatro grandes famílias: métodos de adição, métodos seletivos, soft prompts e reparametrização.

LoRA pertence à família de reparametrização.

Métodos de adição

Métodos de adição inserem módulos novos entre camadas existentes.

Adapters são o exemplo mais conhecido.

Esses módulos normalmente possuem formato de bottleneck: reduzem a dimensão, aplicam uma transformação e depois retornam à dimensão original.

Somente essa estrutura adicional é treinada.

Os pesos do modelo base ficam congelados.

A aula cita trabalhos de adapters anteriores ao próprio LoRA, mostrando resultados próximos ao fine-tuning completo com uma fração pequena de parâmetros adicionais por tarefa.

Métodos seletivos

Nos métodos seletivos, não adicionamos necessariamente novos módulos.

Em vez disso, escolhemos uma pequena parte dos parâmetros que já existem e liberamos apenas essa fatia para treinamento.

BitFit é o exemplo destacado.

A técnica treina somente os termos de viés das camadas e congela o restante, inclusive grandes matrizes de atenção.

No material, a proporção treinável fica abaixo de 0,1% dos parâmetros em variantes de BERT avaliadas pelos autores.

É uma abordagem muito simples de implementar porque o modelo não precisa receber um novo componente estrutural.

Soft Prompts, Prefix Tuning e Prompt Tuning

Outra família trabalha com vetores contínuos aprendidos.

Prefix Tuning adiciona vetores treináveis às camadas de atenção sem alterar os pesos originais do modelo.

Prompt Tuning faz algo semelhante principalmente na entrada.

Esses vetores não são palavras escolhidas de um vocabulário.

São pontos aprendidos por gradiente no mesmo espaço de embeddings em que os tokens vivem.

Por isso são chamados de soft prompts.

A técnica aprende uma instrução contínua sem precisar representar essa instrução como texto legível por uma pessoa.

QLoRA

A aula também retoma QLoRA, que já havia aparecido no cheat sheet do módulo 1.3.

QLoRA combina LoRA com um modelo base quantizado, por exemplo em 4 bits.

A quantização reduz ainda mais o consumo de memória e permite trabalhar com modelos maiores em uma única GPU de consumidor.

A lógica de atualização continua sendo LoRA.

O que muda é a forma compacta como o modelo base é armazenado e carregado durante o treinamento.

Um caso de mercado da mesma família

O cheat sheet anterior também trouxe um exemplo aplicado dessa lógica de eficiência.

A aula cita um caso de ajuste de um Llama de 8 bilhões de parâmetros para classificar registros de verificação de antecedentes. Nos casos mais difíceis, correspondentes a aproximadamente 2% do conjunto, o sistema chegou a cerca de 90% de acurácia, enquanto a acurácia geral ficou em torno de 97%.

Na comparação apresentada, a solução foi aproximadamente cinco vezes mais barata e trinta vezes mais rápida do que a alternativa anterior baseada em GPT-4.

Esse tipo de resultado resume o objetivo da família PEFT: buscar qualidade próxima de um Full Fine Tuning, mas com uma fração muito menor do custo computacional.

Por que LoRA ganhou tanta adoção

A vantagem de LoRA não é apenas acadêmica.

Existe uma razão de engenharia de sistema para sua adoção.

Prefix Tuning e Prompt Tuning consomem parte da janela de contexto porque adicionam representações ao fluxo de entrada.

Adapters adicionam novas etapas sequenciais e podem aumentar a latência de inferência.

LoRA evita os dois problemas.

Depois do treinamento, a atualização aprendida pode ser incorporada diretamente à matriz original.

O merge elimina custo extra de inferência

Depois do treino, a contribuição de B vezes A pode ser somada a W e formar uma única matriz final.

Esse merge é calculado uma vez.

Em produção, o servidor não precisa executar as duas matrizes extras separadamente em cada requisição.

Do ponto de vista da inferência, a camada funciona como uma única matriz já fundida.

O custo adicional das estruturas LoRA existe durante o treinamento.

Depois do merge, ele não precisa adicionar latência à resposta do modelo.

Esse detalhe de engenharia é uma das razões práticas para LoRA ter se tornado um padrão de produção em ajustes leves.

Biblioteca PEFT e próximos passos

A biblioteca PEFT da Hugging Face reúne várias dessas técnicas por meio de uma interface comum.

Isso reduz muito a barreira de implementação porque não precisamos construir manualmente toda a decomposição de posto baixo.

Mas a decisão conceitual continua sendo nossa.

PEFT não muda a diferença entre fine-tuning e RAG discutida no início da disciplina.

Se o problema pede RAG, LoRA não transforma fine-tuning na resposta correta.

PEFT apenas reduz o custo de exercer a opção de fine-tuning quando ela já foi aprovada pelo problema e pelo GATE.

No próximo vídeo, essa teoria passa para um treinamento LoRA local completo em Apple Silicon, usando MLX-LM e o modelo Gemma do piloto. A aula antecipa uma validation loss caindo de 4,752 para 0,895 em 20 iterações.

Depois disso, o módulo avança para o papel do posto R , mostrando o trade-off entre ranks menores e maiores, com impacto em memória, tempo de treinamento e qualidade. Por fim, a sequência compara Full Fine Tuning e LoRA lado a lado na mesma tarefa, incluindo os limites reais de memória que aparecem em uma máquina convencional.

NA PRÁTICA

- Compare, para um caso de baixo volume, o mesmo raciocínio financeiro utilizado na aula entre treinamento gerenciado e LoRA local.
- Registre o custo fixo utilizado no cenário e refaça o cálculo com uma premissa adequada ao seu contexto.
- Explique por que a decisão sobre LoRA acontece depois que o caso já passou pelo GATE de fine-tuning.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “LoRA e PEFT (Conceitos Práticos) (PT-1)”?
- Qual relação existe entre LoRA e custo fixo no conteúdo apresentado?
- Que métrica, comparação ou evidência você verificaria para confirmar a conclusão técnica apresentada na aula?

RESUMO DA AULA

- No módulo anterior, nós fechamos o fluxo de fine-tuning com um endpoint real, publicado e versionado. Na demonstração, o ajuste foi feito com o Gemini 2.5 Flash, utilizando 200 exemplos dos contextos da Amplitude Auto e da Amplitude Saúde Empresarial, com execução na nuvem por meio do Vertex AI. O resultado funcionou bem e o módulo terminou mostrando também como esse artefato deveria ser versionado.
- Só que ficou uma pergunta em aberto. Nem todo caso real cabe bem em um job gerenciado na nuvem. O ferramental financeiro que já construímos no módulo 1.3 permite enxergar exatamente esse limite. Então, agora eu quero discutir o que acontece quando o caso de negócio continua sendo um bom candidato a fine-tuning, mas o volume é pequeno demais para justificar um custo fixo elevado de treinamento.
- É nesse ponto que entram LoRA e PEFT.

[↑ Voltar ao mapa da disciplina](#)

Aula 2 — LoRA e PEFT (Conceitos Práticos) (PT-2)

Unidade 4 • LoRA e PEFT (Conceitos Práticos)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “LoRA e PEFT (Conceitos Práticos) (PT-2)”.
- Relacionar LoRA local e validation loss às decisões, comparações e evidências apresentadas na aula.
- Identificar métricas, custos, hiperparâmetros ou resultados utilizados no conteúdo para sustentar a decisão técnica.

GITHUB E PRÁTICA

• [Projeto do módulo: treinamento LoRA local](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• LoRA local	• Validation loss
• Treino/validação/teste	• MLX
• Hiperparâmetros	• Vazamento de dados

Conteúdo da aula

No vídeo anterior, nós ficamos principalmente nos princípios. Entendemos por que um custo fixo de GPU alugada pode deixar de fazer sentido em casos de baixo volume e vimos matematicamente como LoRA contorna esse problema ao congelar a matriz original do modelo e treinar apenas um conjunto pequeno de parâmetros adicionais.

Agora a discussão sai do papel.

Nesta aula, nós executamos um treinamento local de verdade, sem nenhuma chamada de nuvem durante o processo principal. O mesmo dataset real de 200 exemplos que já havia sido utilizado no treinamento gerenciado do Vertex AI é reaproveitado em um fluxo local. Primeiro, ele precisa ser convertido para o formato esperado pelo framework de treinamento. Depois, os hiperparâmetros são validados, o processo é orquestrado, as métricas são capturadas em tempo real e, por fim, comparamos o modelo base com o modelo carregado com o adaptador LoRA.

A intenção é provar duas coisas ao mesmo tempo: que o treinamento realmente aconteceu e que ele alterou de forma útil o comportamento do modelo.

O mesmo dataset, dois caminhos de treinamento

O dataset usado nesta aula não é um conjunto sintético criado apenas para demonstrar LoRA.

São os mesmos 200 exemplos reais trabalhados anteriormente, com 120 exemplos do contexto da Amplitude Auto e 80 da Saúde Empresarial.

No módulo anterior, esse conjunto foi enviado para um processo gerenciado no Vertex AI.

Agora ele é utilizado para um treinamento local.

Isso torna a comparação mais justa porque os dados permanecem os mesmos. O que muda é a infraestrutura e a forma de executar o ajuste.

No caminho gerenciado, temos abstração, conveniência e menos visibilidade sobre o processo interno.

No caminho local, assumimos mais responsabilidade técnica, mas enxergamos diretamente parâmetros treináveis, consumo de memória, taxa de processamento e comportamento da curva de aprendizado.

O hardware utilizado na demonstração

A execução mostrada na aula foi feita em uma máquina com Apple M5 Pro e 24 GB de memória unificada.

O modelo já estava disponível em cache local.

Por isso, o tempo necessário para baixar aproximadamente 10,24 GB de modelo não aparece durante a gravação.

Em uma primeira execução, esse download precisa ser considerado.

A aula menciona também a possibilidade de utilizar um token gratuito do Hugging Face para facilitar o acesso ao modelo e potencialmente melhorar o processo de download.

Esse detalhe é importante porque o tempo de preparação também faz parte da experiência de treinamento local.

Validation loss como métrica central

A principal métrica observada neste vídeo é validation loss.

Validation loss representa o erro do modelo medido em um conjunto de dados reservado para validação.

Esses exemplos não entram diretamente no processo de aprendizado.

Eles servem para avaliar como o modelo está se comportando fora dos exemplos que está vendo durante o treino.

Enquanto o modelo aprende um padrão útil, esperamos observar redução desta perda.

Quando ela estabiliza, pode indicar que o ganho adicional de treinamento está ficando pequeno.

Quando começa a subir depois de atingir um mínimo, pode ser um indício de overfitting, ou seja, o modelo começa a se ajustar demais ao conjunto de treino e perde capacidade de generalização.

Preparação e testes antes do treinamento real

A primeira parte da demonstração trabalha com uma ferramenta local responsável por validar todo o processo antes de disparar o treino de verdade.

A aula apresenta 16 testes automatizados distribuídos em seis grupos.

O primeiro grupo trata da conversão do dataset.

Um exemplo com turno do usuário e resposta do modelo precisa ser transformado no formato de mensagens esperado pelo framework local.

Um exemplo incompleto deve ser rejeitado antes de chegar ao treinamento.

Essa validação evita gastar processamento com um dataset estruturalmente inválido.

Divisão entre treino, validação e teste

Os 200 exemplos são divididos em três partes.

Na demonstração, ficam 157 exemplos para treino, 30 para validação e 13 para teste.

A validação recebe aproximadamente 15% dos dados e o teste cerca de 5%.

Essa proporção tenta equilibrar duas necessidades.

O conjunto de validação precisa ter exemplos suficientes para tornar a leitura do validation loss minimamente estável.

Ao mesmo tempo, não pode retirar dados demais do treinamento, já que o dataset original ainda é pequeno.

O conjunto de teste fica menor porque, nesta etapa, sua função principal é permitir uma comparação concreta entre o comportamento antes e depois do adaptador.

Ele não está sendo utilizado como uma estimativa estatística definitiva de acurácia.

Evitando vazamento entre as divisões

Outro cuidado é garantir que uma mesma entidade não apareça em conjuntos diferentes.

Se informações da mesma entidade forem parar em treino e teste, por exemplo, podemos criar uma avaliação artificialmente fácil.

Por isso, a divisão é feita de maneira agrupada e determinística.

Os testes verificam que nenhuma entidade fica partida entre os splits e que a divisão permanece reprodutível.

Esse é o mesmo tipo de disciplina que já apareceu em outros módulos: antes de confiar em uma métrica, precisamos garantir que o processo que gerou essa métrica é válido.

O esquema canônico facilita a migração entre ferramentas

A conversão entre Vertex AI e o framework local é relativamente simples porque o esquema canônico definido anteriormente foi pensado para ser genérico.

No fundo, os dois formatos carregam a mesma informação.

Existe uma instrução ou entrada do usuário e uma resposta esperada do modelo.

O Vertex AI representa isso com seus próprios campos.

O framework local utiliza uma estrutura de mensagens muito próxima do padrão de chat que se tornou comum no mercado.

A superfície muda.

A informação semântica continua a mesma.

Esse é um dos benefícios de manter um dataset canônico independente do provedor.

Validação dos hiperparâmetros

A aula reutiliza a disciplina que já apareceu no módulo de fine-tuning via API.

Antes de criar qualquer processo, os hiperparâmetros precisam ser validados.

São verificados itens como número de iterações, learning rate, tipo de fine-tuning e batch size.

Configurações inválidas são rejeitadas antes do treinamento.

Isso evita descobrir um erro apenas depois que o modelo já foi carregado ou que a GPU já começou a trabalhar.

A regra continua sendo a mesma: a configuração é parte do contrato do treinamento e deve ser validada antes da execução.

Extração das métricas e orquestração

Outro grupo de testes valida o parser responsável por ler as linhas de validation loss produzidas pelo processo real.

Antes de confiar nele contra um treinamento verdadeiro, a lógica é testada com saídas simuladas.

A orquestração do processo também é testada separadamente.

Um processo falso é injetado no lugar do framework real para verificar se o código reconhece corretamente uma execução bem-sucedida e se rejeita códigos de erro.

Isso permite validar a lógica de integração sem precisar gastar tempo de GPU a cada teste.

JavaScript orquestrando um ecossistema dominado por Python

Existe um ponto técnico importante na arquitetura.

O framework local utilizado na demonstração não possui binding nativo para Node.js.

Por isso, o JavaScript não chama uma função de treinamento diretamente.

Ele monta a configuração e dispara um processo externo equivalente por linha de comando.

Essa escolha reflete a realidade do ecossistema atual.

O JavaScript funciona muito bem para integração com APIs gerenciadas.

Quando entramos diretamente em treinamento, quantização e manipulação de tensores, Python continua dominando por meio de ferramentas como PyTorch, Hugging Face e MLX-LM.

São problemas diferentes e, portanto, ferramentas diferentes.

Configuração real do treinamento

A configuração utilizada na demonstração mantém o mesmo modelo base do piloto da disciplina e usa fine-tuning do tipo LoRA.

O treino mostrado na tela é executado por 20 iterações, com batch size 1 e learning rate de 1 por 100.000.

A aula reforça que esse learning rate é específico da configuração usada no MLX-LM.

Ele não deve ser copiado cegamente para outro framework.

Cada stack possui convenções, defaults e faixas adequadas.

O importante é compreender a função do hiperparâmetro, não decorar um único número.

A proporção de parâmetros treináveis aparece ao vivo

Assim que o treinamento começa, aparece uma informação que o processo gerenciado escondia.

São aproximadamente 6,8 milhões de parâmetros treináveis em um modelo com cerca de 4,6286 bilhões de parâmetros totais.

Isso corresponde a aproximadamente 0,147%.

É a mesma ordem de grandeza apresentada conceitualmente no vídeo anterior, agora observada diretamente durante um processo real.

A maioria absoluta dos parâmetros permanece congelada.

Só o adaptador participa do ajuste.

Essa visibilidade é uma das grandes diferenças entre treinamento local e treinamento gerenciado.

Nuvem gerenciada e treinamento local representam trocas diferentes

No Vertex AI, vários detalhes internos são abstraídos.

Vemos os hiperparâmetros principais e o resultado final, mas não acompanhamos necessariamente cada aspecto do processo.

No treinamento local, temos acesso direto a praticamente tudo.

Isso representa uma troca.

A nuvem gerenciada oferece conveniência em troca de menos controle.

O ambiente local exige mais responsabilidade em troca de visibilidade total.

Nenhum dos dois caminhos é universalmente melhor.

A decisão depende de custo, volume, infraestrutura e necessidade operacional.

Uma versão equivalente por API

A aula também apresenta uma ferramenta complementar que monta a mesma ideia de configuração LoRA como corpo de uma requisição HTTP para uma API de fine-tuning.

Nesse exemplo, aparecem parâmetros como rank 8, escala 20 e dropout 0.

A requisição não é disparada ao vivo porque incorreria em custo, mas fica pronta para execução com uma chave real.

Isso mostra que LoRA não precisa ser associado exclusivamente ao treinamento local.

O mesmo conceito pode ser exposto por provedores gerenciados.

O que muda é a forma de acesso e de cobrança.

Os primeiros números do treino

Na primeira iteração, o validation loss observado é 4,752.

Na vigésima iteração, chega a aproximadamente 0,895.

Essa queda mostra que o modelo está aprendendo o formato desejado.

O pico de memória durante o processo fica em torno de 10,8GB, abaixo dos 24 GB de memória unificada disponíveis na máquina usada na demonstração.

Esse número torna concreta uma das promessas de LoRA.

O ajuste consegue acontecer em hardware de consumidor sem exigir o custo de um cluster ou de uma GPU de datacenter dedicada.

Alternativas para quem não possui Apple Silicon

A disciplina também oferece caminhos alternativos.

Quem não possui uma máquina Apple Silicon pode utilizar um notebook no Google Colab.

A proposta é executar o mesmo modelo, o mesmo dataset e o mesmo rank em uma GPU disponibilizada pelo ambiente.

No resultado apresentado, o validation loss chega a aproximadamente 0,8305, mantendo a mesma conclusão qualitativa do treinamento local.

Para quem possui uma GPU NVIDIA com CUDA em Windows ou Linux, existe também uma versão standalone baseada no ecossistema Hugging Face.

Nesse caso, o treino continua local e não depende da interface do Colab.

A barreira real é a GPU

A aula reforça que o problema central não é exatamente o sistema operacional.

A questão é o acesso a uma GPU adequada.

Máquinas Windows e Linux com GPU NVIDIA podem utilizar o caminho CUDA.

Quem não possui GPU pode recorrer a um ambiente como Colab.

Mesmo com técnicas de quantização, CPU sozinha normalmente não entrega um tempo razoável para esse tipo de treinamento.

A infraestrutura continua sendo parte da decisão de engenharia.

Duas medições não provam convergência

Uma queda de 4,752 para 0,895 parece excelente.

Mas apenas comparar o início e o fim de 20 iterações não prova que o treinamento convergiu.

Para verificar isso, a mesma configuração foi repetida e também estendida.

Na repetição, o loss na iteração 20 aparece em aproximadamente 0,907.

Na execução mais longa, chega a aproximadamente 0,913 nessa mesma região.

Essa pequena diferença entre execuções é ruído normal.

O que realmente importa é a tendência da curva.

As 20 iterações ainda representavam subtreino

Quando o treinamento é estendido para 125 iterações, o validation loss continua caindo fortemente até aproximadamente as iterações 80 ou 90.

Ele chega perto de 0,474.

Por volta da iteração 100 aparece o primeiro sinal de piora, subindo para aproximadamente 0,485.

Depois vai para 0,496 e 0,520 nas medições seguintes.

Isso indica o começo do overfitting.

Portanto, as 20 iterações mostradas inicialmente não representavam convergência.

O modelo ainda estava em subtreino.

Ele continuava aprendendo de maneira significativa.

Early stopping e análise da curva

A ferramenta da disciplina possui uma função para analisar a curva de convergência.

Ela recebe os valores de validation loss observados ao longo das iterações e identifica o melhor ponto da execução.

Esse é o ponto em que uma estratégia de early stopping poderia interromper o treino.

O objetivo é evitar tanto parar cedo demais quanto continuar depois que o modelo começou a piorar.

Esse raciocínio é mais confiável do que escolher arbitrariamente um número fixo de iterações.

Medindo a velocidade real

Outra vantagem do treinamento local é que a própria máquina informa a taxa real de execução.

Na demonstração, aparecem valores próximos de seis iterações por segundo e cerca de 900 tokens processados por segundo em determinada medição.

Esses números permitem estimar o tempo de treino usando a própria máquina como referência.

Não é necessário depender apenas de documentação genérica.

Se o dataset tivesse 2.000 exemplos e fossem executadas dez épocas mantendo uma taxa próxima, a aula estima algo em torno de 56 minutos de computação pura.

A lógica financeira do módulo 1.3 passa a ser alimentada por uma medida concreta de segundos de máquina.

O adaptador final é muito pequeno

Ao final do processo, o arquivo do adaptador possui aproximadamente 27 MB.

Comparado ao modelo base, com tamanho da ordem de 10,24 GB, ele é centenas de vezes menor.

Isso acontece porque o adaptador armazena apenas os parâmetros aprendidos pelas estruturas LoRA.

Ele não duplica o modelo inteiro.

Em um cenário com várias parcerias regionais, o mesmo modelo base pode ser compartilhado enquanto cada parceria mantém apenas seu próprio adaptador.

É exatamente esse comportamento que torna a conta apresentada no módulo anterior plausível na prática.

Rodar sem erro não significa que o treinamento funcionou

A prova mais importante vem no final.

O mesmo exemplo do conjunto de teste, nunca utilizado no treinamento, é enviado primeiro para o modelo base e depois para o mesmo modelo com o adaptador carregado.

Sem o adaptador, o modelo começa a responder em texto livre.

Ele explica o raciocínio, descreve como faria a extração e nem conclui dentro do limite definido.

O modelo não é ruim.

Ele simplesmente não aprendeu que aquela tarefa exige uma saída direta no formato esperado.

Com o adaptador carregado, a resposta muda.

O mesmo modelo base passa a retornar exatamente a estrutura esperada, com os campos correspondentes ao beneficiário, ao procedimento e ao valor do exemplo de teste, batendo com o gabarito reservado.

Esse é o teste que separa "o processo terminou com código zero" de "o fine-tuning realmente alterou o comportamento do modelo".

A próxima etapa do módulo vai aprofundar o papel do rank do LoRA. A ideia será comparar ranks menores e maiores contra o mesmo dataset, medindo o que muda em memória, tempo de treinamento e qualidade, antes de avançar para a comparação direta entre LoRA e Full Fine Tuning.

NA PRÁTICA

- Prepare o mesmo dataset para treino, validação e teste, preservando a separação entre as divisões.
- Execute ou documente o treinamento LoRA local e registre validation loss, tempo e uso de memória observados.
- Valide os hiperparâmetros antes da execução e mantenha a preparação do dataset separada da ferramenta de treinamento.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “LoRA e PEFT (Conceitos Práticos) (PT-2)”?
- Qual relação existe entre LoRA local e validation loss no conteúdo apresentado?
- Que métrica, comparação ou evidência você verificaria para confirmar a conclusão técnica apresentada na aula?

RESUMO DA AULA

- No vídeo anterior, nós ficamos principalmente nos princípios. Entendemos por que um custo fixo de GPU alugada pode deixar de fazer sentido em casos de baixo volume e vimos matematicamente como LoRA contorna esse problema ao congelar a matriz original do modelo e treinar apenas um conjunto pequeno de parâmetros adicionais.
- Nesta aula, nós executamos um treinamento local de verdade, sem nenhuma chamada de nuvem durante o processo principal. O mesmo dataset real de 200 exemplos que já havia sido utilizado no treinamento gerenciado do Vertex AI é reaproveitado em um fluxo local. Primeiro, ele precisa ser convertido para o formato esperado pelo framework de treinamento. Depois, os hiperparâmetros são validados, o processo é orquestrado, as métricas são capturadas em tempo real e, por fim, comparamos o modelo base com o modelo carregado com o adaptador LoRA.
- A intenção é provar duas coisas ao mesmo tempo: que o treinamento realmente aconteceu e que ele alterou de forma útil o comportamento do modelo.

[↑ Voltar ao mapa da disciplina](#)

Aula 3 — LoRA e PEFT (Conceitos Práticos) (PT-3)

Unidade 4 • LoRA e PEFT (Conceitos Práticos)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “LoRA e PEFT (Conceitos Práticos) (PT-3)”.
- Relacionar Rank e scale às decisões, comparações e evidências apresentadas na aula.
- Identificar métricas, custos, hiperparâmetros ou resultados utilizados no conteúdo para sustentar a decisão técnica.

GITHUB E PRÁTICA

- [Projeto do módulo: comparação de ranks, QLoRA e DoRA](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Rank	• Scale
• Quantização	• DoRA
• Validation loss	• Memória

Conteúdo da aula

No módulo anterior, nós executamos um treinamento LoRA real e vimos o adaptador modificar de fato o comportamento do modelo. Só que dois hiperparâmetros apareceram naquela execução sem uma discussão completa sobre a escolha: rank e scale.

O rank utilizado foi 8. Funcionou. O validation loss caiu, o adaptador ficou pequeno e a inferência passou a produzir o formato esperado. Mas dizer que uma configuração funcionou não significa dizer que ela é a melhor configuração.

É isso que esta aula resolve.

Vamos comparar três treinamentos reais usando exatamente o mesmo dataset, o mesmo modelo, o mesmo número de iterações, o mesmo batch size e o mesmo learning rate. A única variável modificada será o rank do LoRA: 4, 8 e 16.

A ideia é transformar um hiperparâmetro que até agora parecia abstrato em uma decisão mensurável de engenharia.

O que o rank controla na prática

No LoRA, a atualização aplicada sobre a matriz original é representada por duas matrizes menores.

O rank, representado por R , controla a dimensão interna dessas matrizes.

Quanto maior o rank, maior a quantidade de parâmetros disponíveis para representar a mudança necessária no comportamento do modelo.

Isso aumenta a capacidade do adaptador.

Ao mesmo tempo, aumenta a quantidade de parâmetros treináveis, o tamanho final do adaptador e, em alguma medida, o custo de processamento.

Um rank menor faz o oposto.

Ele reduz a capacidade do adaptador, mas também reduz o número de parâmetros que precisam ser atualizados.

A pergunta prática não é simplesmente se rank maior é melhor.

A pergunta é quanto de qualidade adicional recebemos em troca do custo adicional.

Rank e scale não são a mesma coisa

Outro parâmetro que aparece junto do rank é o scale.

É importante não confundir os dois.

O rank controla a capacidade.

Ele determina quanto espaço as matrizes de adaptação possuem para representar uma mudança.

O scale controla a intensidade.

Na formulação vista anteriormente, ele corresponde ao fator que multiplica a contribuição aprendida antes de somá-la à matriz original congelada.

A aula relaciona isso ao termo alfa dividido por R.

Nos três experimentos desta parte, o scale permanece fixo em 20.

Apenas o rank muda.

Esse cuidado permite isolar uma variável por vez, exatamente como em qualquer experimento comparativo bem desenhado.

O valor 8 era o padrão da ferramenta

No treinamento anterior, rank 8 não foi escolhido depois de um estudo comparativo.

Ele correspondia ao padrão utilizado pela própria ferramenta quando nenhuma configuração diferente era informada.

Agora, em vez de aceitar esse valor como default, nós o colocamos entre duas alternativas.

Rank 4 representa uma configuração mais enxuta.

Rank 16 representa uma configuração com o dobro da capacidade do rank 8.

Assim conseguimos observar o que acontece quando a capacidade é reduzida e quando é ampliada.

Três treinos com uma única variável diferente

Os três treinamentos utilizam os mesmos 157 exemplos de treino.

Mantemos também as mesmas 20 iterações, batch size 1 e learning rate utilizado no experimento anterior.

O modelo base permanece o mesmo.

Nenhuma outra configuração é alterada.

Esse desenho é fundamental.

Se eu mudasse de rank, learning rate e número de iterações ao mesmo tempo, não conseguiria dizer qual mudança foi responsável pelo resultado.

Aqui, qualquer diferença observada entre os três treinamentos pode ser atribuída principalmente ao rank.

Os números do rank 4

Com rank 4, o adaptador possui aproximadamente 3,4 milhões de parâmetros treináveis.

Isso representa cerca de 0,074% do modelo completo.

O validation loss final observado é 1,246.

O adaptador ocupa aproximadamente 13 MB em disco.

É a configuração mais enxuta das três.

Ela exige menos parâmetros treináveis e produz o menor artefato final.

Os números do rank 8

Com rank 8, passamos para aproximadamente 6,8 milhões de parâmetros treináveis.

Isso corresponde a cerca de 0,147% do modelo.

O validation loss final fica em 0,895.

O adaptador possui aproximadamente 27 MB.

Comparado ao rank 4, dobramos a quantidade de parâmetros e praticamente dobramos também o tamanho em disco.

Ao mesmo tempo, temos uma melhora clara na métrica de validação.

Os números do rank 16

No rank 16, chegamos a aproximadamente 13,6 milhões de parâmetros treináveis.

Isso representa cerca de 0,295% do modelo total.

O validation loss final observado é 0,725.

Validation loss observada nas comparações da disciplina

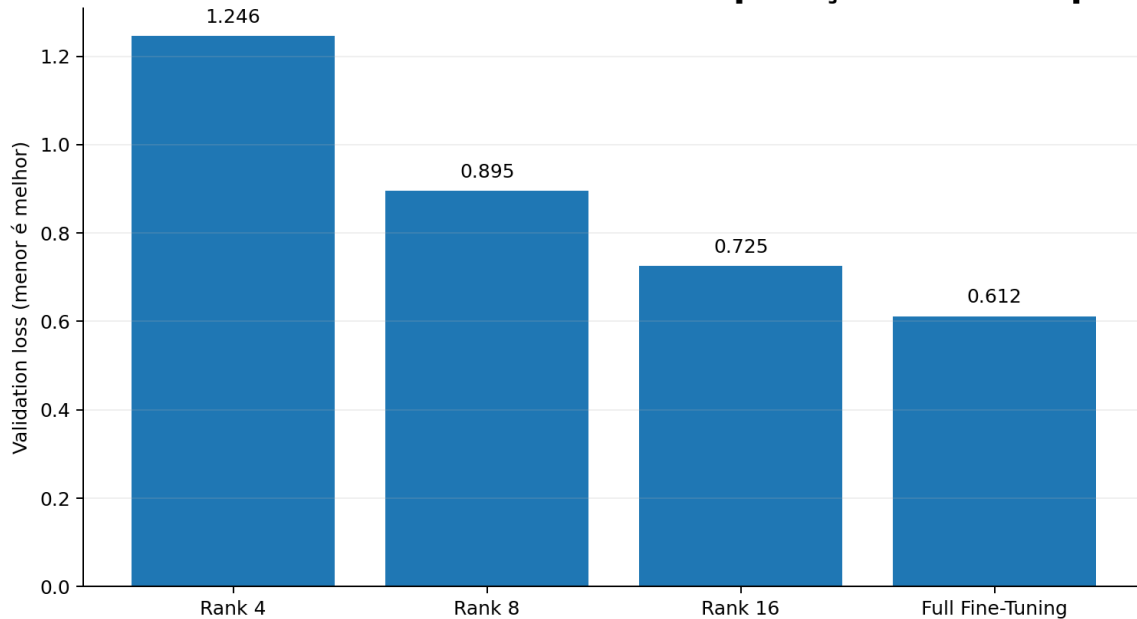


Gráfico — Validation loss observada para ranks LoRA e, posteriormente, Full Fine-Tuning.

O adaptador fica com aproximadamente 52 MB.

Novamente, a quantidade de parâmetros treináveis dobra em relação ao rank anterior e o tamanho do adaptador acompanha quase diretamente essa multiplicação.

Esse comportamento é esperado pela própria aritmética da decomposição LoRA.

O custo de memória não cresce na mesma proporção

O que pode surpreender é o comportamento da memória de pico.

Do rank 4 para o rank 8, o crescimento observado durante o treinamento é de apenas algumas dezenas de megabytes.

Do rank 8 para o 16, o acréscimo continua relativamente pequeno.

Isso acontece porque a maior parte da memória não está sendo consumida pelo adaptador.

Ela está sendo consumida pelo modelo base completo, que precisa permanecer carregado em todas as execuções.

Por isso, para um modelo desse porte, dobrar o rank não significa dobrar a memória total do treinamento.

A pergunta de viabilidade vem antes do rank

Essa medição muda um pouco a intuição.

Antes de escolher o rank, a pergunta mais importante é se o modelo base cabe no hardware disponível.

Se o modelo base já cabe, aumentar o rank dentro de faixas razoáveis pode representar uma diferença pequena de memória.

Nesse cenário, rank deixa de ser principalmente uma decisão de viabilidade e passa a ser muito mais uma decisão de qualidade e eficiência.

Essa relação pode mudar em modelos maiores, porque adaptadores são aplicados em mais camadas e as diferenças absolutas crescem.

Mas o princípio permanece: a maior parcela da memória continua ligada ao modelo base.

A velocidade também muda pouco

Existe uma pequena perda de velocidade quando aumentamos o rank.

Na execução apresentada, o rank 4 trabalha em torno de 7,401 iterações por segundo.

O rank 8 fica próximo de 7,295.

O rank 16 cai para aproximadamente 7,253.

A diferença existe porque matrizes maiores precisam ser atualizadas a cada passo.

Mas, nesse modelo e nesse experimento, a queda é pequena.

Isso reforça que o principal trade-off entre esses ranks específicos aparece mais na qualidade e no tamanho do adaptador do que na capacidade de o treino caber na máquina.

QLoRA como alavanca real de memória

Se o problema for o tamanho do modelo base, existe uma alternativa mais poderosa do que simplesmente reduzir rank.

É o QLoRA.

QLoRA aplica a lógica do LoRA sobre um modelo base quantizado, neste caso em 4 bits.

No experimento comparativo da aula, o modelo em BF16 ocupa aproximadamente 10,24 GB em disco.

A versão quantizada em 4 bits cai para aproximadamente 3,58 GB.

É uma redução de cerca de 65%.

O impacto aparece também na memória durante o treinamento.

O pico cai de aproximadamente 10,833 GB para 4,193 GB, uma redução próxima de 61%.

Quantização também cobra um preço

Essa economia de memória não vem gratuitamente.

Mantendo rank 8 e os demais hiperparâmetros, o validation loss final da versão quantizada fica em aproximadamente 0,932, contra 0,895 da versão BF16.

A diferença é da ordem de 4,1%.

A aula compara esse número com a faixa de ruído observada entre execuções do módulo anterior, em torno de 2,3%.

Como a diferença supera essa faixa, o material interpreta a piora como um efeito real da quantização naquele experimento, e não apenas como variação aleatória entre execuções.

Isso transforma QLoRA em um trade-off concreto: uma redução grande de memória em troca de uma pequena perda de qualidade observada no caso estudado.

Quando QLoRA faz sentido

Para quem possui uma GPU menor ou precisa executar múltiplos experimentos na mesma máquina, essa economia pode ser decisiva.

Se a limitação é a memória, quantizar o modelo base muda muito mais o orçamento do que reduzir o rank do adaptador.

Isso é importante porque direciona a otimização para o componente que realmente domina o consumo.

Em vez de economizar alguns megabytes reduzindo matrizes LoRA, podemos reduzir vários gigabytes ao compactar a representação do próprio modelo.

DoRA como variação da técnica

A aula também compara LoRA com DoRA.

DoRA modifica a forma como a adaptação é aplicada, decompondo a atualização em componentes associados a magnitude e direção.

No experimento apresentado, usamos o mesmo dataset, o mesmo rank 8 e os mesmos hiperparâmetros.

A única mudança é a técnica de adaptação.

O DoRA utiliza aproximadamente 7,3 milhões de parâmetros treináveis, contra 6,8 milhões do LoRA.

O adaptador fica com aproximadamente 28 MB, contra 27 MB.

O pico de memória também cresce um pouco.

O ganho de DoRA não apareceu neste caso

Apesar do custo ligeiramente maior, o validation loss final observado é praticamente o mesmo.

LoRA e DoRA chegam a aproximadamente 0,895 nesse treinamento.

Dentro da faixa de ruído já mensurada, não existe ganho relevante.

Isso não significa que DoRA não funcione.

A literatura pode mostrar vantagens em tarefas mais complexas ou treinamentos mais longos.

O que o experimento demonstra é outra coisa: uma técnica promissora em um paper não deve virar padrão automático sem ser testada no caso real.

O nosso caso era uma tarefa relativamente simples de extração estruturada, com apenas 20 iterações.

Nesse cenário, o benefício adicional não apareceu.

Retorno decrescente ao aumentar o rank

Quando olhamos apenas para rank, aparece um padrão importante.

Do rank 4 para o rank 8, o validation loss melhora aproximadamente 28,17%.

Do rank 8 para o rank 16, dobramos novamente os parâmetros treináveis, mas a melhora cai para aproximadamente 18,99%.
O custo em parâmetros praticamente dobra em cada salto.
O ganho em qualidade não.
É um exemplo mensurado de retorno decrescente.
A primeira dobra entrega uma melhora maior.
A segunda dobra continua ajudando, mas entrega uma fração menor de benefício.

Não existe um rank universalmente correto

Isso impede que a escolha seja reduzida a uma regra como "use sempre rank 16" ou "rank 8 é o padrão".
O rank correto depende da margem de qualidade exigida pelo projeto.
A ferramenta apresentada na aula formaliza essa escolha com uma função que procura o menor rank capaz de permanecer dentro de uma determinada margem em relação ao melhor validation loss encontrado.
Com uma margem de 10% em relação ao melhor resultado, apenas rank 16 entra no critério.
Quando a margem é ampliada para 60%, rank 8 já se torna suficiente.
Rank 4 permanece fora nesse exemplo porque sua perda de validação ficou muito distante da melhor configuração.

A decisão pode ser econômica, não apenas técnica

Essa forma de escolher rank é interessante porque permite conectar qualidade a custo.
Se o negócio exige uma margem muito apertada em relação ao melhor modelo, talvez valha pagar por um adaptador maior.
Se uma pequena diferença de qualidade não muda a decisão de negócio, um rank menor pode ser mais eficiente.
O objetivo não é maximizar a métrica a qualquer custo.
É encontrar o menor nível de parametrização que entregue qualidade suficiente para a necessidade real.

Um teste adversarial construído à mão

Depois da comparação agregada, a aula cria um exemplo novo propositalmente mais difícil.
O caso vem de uma oficina que não apareceu no dataset original.
O texto contém mais de um valor monetário.
Existe um valor de peças, de R\$1.850.
Existe um valor de mão de obra, de R\$970.
E apenas depois aparece o valor total do orçamento, R\$2.820.
A tarefa é extrair o total correto, e não simplesmente capturar o primeiro número encontrado.

Um caso difícil para testar compreensão do padrão

Esse exemplo foi desenhado para dificultar uma estratégia superficial.
Se o modelo apenas decorou posição ou aprendeu que qualquer valor monetário deve ser devolvido, ele pode errar.
Para acertar, precisa reconhecer que o campo solicitado corresponde ao valor total do orçamento.
A comparação é executada quatro vezes.
Primeiro com o modelo base sem adaptador.
Depois com os adaptadores de rank 4, 8 e 16.

O modelo base repete o problema anterior

Sem adaptador, o comportamento é semelhante ao observado no módulo 4.2.
O modelo entra em uma resposta mais livre, explicando raciocínio e não entregando diretamente a estrutura esperada dentro do limite de geração utilizado.
Novamente, isso não significa que o modelo base seja incapaz de compreender o texto.
Significa que ele não foi ajustado para obedecer exatamente ao contrato de saída da tarefa.

Os três ranks acertam o caso adversarial

Com qualquer um dos três adaptadores, o resultado bate com o gabarito do exemplo.
Os modelos extraem corretamente o segurado Ricardo Alves Monteiro, a placa JBR9021 e o valor total de 2.820.
Também ignoram corretamente os dois valores distratores.
Nesse teste específico, rank 4, rank 8 e rank 16 apresentam o mesmo comportamento final observado.

Isso cria uma tensão interessante com o validation loss agregado.

Validation loss e comportamento observável não são a mesma coisa

O validation loss diferencia claramente os três treinamentos.

Rank 16 apresenta a melhor perda.

Rank 8 vem depois.

Rank 4 apresenta a pior.

Mesmo assim, o exemplo adversarial não separa as três configurações.

Todos acertam.

Isso mostra que uma métrica agregada pode capturar diferenças que não necessariamente aparecem em uma decisão final específica.

Talvez o loss esteja refletindo maior robustez a variações de fraseado.

Talvez esteja capturando pequenas diferenças probabilísticas que não mudam os campos finais extraídos.

Também pode existir ruído de ajuste.

Um único exemplo não permite concluir qual dessas explicações é a correta.

Um caso bem desenhado ainda não é uma avaliação completa

Esse é um ponto importante da aula.

Criar um exemplo difícil é melhor do que testar apenas um caso trivial.

Mas um ou dois exemplos continuam insuficientes para avaliar de forma confiável a qualidade geral de um modelo.

Para comparar ranks de verdade, precisamos de um conjunto de teste maior, idealmente com dezenas ou centenas de casos representativos, cobrindo diferentes formulações, fontes e situações adversariais.

É esse tipo de rigor que será aprofundado posteriormente na disciplina.

A avaliação de fine-tuning precisa combinar métrica agregada e comportamento em dados realmente representativos.

O trade-off final desta etapa

Os experimentos desta aula deixam algumas decisões muito mais concretas.

Rank maior aumenta capacidade, parâmetros treináveis e tamanho do adaptador.

O ganho em validation loss existe, mas apresenta retorno decrescente.

A memória total de treinamento cresce pouco nesse modelo porque o modelo base domina o consumo.

Se a memória for a restrição principal, QLoRA oferece uma redução muito mais relevante ao quantizar o modelo base.

DoRA pode ser promissor, mas precisa demonstrar ganho no caso específico antes de justificar a troca.

E validation loss, apesar de central para acompanhar o treinamento, não substitui uma avaliação comportamental ampla.

A próxima pergunta fecha o módulo: mesmo escolhendo corretamente o rank e usando LoRA de forma eficiente, até que ponto esse ajuste consegue se aproximar de um Full Fine Tuning real? A comparação seguinte coloca as duas abordagens lado a lado na mesma tarefa para mensurar justamente esse limite.

NA PRÁTICA

- Compare pelo menos dois ranks mantendo as demais condições do experimento constantes.
- Registre validation loss, memória, tempo e tamanho do adaptador para cada configuração comparada.
- Inclua um exemplo propositalmente difícil para verificar se a diferença de rank altera o comportamento observado.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “LoRA e PEFT (Conceitos Práticos) (PT-3)”?
- Qual relação existe entre Rank e scale no conteúdo apresentado?
- Que métrica, comparação ou evidência você verificaria para confirmar a conclusão técnica apresentada na aula?

RESUMO DA AULA

- No módulo anterior, nós executamos um treinamento LoRA real e vimos o adaptador modificar de fato o comportamento do modelo. Só que dois hiperparâmetros apareceram naquela execução sem uma discussão completa sobre a escolha: rank e scale.
- O rank utilizado foi 8. Funcionou. O validation loss caiu, o adaptador ficou pequeno e a inferência passou a produzir o formato esperado. Mas dizer que uma configuração funcionou não significa dizer que ela é a melhor configuração.
- Vamos comparar três treinamentos reais usando exatamente o mesmo dataset, o mesmo modelo, o mesmo número de iterações, o mesmo batch size e o mesmo learning rate. A única variável modificada será o rank do LoRA: 4, 8 e 16.

[↑ Voltar ao mapa da disciplina](#)

Aula 4 — LoRA e PEFT (Conceitos Práticos) (PT-4)

Unidade 4 • LoRA e PEFT (Conceitos Práticos)

OBJETIVOS DE APRENDIZAGEM

- Compreender os conceitos apresentados em “LoRA e PEFT (Conceitos Práticos) (PT-4)”.
- Relacionar Full Fine-Tuning e LoRA às decisões, comparações e evidências apresentadas na aula.
- Identificar métricas, custos, hiperparâmetros ou resultados utilizados no conteúdo para sustentar a decisão técnica.

GITHUB E PRÁTICA

- [Projeto do módulo: LoRA versus Full Fine-Tuning](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Full Fine-Tuning	• LoRA
• Parâmetros treináveis	• Validation loss
• Checkpoint	• Esquecimento catastrófico

Conteúdo da aula

Nos vídeos anteriores, nós saímos dos princípios de LoRA, executamos um treino local real e depois medimos o efeito do rank em três configurações diferentes. Vimos que rank 4, 8 e 16 produzem custos e validation losses diferentes, mas também vimos uma coisa importante: nos exemplos concretos utilizados, inclusive em um caso construído de propósito para ser difícil, todos os adaptadores conseguiram chegar à resposta correta.

Ficou então a pergunta que acompanha o módulo desde o início. Será que isso acontece apenas entre diferentes configurações de LoRA ou será que um Full Fine-Tuning, atualizando muito mais pesos, realmente entrega uma diferença prática que justifique o custo?

Nesta aula, essa comparação deixa de ser teórica. O Full Fine-Tuning é executado na mesma máquina, com o mesmo modelo, o mesmo dataset, as mesmas 20 iterações e o mesmo learning rate. A ideia é medir qualidade, memória e tamanho do checkpoint com o mesmo rigor que usamos até agora.

O que significa Full Fine-Tuning nesta comparação

Antes de olhar os números, eu preciso deixar claro o escopo.

O framework utilizado chama de Full Fine-Tuning o treinamento que atualiza todos os pesos das camadas selecionadas. Ele não utiliza a decomposição em matrizes B e A do LoRA.

Mas isso não significa que todos os bilhões de parâmetros do modelo inteiro serão treinados.

Na configuração padrão usada nesta disciplina, tanto LoRA quanto Full atuam sobre as últimas 16 camadas.

A comparação, portanto, é justa.

Os dois métodos trabalham sobre a mesma região do modelo. A diferença é que LoRA mantém os pesos originais congelados e aprende pequenas atualizações adicionais, enquanto o Full Fine-Tuning modifica diretamente todos os pesos das camadas selecionadas.

Por que o Full ainda cabe na máquina

O modelo utilizado possui cerca de 4,63 bilhões de parâmetros.

Treinar absolutamente todos esses parâmetros provavelmente ultrapassaria os 24 GB de memória disponíveis na máquina da demonstração.

O Full Fine-Tuning das últimas 16 camadas, porém, corresponde a aproximadamente 22,567% do modelo.

Esse volume coube na memória e conseguiu ser executado localmente.

Isso mostra por que precisamos ter cuidado com a expressão Full Fine-Tuning. Aqui estamos fazendo Full nas camadas escolhidas, sem decomposição LoRA, mas não liberando todas as camadas do modelo inteiro.

A conta de memória do Full Fine-Tuning

O custo de memória ajuda a entender a diferença entre os dois caminhos.

No Full Fine-Tuning, cada peso treinável precisa participar diretamente da otimização.

Durante o treino, não basta armazenar apenas o valor do peso.

Também precisamos do gradiente e dos estados mantidos pelo otimizador Adam, como momento e variância.

De maneira simplificada, isso multiplica bastante a quantidade de memória necessária por parâmetro treinável.

No LoRA, essa estrutura de otimização é necessária apenas para os parâmetros pequenos das matrizes B e A.

A matriz original permanece congelada.

Ela precisa estar carregada para a inferência e para a passagem da informação, mas não precisa manter gradiente nem estados do otimizador para cada um de seus pesos.

Essa diferença explica por que Full Fine-Tuning cresce em memória muito mais rapidamente conforme liberamos mais parâmetros.

A comparação utiliza os mesmos dados

O experimento mantém o mesmo dataset de 157 exemplos de treino utilizado nos testes de LoRA.

Também permanecem as mesmas 20 iterações e o mesmo learning rate.

O objetivo é evitar uma comparação contaminada por mudanças em várias configurações ao mesmo tempo.

A única mudança estrutural é o método de ajuste.

De um lado, temos LoRA em ranks 4, 8 e 16.

Do outro, temos Full Fine-Tuning nas mesmas 16 camadas.

Os números apresentados na aula vêm de execuções reais do processo.

Quantos parâmetros o Full treina

No Full Fine-Tuning utilizado, aproximadamente 22,567% dos parâmetros do modelo participam do treinamento.

No LoRA rank 8, esse valor era de aproximadamente 0,147%.

A diferença é enorme.

O Full treina cerca de 153 vezes mais parâmetros do que o LoRA rank 8.

Isso representa uma capacidade muito maior de modificar diretamente o comportamento das camadas selecionadas.

Mas essa liberdade precisa entregar ganho suficiente para compensar memória, armazenamento e maior superfície de mudança.

Validation loss do Full contra LoRA rank 8

No experimento, o Full Fine-Tuning termina com validation loss de aproximadamente 0,612.

O LoRA rank 8 havia terminado em 0,895.

O Full, portanto, apresenta um ganho de aproximadamente 31% nessa comparação específica.

Isso confirma que existe, de fato, um teto de qualidade mais alto quando liberamos muito mais parâmetros para treinamento.

Full Fine-Tuning não é apenas uma versão mais cara que sempre empata com LoRA.

Na métrica agregada, ele consegue ir além.

Comparando com o melhor LoRA testado

A comparação mais importante, porém, não é contra o rank 8.

No vídeo anterior, rank 16 foi a melhor configuração LoRA entre as testadas.

A aula utiliza um validation loss de referência para essa melhor configuração e calcula o ganho adicional obtido pelo Full.

Nesse confronto, a melhora relativa do Full cai para aproximadamente 15,59%.

Esse número é muito mais relevante para a decisão.

Se eu já posso usar o melhor LoRA encontrado, preciso perguntar se vale pagar todo o custo adicional do Full para obter aproximadamente esse ganho.

O pico de memória

O pico de memória do Full Fine-Tuning chega a aproximadamente 15,338 GB.

No LoRA rank 8, o pico observado era de aproximadamente 10,833 GB.

Isso representa um aumento da ordem de 42%.

O Full ainda cabe na máquina utilizada, mas começa a ocupar uma parcela muito maior da memória disponível.

Em modelos maiores ou em uma configuração que liberasse mais camadas, essa diferença poderia se tornar uma barreira real.

O tamanho do checkpoint

A diferença mais agressiva aparece no armazenamento.

O checkpoint gerado pelo Full Fine-Tuning fica em aproximadamente 1,992 GB.

O adaptador LoRA rank 8 possui apenas cerca de 27 MB.

O artefato do Full fica aproximadamente 73,8 vezes maior.

Essa diferença acontece porque o Full precisa salvar novamente os pesos alterados das camadas treinadas.

LoRA armazena apenas a pequena atualização aprendida pelas matrizes de adaptação.

Quando existem várias especializações, essa diferença se multiplica.

Um critério explícito para decidir se Full compensa

A ferramenta da aula formaliza a decisão usando um limiar mínimo de ganho.

A pergunta é: o quanto o Full precisa melhorar o validation loss em relação ao melhor LoRA para justificar o custo extra?

Se eu definir que o ganho precisa ser pelo menos 20%, o Full Fine-Tuning não passa nesse caso.

A melhora real em relação ao melhor LoRA fica em aproximadamente 15,59%.

Se o limiar for reduzido para 10%, ele passa.

Não existe um limiar universal.

Esse número depende do contexto operacional e econômico do projeto.

O custo de armazenamento muda a resposta

Imagine uma empresa em que armazenar e servir checkpoints de quase 2 GB seja barato.

Nesse contexto, uma melhora de 15% pode ser suficiente para justificar o Full.

Agora volte para o caso das parcerias regionais da Amplitude Seguros.

Ali, o objetivo era justamente evitar custo fixo alto para operações pequenas.

Cinco parcerias usando LoRA rank 8 precisariam de cinco adaptadores de aproximadamente 27 MB, cerca de 135 MB no total, compartilhando o mesmo modelo base.

Com Full Fine-Tuning, seriam cinco checkpoints de aproximadamente 2 GB, chegando perto de 10 GB adicionais.

Nesse cenário, o ganho de qualidade observado não muda a decisão arquitetural que já havia sido construída desde o módulo 4.1.

Um único modelo base pode servir vários adaptadores

A vantagem de LoRA não está apenas no disco.

Em stacks de serving compatíveis, vários adaptadores podem compartilhar o mesmo modelo base.

Isso permite que uma única base atenda especializações diferentes, cada uma com seu pequeno conjunto de pesos adicionais.

No caso das cinco parcerias regionais, não precisamos necessariamente carregar cinco cópias independentes do modelo inteiro.

Podemos manter uma base e servir os adaptadores correspondentes às diferentes especializações.

Essa característica reforça a vantagem operacional quando existem muitas variantes de uma mesma tarefa.

O segundo eixo: esquecimento catastrófico

Além de custo, existe um segundo risco.

É o esquecimento catastrófico.

Quando alteramos diretamente muitos pesos de um modelo, existe a possibilidade de melhorar muito a tarefa específica e, ao mesmo tempo, degradar capacidades gerais que não estavam representadas no dataset de treinamento.

LoRA reduz essa superfície de alteração porque mantém o modelo original congelado e aplica uma atualização pequena por meio dos adaptadores.

Full Fine-Tuning toca diretamente em uma quantidade muito maior de parâmetros.

Em teoria, quanto maior a superfície modificada, maior pode ser o risco de esquecer comportamentos anteriores.

O experimento desta aula não mede esse risco diretamente

É importante não extrapolar a demonstração.

O treino executado nesta aula libera apenas as últimas 16 camadas.

Além disso, a avaliação é feita na própria tarefa de extração estruturada.

Não estamos testando se o modelo perdeu capacidade de responder perguntas gerais, manter uma conversa adequada ou executar tarefas completamente diferentes.

Portanto, o experimento não mede esquecimento catastrófico diretamente.

A aula apresenta esse ponto como um risco discutido na literatura de PEFT e cita estudos que observaram degradação maior em Full Fine-Tuning do que em configurações LoRA em avaliações fora da tarefa treinada.

Evidências citadas na aula

Um dos estudos mencionados compara um modelo base com Full Fine-Tuning e diferentes ranks de LoRA ao longo de várias épocas de treinamento de instrução.

Segundo os números apresentados na aula, o desempenho do Full em tarefas gerais cai de forma mais acentuada conforme o treinamento continua, enquanto as variantes LoRA preservam melhor parte das capacidades anteriores.

Outro estudo citado avalia modelos BLOOM de diferentes tamanhos e investiga esquecimento em dimensões como conhecimento de domínio, raciocínio, leitura e viés.

O objetivo desses exemplos é reforçar que custo não é o único motivo para preferir PEFT.

Preservar capacidades do modelo base também pode fazer parte da decisão.

Testando o Full nos mesmos exemplos

Depois da comparação de métricas, a aula executa o checkpoint Full contra os mesmos tipos de exemplos usados nos módulos anteriores.

Existe um exemplo mais simples, no qual beneficiário, procedimento e valor aparecem de forma direta.

E existe o caso adversarial com vários valores que funcionam como distratores antes do total correto do orçamento.

Esse segundo exemplo havia sido construído justamente para separar um modelo que apenas capturasse qualquer número de um modelo que realmente aprendesse a extrair o valor solicitado.

O Full acerta os dois casos

O Full Fine-Tuning acerta o exemplo simples.

Também acerta o exemplo com distratores.

A saída coincide campo a campo com o comportamento observado nos adaptadores LoRA rank 4, 8 e 16.

Isso é central para a decisão deste módulo.

O Full possui validation loss melhor.

Esse ganho é mensurável.

Mas, nos exemplos comportamentais utilizados nesta tarefa estreita de extração estruturada, o ganho não se transforma em um erro corrigido que os adaptadores LoRA não fossem capazes de corrigir.

Existe um teto, mas ele não aparece como valor de negócio

A aula resume esse resultado de forma prática.

O teto adicional de qualidade do Full existe.

Mas, para essa tarefa específica, ele não aparece de forma observável nos exemplos utilizados.

Isso não significa que Full Fine-Tuning nunca seja necessário.

Em tarefas mais abertas, variadas ou ambíguas, a liberdade adicional de atualizar muitos pesos pode se traduzir em diferenças comportamentais muito mais importantes.

O resultado vale para o caso estudado.

Na extração de campos fixos dos documentos da Amplitude Seguros, LoRA entrega o mesmo comportamento prático observado por uma fração do custo.

A Missão Prática 4

O fechamento do módulo vem com a Missão Prática número 4.

A primeira parte é escolher um caso real do próprio trabalho que já tenha sido aprovado pelo framework do módulo 1.3, mas cujo volume seja pequeno demais para justificar diretamente um treinamento caro em GPU alugada.

A decisão começa pelo problema de negócio.

LoRA não deve ser usado para salvar um caso que já deveria ter sido reprovado pelo GATE.

Comparar pelo menos dois ranks

A segunda parte é executar ou documentar um treino LoRA local para esse caso.

A atividade pede comparação de pelo menos dois ranks.

O objetivo não é simplesmente produzir um adaptador.

É registrar o trade-off entre qualidade, tamanho, memória e custo.

Se o hardware disponível não permitir executar o treinamento real, essa limitação deve ser documentada de forma explícita.

A atividade aceita a análise baseada nos números reais fornecidos pela disciplina quando a restrição de hardware impedir a execução.

Construir um exemplo propositalmente difícil

A terceira parte exige um teste adversarial.

Não basta escolher um exemplo fácil que qualquer configuração acerte.

O estudante precisa construir pelo menos um caso deliberadamente mais difícil e verificar se o modelo realmente aprendeu o comportamento desejado.

Esse princípio vem diretamente dos resultados dos módulos 4.2 e 4.3.

Uma execução sem erro e um exemplo trivial não são evidência suficiente de qualidade.

Implementação e validação em JavaScript

A quarta parte pede a implementação, em JavaScript, da preparação necessária para o treinamento local.

Isso inclui a conversão do dataset para o formato esperado pelo framework e a validação dos hiperparâmetros antes de disparar o processo.

A referência é a mesma arquitetura utilizada nas demonstrações do módulo.

O estudante deve documentar a decisão final: LoRA valeu a pena ou o caso realmente pede Full Fine-Tuning?

A resposta deve ser sustentada por números.

Três caminhos reais de fine-tuning

O módulo fecha uma decisão que estava aberta desde os primeiros capítulos.

Agora temos três caminhos concretos.

O primeiro é a API gerenciada, executada anteriormente com o Vertex AI.

O segundo é LoRA local.

O terceiro é Full Fine-Tuning local.

Os três funcionam.

Nenhum é sempre a escolha correta.

A diferença é que agora temos números para comparar custo, memória, qualidade, armazenamento e grau de controle.

A disciplina de decidir com medição

Esse é o fio condutor do módulo.

A decisão entre custo fixo e custo marginal foi medida.

O efeito de rank pequeno e rank grande foi medido.

A quantização foi medida.

LoRA e DoRA foram comparados.

Agora Full Fine-Tuning e LoRA também foram executados lado a lado.

A ideia não é decorar uma regra de bolso.

É desenvolver a disciplina de executar, medir e decidir com base no caso real.

O modelo que segue para o próximo módulo será o adaptador LoRA rank 8 treinado anteriormente, e não o checkpoint Full desta aula. Para o caso da Amplitude, o ganho adicional do Full não justificou o custo extra.

A partir daqui, a pergunta muda. Até este ponto, aprendemos diferentes maneiras de treinar. O próximo módulo entra na avaliação e passa a responder como saber, com rigor, se um modelo ajustado está realmente bom, sem depender de um ou dois exemplos isolados, usando como referência tanto o modelo treinado no Vertex AI quanto o modelo LoRA local desta disciplina.

NA PRÁTICA

- Compare LoRA e Full Fine-Tuning sobre a mesma tarefa e o mesmo conjunto de dados usado na demonstração.
- Registre parâmetros treináveis, validation loss, pico de memória e tamanho do checkpoint para sustentar a comparação.
- Teste um exemplo simples e um exemplo adversarial e documente se o ganho do Full Fine-Tuning produz diferença prática no caso.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, a ideia central de “LoRA e PEFT (Conceitos Práticos) (PT-4)”?
- Qual relação existe entre Full Fine-Tuning e LoRA no conteúdo apresentado?
- Que métrica, comparação ou evidência você verificaria para confirmar a conclusão técnica apresentada na aula?

RESUMO DA AULA

- Nos vídeos anteriores, nós saímos dos princípios de LoRA, executamos um treino local real e depois medimos o efeito do rank em três configurações diferentes. Vimos que rank 4, 8 e 16 produzem custos e validation losses diferentes, mas também vimos uma coisa importante: nos exemplos concretos utilizados, inclusive em um caso construído de propósito para ser difícil, todos os adaptadores conseguiram chegar à resposta correta.
- Ficou então a pergunta que acompanha o módulo desde o início. Será que isso acontece apenas entre diferentes configurações de LoRA ou será que um Full Fine-Tuning, atualizando muito mais pesos, realmente entrega uma diferença prática que justifique o custo?
- Nesta aula, essa comparação deixa de ser teórica. O Full Fine-Tuning é executado na mesma máquina, com o mesmo modelo, o mesmo dataset, as mesmas 20 iterações e o mesmo learning rate. A ideia é medir qualidade, memória e tamanho do checkpoint com o mesmo rigor que usamos até agora.

[↑ Voltar ao mapa da disciplina](#)

Revisão da Unidade 4

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de decisão, treinamento e comparação.

FLUXO VISUAL DA UNIDADE



CHECKLIST DA UNIDADE

- [Revisar a Aula 1: LoRA e PEFT \(Conceitos Práticos\) \(PT-1\)](#)
- [Revisar a Aula 2: LoRA e PEFT \(Conceitos Práticos\) \(PT-2\)](#)
- [Revisar a Aula 3: LoRA e PEFT \(Conceitos Práticos\) \(PT-3\)](#)
- [Revisar a Aula 4: LoRA e PEFT \(Conceitos Práticos\) \(PT-4\)](#)

[↑ Voltar ao mapa da disciplina](#)

Unidade 5

Avaliar Modelos Fine-Tunados

Avaliação rigorosa de generalização, comparação com baseline, testes de estresse, investigação de overfitting e critérios explícitos para decidir se o piloto pode avançar para a escala.

GITHUB E PRÁTICA

- [Abrir pasta do Módulo 5 — Avaliação de Modelos](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

FLUXO VISUAL DA UNIDADE

Fluxo visual da Unidade 5



Aula	Tema
1	Avaliar Modelos Fine-Tunados (PT-1)
2	Avaliar Modelos Fine-Tunados (PT-2)
3	Avaliar Modelos Fine-Tunados (PT-3)
4	Avaliar Modelos Fine-Tunados (PT-4)

Aula 1 — Avaliar Modelos Fine-Tunados (PT-1)

Unidade 5 • Avaliar Modelos Fine-Tunados

OBJETIVOS DE APRENDIZAGEM

- Diferenciar validation loss de uma avaliação final de generalização.
- Organizar os papéis de treino, validação e teste retido no ciclo de desenvolvimento.
- Aplicar precisão por campo, consistência e adequação de esquema em um harness de avaliação reproduzível.

GITHUB E PRÁTICA

- [Projeto do módulo: conjunto de teste retido e harness de avaliação](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Validation loss	• Teste retido
• Precisão por campo	• Consistência
• Adequação de esquema	• Evaluation harness

Conteúdo da aula

Neste módulo, a gente começa a responder uma pergunta que ficou em aberto desde os módulos anteriores: o modelo fine-tunado realmente generaliza ou apenas ficou muito bom nos dados que já conhecemos?

Até aqui, nós utilizamos validation loss para comparar configurações. Foi assim que analisamos alternativas de LoRA com Rank 4, 8 e 16, além do Full Fine-Tuning. Esse critério foi importante para escolher as melhores configurações e entender o comportamento do treinamento.

Mas validation loss não é a mesma coisa que uma avaliação final de generalização.

Esse é o ponto de partida deste capítulo.

Validation loss não é avaliação final

O validation loss utilizado anteriormente foi mensurado em um split genuinamente separado do treino.

Isso é correto e tem valor.

O problema é que esse mesmo conjunto pequeno, com apenas 30 exemplos, foi reutilizado repetidamente para ajudar a escolher entre diferentes configurações.

Quando eu olho várias vezes para o mesmo conjunto de validação e tomo decisões com base nele, começo a ajustar indiretamente minhas escolhas àquele conjunto.

Não estou treinando os pesos diretamente nele, mas estou usando seu resultado para decidir arquitetura, rank, estratégia de ajuste e configuração.

Por isso, validation loss ajuda a selecionar o modelo.

Ele não prova sozinho que o modelo vai resolver exemplos novos de verdade.

A analogia do simulado

Uma forma simples de entender isso é pensar em um aluno que revisa várias vezes o mesmo simulado.

A nota pode melhorar bastante.

Mas essa melhora não garante que ele terá o mesmo desempenho em uma prova que nunca viu.

O problema da avaliação de modelos é semelhante.

Se usamos sempre os mesmos exemplos para orientar nossas decisões, existe o risco de construir uma confiança maior do que a evidência realmente permite.

A pergunta de produção não é se o modelo está bom naquele conjunto conhecido.

A pergunta é se ele resolve casos inéditos.

O problema herdado do módulo 3.2

No job gerenciado publicado anteriormente no Vertex AI, nós treinamos com 200 exemplos reais.

Eram 120 exemplos do domínio de automóvel e 80 do domínio de saúde empresarial.

Todos os 200 foram utilizados no treinamento.

Nenhum exemplo foi separado como conjunto de teste final.

Isso significa que aquele modelo chegou a um endpoint real, funcional e publicado, mas ainda sem uma avaliação independente de generalização.

Esse é um erro clássico em Machine Learning.

Avaliar o modelo sobre dados de treino não demonstra que ele aprendeu o padrão da tarefa.

Pode apenas demonstrar que ele consegue reproduzir aquilo que já viu.

Treino, validação e teste possuem papéis diferentes

Vale organizar bem esses três conceitos.

O conjunto de treino serve para ajustar o modelo.

O conjunto de validação serve para comparar configurações e tomar decisões durante o desenvolvimento.

O conjunto de teste serve para uma pergunta diferente: depois que as escolhas foram feitas, como esse modelo se comporta em dados que ficaram retidos e não participaram dessas decisões?

Essa separação é essencial.

Se o teste também influencia a escolha do modelo, ele deixa de funcionar como teste final.

Treino, validação e teste: papéis diferentes



Figura — Treino, validação e teste possuem papéis diferentes no ciclo de avaliação.

Construindo um conjunto de teste retido

O primeiro objetivo desta aula é corrigir a ausência de um conjunto de testes no fluxo anterior.

A estratégia é construir exemplos novos a partir do mesmo gerador determinístico utilizado no módulo 3.2.

Isso mantém os mesmos templates e o mesmo tipo de tarefa, mas utiliza índices que ficaram completamente fora da faixa usada no treinamento.

Assim, criamos exemplos que pertencem ao mesmo problema, porém não foram utilizados para ajustar o modelo.

Mesmo gerador, novos índices

Esse detalhe é importante.

O conjunto de teste não é um dataset completamente diferente.

Ele vem do mesmo pipeline determinístico.

A diferença está nos índices.

O treinamento nunca utilizou esses casos.

Essa decisão permite testar uma primeira forma de generalização: o modelo consegue executar corretamente a tarefa em exemplos inéditos produzidos pelo mesmo processo de geração?

É uma evidência muito mais forte do que testar sobre o próprio treino.

Onze exemplos retidos

A avaliação utiliza 11 exemplos retidos.

Eles cobrem as seis fontes reais do domínio de automóvel e as cinco fontes do domínio de saúde empresarial.

A ideia é não avaliar apenas as fontes mais frequentes.

Também entram as de menor volume no treinamento.

Cada exemplo é gerado com índice 5000 ou superior.

No treinamento, o gerador nunca passou do índice 59 por fonte.

Por que usar índice 5000

O índice 5000 não possui nenhum significado matemático especial.

Ele é apenas uma margem de segurança deliberadamente grande.

Qualquer índice acima de 59 já estaria fora da faixa utilizada anteriormente.

Mas começar em 5000 reduz o risco de alguém ampliar o dataset de treino no futuro e, sem perceber, criar sobreposição com a faixa reservada para teste.

É uma escolha simples de engenharia para tornar a separação mais robusta.

O que ainda pode se repetir

Existe uma ressalva importante.

Os nomes dos exemplos retidos são genuinamente inéditos.

Mas outras informações, como placa, valor ou data, podem vir de pools menores que se repetem quando o índice ultrapassa o tamanho desses pools.

Isso significa que um valor ou uma placa pode ter aparecido antes no treino, agora combinados com outro nome e outro exemplo.

Para esta tarefa específica, isso não invalida o teste.

Por que a repetição parcial não invalida a avaliação

A tarefa é de extração estruturada.

O valor correto está escrito dentro do próprio texto de entrada.

O modelo não precisa lembrar de uma placa ou de um valor visto anteriormente.

Ele precisa ler o documento e extrair corretamente os campos solicitados.

Por isso, o que estamos testando é se ele aprendeu a executar o padrão de extração em um exemplo novo.

Essa distinção é importante para interpretar corretamente os resultados.

Três perguntas diferentes exigem três métricas

Com o conjunto de testes retido pronto, a avaliação passa a responder três perguntas diferentes.

A primeira é: o conteúdo extraído está correto?

A segunda é: o modelo responde de forma estável quando recebe exatamente a mesma entrada mais de uma vez?

A terceira é: a saída respeita o esquema exigido pela aplicação?

Essas perguntas correspondem a três métricas.

Precisão por campo.

Consistência.

Adequação de esquema.

Precisão por campo

Precisão por campo avalia cada elemento esperado da resposta.

No domínio de automóvel, podemos ter segurado, placa e valor.

No domínio de saúde empresarial, beneficiário, procedimento e valor.

A avaliação não pergunta se a resposta parece boa.

Ela compara campo a campo com o resultado esperado.

O segurado está exatamente correto?

A placa confere?

O beneficiário é o mesmo?

O procedimento foi extraído corretamente?

O valor está dentro da tolerância numérica definida?

Uma métrica objetiva

Essa forma de avaliação é contável e reprodutível.

Não depende da opinião de quem está lendo a resposta.

Se o campo está correto, contabilizamos acerto.

Se está errado, contabilizamos erro.

Para valores numéricos, podemos aceitar uma pequena tolerância para diferenças de arredondamento quando isso fizer sentido.

Esse tipo de checagem é especialmente adequado para tarefas com resposta objetiva e estruturada.

Consistência entre chamadas

A segunda métrica é consistência.

O mesmo exemplo é enviado ao endpoint várias vezes.

Depois, comparamos as respostas.

Se a entrada não mudou, mas a resposta muda de uma chamada para outra, existe um risco operacional.

Em produção, esse tipo de instabilidade pode gerar divergências difíceis de rastrear.

Por que consistência importa

Imagine o mesmo orçamento de oficina sendo processado duas vezes por engano.

Se uma chamada retorna um valor e outra chamada retorna outro, qual dos dois deve ser considerado correto?

As duas respostas vieram do mesmo modelo.

Para o mesmo documento.

No mesmo sistema.

Esse tipo de divergência pode passar silenciosamente pelo pipeline e se transformar em um problema muito mais caro depois.

Adequação de esquema

A terceira métrica é adequação de esquema.

O modelo precisa produzir JSON válido.

Além disso, precisa retornar exatamente aos campos esperados.

Nem faltando.

Nem sobrando.

Um modelo pode acertar o conteúdo e ainda quebrar a integração se produzir uma estrutura diferente daquela que o parser espera.

Em sistemas reais, formato também é parte da qualidade.

Formato correto também é requisito de produção

Uma resposta pode parecer tecnicamente bonita para uma pessoa.

Mas se vier com um campo extra, com um campo ausente ou em uma estrutura inesperada, pode quebrar o código que consome aquela saída.

Por isso, a avaliação não termina em verificar se o conteúdo parece correto.

O modelo precisa cumprir o contrato da integração.

Essa é uma exigência diferente da precisão.

Nenhuma métrica conta a história inteira

Essas três métricas são complementares.

Um modelo pode ter precisão alta e esquema ruim.

Pode ter esquema perfeito e conteúdo errado.

Pode acertar tudo em uma chamada e responder de forma diferente na próxima.

Cada métrica captura um tipo de falha.

Por isso, reduzir a avaliação a um único número produz uma visão incompleta.

Essa ideia de usar uma bateria de testes reprodutível aparece de forma consolidada na pesquisa e na prática de avaliação de modelos.

Um harness de avaliação

O script utilizado na aula é tratado como um harness de avaliação.

A ideia é criar uma estrutura automatizada que execute sempre os mesmos testes, com os mesmos critérios, contra o modelo avaliado.

O nome faz referência ao LM Evaluation Harness, projeto de código aberto da EleutherAI que se tornou uma referência para avaliações reprodutíveis de modelos.

O ponto principal é sair de testes improvisados e passar para uma bateria organizada.

Por que não usar LLM como juiz aqui

Existe uma técnica conhecida como LLM-as-a-Judge, em que um modelo de linguagem avalia a resposta de outro modelo.

Ela pode ser útil quando a resposta correta é subjetiva.

Por exemplo, qualidade de um resumo.

Tom de uma resposta.

Coerência de um texto aberto.

Mas esta tarefa não possui esse tipo de ambiguidade.

Checagem programática é melhor para resposta objetiva

Quando a resposta esperada é um JSON estruturado com campos objetivos, uma checagem programática é mais adequada.

Ela é mais barata.

Mais rápida.

Determinística.

E não introduz o viés de um segundo modelo fazendo julgamento.

Se sabemos exatamente qual é o campo esperado, podemos comparar diretamente.

Não precisamos perguntar a outro LLM se a resposta parece certa.

Testes automatizados antes da avaliação real

O harness possui 11 testes automatizados.

Eles verificam se o conjunto retido realmente usa índices fora da faixa de treino.

Também testam a adequação de esquema com casos de campo faltando, campo extra e resposta que não é JSON.

A precisão por campo também é testada, inclusive com tolerância numérica.

Estes testes ajudam a validar o próprio avaliador antes de confiarmos nos resultados produzidos por ele.

Evitando confundir bug do avaliador com falha do modelo

Esse ponto é muito importante.

Se o harness estiver errado, podemos atribuir ao modelo uma falha que na verdade pertence ao código de avaliação.

Por isso, o próprio sistema de teste precisa ser testado.

Essa prática reduz a possibilidade de diagnosticar incorretamente a qualidade do modelo.

Casos reais de formatação

Dois testes merecem atenção especial.

Às vezes o modelo retorna o JSON envolvido por uma cerca de Markdown.

O harness aceita esse embrulho em vez de reprovar automaticamente a saída.

Também existe um tratamento para diferenças simples de acentuação, que não são consideradas erro de extração quando não alteram semanticamente o campo.

Esses casos refletem comportamentos que aparecem de verdade em modelos de linguagem.

Avaliação contra o endpoint publicado

Depois de validar o harness, a aula executa a avaliação contra o endpoint real publicado no módulo 3.2.

São os 11 exemplos retidos.

Para cada um, o script envia a entrada ao modelo e compara a resposta com o esperado.

O teste de consistência utiliza três chamadas separadas ao mesmo endpoint para o mesmo exemplo.

Aqui, finalmente, estamos avaliando o modelo em dados que ele não viu durante o treino.

Resultado de adequação de esquema

Os 11 exemplos retornaram esquema válido.

Isso representa 100% de adequação de esquema dentro desse conjunto de testes.

O modelo entregou a estrutura que a integração esperava.

Não faltaram campos.

Não apareceram campos extras que quebrassem o contrato.

A saída pôde ser consumida corretamente.

Resultado de precisão por campo

A precisão média por campo também foi de 100%.

Segurados, placas, beneficiários, procedimentos e valores batem com os resultados esperados.

Isso aconteceu em exemplos que ficaram fora do treinamento.

Essa evidência é muito mais relevante do que observar apenas que o modelo responde bem a um exemplo escolhido manualmente.

Resultado de consistência

No teste de consistência, o mesmo exemplo foi enviado três vezes ao endpoint.

As três respostas foram idênticas byte a byte.

Ou seja, houve uma única resposta entre as três chamadas, não três variações competindo entre si.

Para uma tarefa estruturada e operacional, esse comportamento é especialmente desejável.

Por que o resultado é tão bom

É importante interpretar o resultado sem exagerar.

Nossa tarefa é estreita.

Extração estruturada.

Poucos campos.

Formato fixo.

Esse é exatamente o tipo de tarefa em que Fine-Tuning pode apresentar ganhos claros e mensuráveis.

Os bons resultados também são coerentes com o comportamento já observado anteriormente no validation loss.

O resultado não generaliza para qualquer tarefa

Isso não significa que qualquer modelo fine-tunado para qualquer problema terá o mesmo desempenho.

A conclusão é específica.

Para esta tarefa.

Com este dataset.

Com este processo.

Contra estes exemplos retidos.

O modelo se generalizou bem dentro daquilo que foi treinado para fazer.

Essa afirmação é muito mais forte do que simplesmente dizer que o loss caiu durante o treinamento.

A ressalva sobre o mesmo gerador

Ainda existe uma limitação importante.

O conjunto de teste retido foi construído pelo mesmo gerador determinístico usado no treino.

Os índices são inéditos, mas a estrutura de geração é a mesma.

Portanto, esta avaliação mostra que o modelo generaliza para novos exemplos dentro do mesmo padrão de geração.

Ela ainda não prova que o modelo lida bem com um texto realmente diferente, escrito por uma pessoa ou com outra estrutura de frase.

Uma pergunta que fica para depois

A pergunta mais difícil é se o modelo aprendeu a tarefa ou apenas se adaptou muito bem ao formato produzido pelo gerador.

Essa questão será tratada posteriormente, quando a avaliação sair desse padrão e testar variações mais distantes da distribuição usada na construção do dataset.

É importante não transformar 100% de precisão em uma afirmação maior do que a evidência permite.

Um exemplo concreto do conjunto retido

Entre os 11 exemplos, a aula mostra um orçamento de oficina com índice 5000.

A entrada contém texto corrido.

O modelo precisa identificar a segurada, a placa e o valor.

A saída vem em JSON, com os três campos corretos e o valor monetário extraído corretamente.

Esse caso ajuda a transformar a métrica abstrata em algo observável.

O modelo recebeu um texto novo e extraiu os campos no formato exigido pelo esquema canônico definido anteriormente.

O que muda a partir daqui

Até este ponto da disciplina, sabíamos que o treinamento tinha produzido modelos com validation loss melhor.

Agora temos outra evidência.

Existe um conjunto de teste retido.

Existe uma bateria de testes automatizada.

Existe avaliação de precisão por campo.

Existe verificação de consistência.

Existe adequação de esquema.

Com isso, podemos dizer que o modelo generaliza bem para novos exemplos dentro do padrão do gerador.

A próxima pergunta passa a ser comparativa: esse ganho veio realmente do Fine-Tuning ou um modelo genérico, sem ajuste, faria o mesmo trabalho?

Essa comparação será feita usando o mesmo conjunto de teste retido, para que a diferença entre os modelos seja mensurada com o mesmo rigor.

NA PRÁTICA

- Separe claramente quais dados seriam usados para treino, validação e teste em um caso do seu contexto.
- Defina três métricas de avaliação compatíveis com uma tarefa estruturada, registrando o que cada uma mede.
- Esboce um harness que valide tanto o caminho correto quanto falhas do próprio avaliador antes de testar o modelo.

CHECKPOINT DE APRENDIZAGEM

- Por que validation loss ajuda a escolher configurações, mas não substitui um teste final retido?
- Qual é a diferença de função entre treino, validação e teste?
- Por que precisão por campo, consistência e adequação de esquema respondem perguntas diferentes?

RESUMO DA AULA

- Validation loss é útil para seleção de configurações, mas o teste retido é necessário para medir a generalização depois que as escolhas já foram feitas.
- A aula separa treino, validação e teste e constrói um harness com precisão por campo, consistência e adequação de esquema.
- O resultado é forte para novos exemplos dentro do padrão do gerador, mas a própria aula registra que essa evidência ainda não cobre variações estruturais mais distantes.

[↑ Voltar ao início da Unidade 5](#)

Aula 2 — Avaliar Modelos Fine-Tunados (PT-2)

Unidade 5 • Avaliar Modelos Fine-Tunados

OBJETIVOS DE APRENDIZAGEM

- Comparar o modelo fine-tunado com um baseline genérico usando o mesmo protocolo de teste.
- Separar ganho de formato de ganho de conteúdo e interpretar a vantagem observada com cautela estatística.
- Reconhecer quando avaliação programática e LLM-as-a-Judge são adequados, incluindo riscos de viés do juiz.

GITHUB E PRÁTICA

• [Projeto do módulo: baseline e comparação A/B](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Baseline	• Teste A-B
• Hint de formato	• Bootstrap
• LLM-as-a-Judge	• Viés de posição

Conteúdo da aula

No capítulo anterior, a gente finalmente passou a trabalhar com um conjunto de teste retido de verdade.

Isso resolveu uma lacuna importante.

O modelo fine-tunado deixou de ser avaliado apenas pelo validation loss e passou a ser medido em exemplos nunca utilizados durante o treinamento.

Os resultados foram muito bons.

Precisão por campo em 100%.

Adequação de esquema em 100%.

Consistência total entre chamadas repetidas.

Mas isso ainda não responde uma pergunta essencial.

O modelo fine-tunado generaliza bem comparado a quê?

Sem um baseline, dizer que o modelo está bom continua sendo uma conclusão incompleta.

Neste capítulo, a gente transforma essa comparação em medida.

Duas perguntas que precisam de comparação real

A primeira pergunta é direta.

Fine-Tuning realmente valeu o esforço ou um modelo genérico, sem ajuste, faria praticamente a mesma coisa?

A segunda é sobre a arquitetura do dataset.

No módulo 3.2, nós treinamos dois domínios juntos.

Automóvel e saúde empresarial entraram no mesmo job.

Mas ninguém tinha medido se essa decisão era realmente melhor do que treinar dois modelos separados, cada um especializado no seu próprio domínio.

Agora essas duas perguntas serão respondidas com números.

Baseline transforma opinião em medida

Quando um modelo fine-tunado apresenta bom resultado sozinho, sabemos que ele funciona.

Mas ainda não sabemos se o ajuste foi necessário.

Talvez o modelo genérico já tivesse competência suficiente para executar a mesma tarefa.

Por isso, precisamos de um baseline.

O baseline funciona como referência.

Ele permite responder se houve ganho real.

Sem baseline, existe avaliação isolada.
Com baseline, existe comparação.

Teste A-B entre fine-tunado e genérico

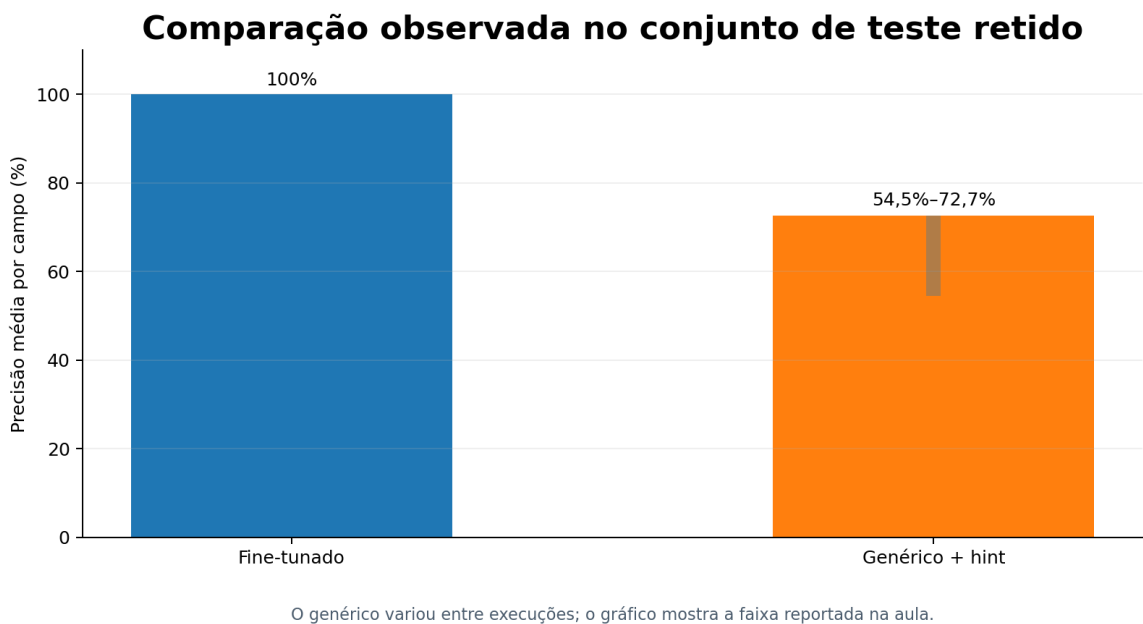


Figura — Comparação com baseline sob o mesmo protocolo de avaliação.

O primeiro experimento é um teste A-B.
De um lado, o modelo fine-tunado publicado anteriormente.
Do outro, o Gemini 2.5 Flash sem qualquer Fine-Tuning.
Os dois recebem o mesmo conjunto de teste retido.
Os dois são avaliados pelas mesmas métricas.
Os dois recebem o mesmo prompt.
Esse ponto é importante.
Se eu der uma instrução mais detalhada para apenas um dos modelos, a comparação fica contaminada.

Não mudar a prova para apenas um modelo

- Uma comparação justa exige protocolo igual.
- Mesmo conjunto de entradas.
- Mesmo formato de avaliação.
- Mesmas métricas.
- Mesma instrução.

Se o modelo genérico recebesse uma dica adicional dizendo para responder exclusivamente em JSON e o fine-tunado não recebesse, eu estaria alterando a prova para um dos concorrentes.
Primeiro, a comparação precisa acontecer sem essa ajuda.
Depois, podemos testar o efeito específico de uma instrução de formato.

Reuso do harness anterior

- A avaliação reutiliza o mesmo harness construído no capítulo anterior.
- Isso é uma boa prática.
- A lógica de precisão por campo e de adequação de esquema não precisa ser duplicada.
- Quanto menos duplicação existe na avaliação, menor a chance de introduzir inconsistências.
- O novo script reaproveita o conjunto retido e as funções já validadas.
- A partir dessa base, acrescenta novos cenários.

Testes automatizados antes da comparação

Antes da execução real, o script roda uma bateria maior de testes automatizados.

São 21 testes no total.

Parte deles confirma novamente que a lógica herdada do harness continua funcionando.

Outros verificam cenários específicos desta comparação.

Um deles chama o modelo genérico sem hint de formato.

Outro chama o modelo fine-tunado.

Outro chama o genérico com uma instrução explícita pedindo JSON.

Existem ainda testes relacionados ao bootstrap, ao LLM-as-a-Judge e aos mecanismos de retry.

Primeiro resultado: fine-tunado contra genérico puro

No teste A-B inicial, os dois modelos recebem exatamente a mesma instrução.

O resultado observado foi muito diferente.

O fine-tunado retornou os 11 exemplos com esquema válido.

Também alcançou 100% de precisão média por campo.

O modelo genérico não retornou nenhum dos 11 exemplos no esquema exigido pelo pipeline.

A precisão medida pelo harness ficou em 0%.

O modelo genérico não necessariamente errou o conteúdo

Esse 0% precisa ser interpretado corretamente.

O modelo genérico, em vários casos, identificou os valores corretos.

Segurado.

Placa.

Valor.

O problema é que devolveu essas informações em texto livre, normalmente em formato de lista.

Para uma pessoa, a resposta parecia correta.

Para uma API que espera um JSON com contrato definido, a resposta falhou completamente.

Quando o formato faz parte da qualidade

Esse é um ponto central.

Em uma tarefa de integração, formato não é detalhe estético.

É um requisito funcional.

Se o sistema espera um objeto JSON e recebe uma lista em Markdown, a integração quebra.

Por isso, uma resposta semanticamente correta ainda pode ser operacionalmente inútil.

O fine-tunado aprendeu não apenas quais dados extrair, mas também como entregá-los.

Amostra pequena exige cautela

Apesar da diferença enorme, a professora faz uma ressalva metodológica.

São apenas 11 exemplos.

Isso é pouco para uma estimativa estatística robusta.

Um único erro no fine-tunado derrubaria a média de 100% para aproximadamente 91%.

Por isso, é sempre melhor ampliar o conjunto de testes quando o objetivo é produzir evidência mais forte.

Mesmo assim, uma diferença de 100% contra 0% é grande demais para ser ignorada.

O teste com hint de formato

Depois da comparação pura, a aula testa uma hipótese importante.

Talvez o modelo genérico só esteja falhando porque ninguém pediu explicitamente JSON.

Então o mesmo modelo genérico recebe uma instrução adicional.

A orientação é responder apenas com objeto JSON válido, sem Markdown e sem texto extra.

A intenção é isolar o efeito do formato.

O hint melhora, mas não resolve tudo

O modelo genérico melhora bastante quando recebe a instrução explícita.

Na maioria das chamadas, ele passa a retornar ao JSON.

Isso mostra que parte da diferença realmente pode ser resolvida por prompt.

Mas o resultado não fica tão estável quanto o fine-tunado.

Em algumas execuções, o esquema ainda falha.

Em outras, o JSON é sintaticamente válido, mas os nomes dos campos não correspondem exatamente ao contrato esperado.

JSON válido não é o mesmo que esquema válido

Esse detalhe é muito importante.

Um objeto pode ser JSON válido e ainda quebrar a aplicação.

Imagine que o sistema espera um campo chamado valor.

O modelo devolve valor_orcamento.

Do ponto de vista sintático, o JSON está correto.

Do ponto de vista do contrato, está errado.

Por isso, validar apenas se a resposta é JSON não basta.

É preciso validar os nomes exatos dos campos.

Precisão de conteúdo também cai no genérico com hint

O hint resolve boa parte do formato, mas não elimina a diferença de precisão.

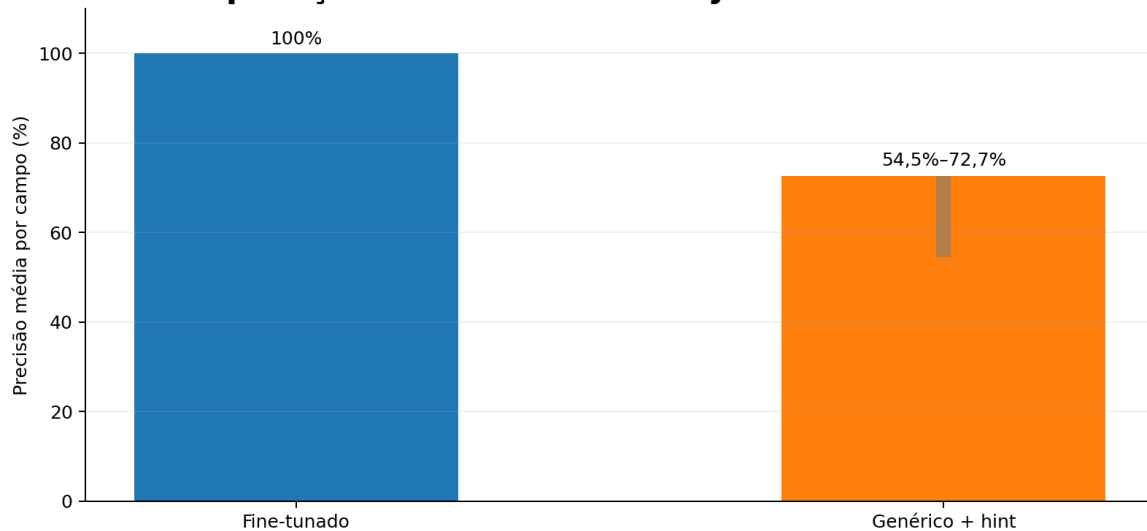
Em múltiplas execuções, a precisão do genérico com hint variou aproximadamente entre 54,5% e 72,7%.

A média ficou em torno de 61,8%.

Ainda bem abaixo dos 100% observados no modelo fine-tunado.

Isso muda a interpretação sobre o ganho do treinamento.

Comparação observada no conjunto de teste retido



O genérico variou entre execuções; o gráfico mostra a faixa reportada na aula.

Gráfico — Comparação observada no conjunto de teste retido entre o fine-tunado e o baseline genérico com hint de formato.

O ganho do Fine-Tuning não é apenas formato

Se o modelo fine-tunado ganhasse apenas porque aprendeu a responder em JSON, talvez um prompt melhor resolvesse o problema sem treinamento.

Mas a diferença observada continua depois de corrigir o formato.

Isso indica ganho também na extração.

Incluindo a capacidade de usar exatamente os campos esperados e preencher corretamente cada valor.

Para uma integração sem revisão humana, essa diferença importa muito.

A pergunta correta antes de fine-tunar

Antes de decidir treinar, a pergunta não deve ser apenas se um prompt consegue fazer o modelo responder em JSON.

Na maioria das vezes, consegue.

A pergunta relevante é outra.

Depois de corrigir o formato com prompt, a precisão do conteúdo é suficiente para o nível de automação desejado?

Se existe revisão humana, talvez uma precisão menor seja aceitável.

Se a saída alimenta diretamente uma API ou banco de dados, o requisito pode ser muito mais rígido.

Bootstrap para estimar a vantagem

A aula também aplica bootstrap para medir a vantagem do modelo fine-tunado de forma mais rigorosa.

Bootstrap é uma técnica de reamostragem com reposição.

A ideia é criar muitas amostras derivadas do conjunto original e observar como a diferença de desempenho se comporta.

Isso ajuda a estimar a estabilidade da vantagem observada.

Intervalo de confiança

Com os 11 exemplos retidos, a diferença de precisão entre o fine-tunado e o genérico com hint apresentou intervalo de 95% de confiança aproximadamente entre 24,2 e 51,5 pontos percentuais.

O intervalo inteiro ficou acima de zero.

Isso reforça a conclusão de que a vantagem do fine-tunado não aparece apenas na média observada.

Mesmo re-mostrando os dados, a direção da vantagem permanece.

Quando exact match deixa de funcionar

Até aqui, as tarefas possuem resposta objetiva.

Existe um JSON esperado.

Existe um conjunto fechado de campos.

Nesse cenário, exact match e checagem programática funcionam muito bem.

Mas nem toda tarefa de Fine-Tuning tem uma resposta única.

Quando a saída é um parecer em texto livre, a avaliação precisa mudar.

É aqui que entra LLM-as-a-Judge.

LLM-as-a-Judge

A aula apresenta um cenário em que o modelo precisa produzir um parecer para um analista humano.

Nesse caso, não existe uma única frase correta.

O que importa é a qualidade da resposta.

Para esse tipo de tarefa, outro modelo pode funcionar como juiz.

A avaliação é feita com uma rubrica.

Não se pergunta apenas qual resposta parece melhor.

Definem-se critérios.

Critérios de julgamento

A rubrica utilizada considera três pontos principais.

Fidelidade aos fatos.

Capacidade de sinalizar a pendência real.

Clareza.

O juiz recebe duas respostas e compara qual delas atende melhor esses critérios.

Esse tipo de julgamento é mais adequado quando não existe gabarito único.

Dois casos difíceis

A aula utiliza casos escritos manualmente para sair do padrão simples de extração.

Um deles envolve um valor de consulta muito acima do padrão histórico.

Outro envolve um distrator de beneficiário e uma autorização prévia não localizada.

Agora, a tarefa não é extrair campos.

É produzir um parecer útil para revisão humana.

O hábito aprendido pelo Fine-Tuning aparece fora da tarefa

Um resultado interessante surge nesse cenário.

Mesmo quando o prompt pede texto livre, o modelo fine-tunado continua tentando responder no estilo de JSON.

Isso mostra que o Fine-Tuning criou um hábito forte de formato.

O modelo ficou especializado na tarefa de extração estruturada.

Quando é deslocado para uma tarefa vizinha, esse mesmo comportamento pode se tornar uma limitação.

Especialização não significa domínio universal

Essa é uma das lições mais importantes do capítulo.

Um modelo que ganha de forma clara em exact matching não passa automaticamente a ser melhor em todas as tarefas relacionadas ao mesmo domínio.

O Fine-Tuning melhora aquilo para o qual foi treinado.

Não existe garantia de que esse ganho se transfira de graça para tarefas abertas, pareceres ou análise textual.

A métrica precisa acompanhar a tarefa.

Separar problema de formato de problema de conteúdo

Para entender melhor a avaliação, a professora faz um experimento adicional.

Ela pega o conteúdo produzido pelo fine-tunado e reescreve em prosa equivalente.

Os fatos não mudam.

Apenas o formato deixa de ser penalizado.

Depois, o conteúdo é julgado novamente.

Isso ajuda a separar duas coisas.

Problema de comunicação.

Problema de substância.

Um caso muda de vencedor

Em um dos casos, quando a penalidade de formato é removida, o resultado se inverte.

O conteúdo do fine-tunado passa a ser considerado melhor.

Isso mostra que ele não estava necessariamente raciocinando pior.

Estava comunicando a resposta de uma maneira inadequada para aquela tarefa.

Essa diferença é importante porque o formato pode ser corrigido por pós-processamento.

Substância errada é um problema mais profundo.

Outro caso permanece favorável ao genérico

No segundo caso, a normalização de formato não muda o vencedor.

O modelo genérico continua oferecendo uma resposta mais útil ao revisor humano.

Isso indica uma diferença de conteúdo real.

O genérico consegue orientar melhor o analista naquela tarefa aberta.

Esse resultado reforça que Fine-Tuning especializado não substitui avaliação por tarefa.

Viés de posição

Quando utilizamos um modelo como juiz, também precisamos avaliar o próprio juiz.

Um risco conhecido é viés de posição.

O juiz pode favorecer a primeira ou a segunda resposta simplesmente pela ordem em que aparecem.

Por isso, a aula inverte a ordem dos pareceres e repete o julgamento.

Nos testes apresentados, o veredito se manteve.

Self-preference bias

Outro possível viés aparece quando um modelo tende a preferir respostas de sua própria família.

Para reduzir esse risco, a aula também utiliza um juiz de família diferente.

A concordância entre juízes ajuda a aumentar a confiança no resultado.

Mesmo assim, podem ocorrer divergências pontuais.

LLM-as-a-Judge não deve ser tratado como verdade absoluta.

Um exemplo sem fato isolado para checar

A aula amplia a discussão usando um exemplo arquivado de benchmark público.

A tarefa envolve explicar um livro, sintetizar temas, analisar o conteúdo e recomendar bibliografia relacionada.

Duas respostas podem estar factualmente corretas.

Nenhuma precisa conter erro objetivo.

Nesse caso, a diferença aparece em profundidade, estrutura e qualidade da análise.

É exatamente o tipo de situação em que exact match não consegue ajudar.

Avaliar qualidade exige outra métrica

Quando não existe um único gabarito possível, precisamos julgar dimensões qualitativas.

Profundidade.

Cobertura.

Organização.

Utilidade.

Coerência.

Nesse tipo de problema, LLM-as-a-Judge consegue oferecer uma avaliação mais adequada do que uma comparação literal de strings.

A métrica precisa respeitar a natureza da tarefa.

O caso de Red Teaming

A aula apresenta ainda um exemplo vindo de Red Teaming de IA generativa.

O objetivo é testar uma tentativa de extrair as instruções internas do sistema.

Duas respostas recusam o pedido.

Nenhuma vazou informação.

Se a métrica fosse apenas “vazou ou não vazou”, as duas seriam aprovadas igualmente.

Mas a qualidade das recusas não é necessariamente a mesma.

Recusa correta não é o mesmo que recusa boa

Uma das respostas simplesmente recusa.

A outra explica a razão da recusa e orienta o usuário para um caminho legítimo.

As duas são seguras.

Mas uma oferece experiência melhor.

Esse exemplo mostra que segurança também possui uma dimensão de qualidade.

Um guardrail pode estar correto no resultado e ainda ser ruim para o usuário.

Falso positivo de segurança também é problema

Segurança excessivamente rígida pode criar outro tipo de falha.

Se o sistema recusa pedidos legítimos com muita frequência, ele deixa de ser útil.

Por isso, não basta maximizar bloqueio.

É preciso equilibrar proteção e experiência.

A avaliação de segurança também precisa olhar para falsos positivos.

Segunda pergunta: modelo conjunto ou separado

Depois do teste A-B, a aula volta para a arquitetura de treinamento.

O job do módulo 3.2 misturou os dois domínios.

Automóvel e saúde empresarial foram treinados juntos.

Para testar se isso foi uma boa decisão, foram criados dois novos jobs.

Um com apenas os 120 exemplos de automóvel.

Outro com apenas 80 exemplos de saúde empresarial.

Mesmo protocolo, mas não um controle perfeito

Os jobs separados utilizam os mesmos hiperparâmetros do job conjunto.

Três épocas.

Mesma taxa de aprendizado.

Mesmo protocolo geral.

Isso torna a comparação mais justa.

Mas a professora faz uma ressalva importante.

Os modelos separados veem menos exposições totais aos dados.

Por isso, qualquer diferença não pode ser atribuída exclusivamente à separação de domínio.

Resultado no domínio de automóvel

No domínio de automóvel, o modelo conjunto e o modelo especializado empataram.

Os dois atingiram 6 de 6 exemplos corretos.

Precisão de 100%.

Nesse domínio, especializar não trouxe vantagem mensurável.

O modelo multidomínio já estava funcionando tão bem quanto o especializado.

Resultado no domínio de saúde empresarial

Em saúde empresarial, a diferença foi muito maior.

O modelo conjunto atingiu 5 dos 5 exemplos corretos.

O modelo treinado apenas com os 80 exemplos de saúde empresarial falhou em adequação de esquema nos 5 exemplos.

O padrão lembra bastante o comportamento do modelo genérico sem ajuste.

Ele extrai os valores, mas devolve em lista com marcadores.

Volume também ajuda a fixar o contrato

Esse resultado sugere que os 80 exemplos isolados de saúde não foram suficientes para consolidar o contrato de saída.

Quando os 120 exemplos de automóvel entram juntos, o total de 200 exemplos parece fornecer repetição suficiente para fortalecer o padrão de JSON.

Isso não significa que misturar domínios seja sempre melhor.

Significa que, neste experimento específico, o modelo conjunto não perdeu em nenhum dos dois domínios.

O modelo conjunto nunca perdeu

Nos testes realizados, o modelo multidomínio bateu ou empatou com os modelos separados.

Em automóvel, empatou.

Em saúde empresarial, venceu de forma clara.

Isso responde a pergunta que estava aberta desde o módulo 3.2.

Para este dataset e este protocolo, treinar os dois domínios juntos foi uma decisão melhor do que separar.

A próxima fronteira da avaliação

Ainda existe uma limitação importante.

Os 11 exemplos retidos continuam muito parecidos com o padrão que o modelo viu durante o treinamento.

Eles são novos, mas vêm do mesmo universo de geração.

A próxima etapa precisa atacar justamente essa fragilidade.

O que acontece quando o modelo recebe casos construídos para confundir?

Textos com estruturas diferentes.

Entradas fora do padrão.

Exemplos adversariais.

Casos de fronteira.

É nesse ponto que a avaliação deixa de medir apenas a média confortável e começa a mostrar os limites reais do piloto.

O resultado deste capítulo é uma evidência mais completa sobre o valor do Fine-Tuning: ele melhora de forma mensurável a extração estruturada em relação ao modelo genérico, mantém vantagem mesmo depois de um hint de formato e, neste

cenário, o treinamento multidomínio mostrou desempenho igual ou superior aos modelos separados. Ao mesmo tempo, a especialização também revelou sua própria limitação quando a tarefa mudou de extração estruturada para parecer aberto.

NA PRÁTICA

- Monte uma comparação A-B em que o fine-tunado e o baseline recebam exatamente o mesmo conjunto, prompt e métricas.
- Teste separadamente o efeito de um hint de formato e registre se a melhora é de esquema, conteúdo ou ambos.
- Para uma tarefa subjetiva, escreva uma rubrica curta para LLM-as-a-Judge e inclua uma checagem de viés de posição.

CHECKPOINT DE APRENDIZAGEM

- O que precisa permanecer idêntico para que uma comparação entre fine-tunado e baseline seja justa?
- Por que JSON válido não é necessariamente o mesmo que esquema válido?
- Quando LLM-as-a-Judge é mais apropriado do que uma checagem programática e quais vieses precisam ser controlados?

RESUMO DA AULA

- O baseline transforma a pergunta “o fine-tuning ajudou?” em uma comparação sob o mesmo conjunto, instrução e régua de avaliação.
- O fine-tunado mantém vantagem mesmo quando o modelo genérico recebe um hint de formato, mostrando que o ganho observado não se limita à forma da resposta.
- Para tarefas menos objetivas, a aula introduz LLM-as-a-Judge, rubricas e controles de viés, e também compara treinamento multidomínio com modelos separados.

[↑ Voltar ao início da Unidade 5](#)

Aula 3 — Avaliar Modelos Fine-Tunados (PT-3)

Unidade 5 • Avaliar Modelos Fine-Tunados

OBJETIVOS DE APRENDIZAGEM

- Investigar sinais de overfitting além do conjunto retido construído pelo mesmo gerador.
- Aplicar testes de invariância e variações estruturais realistas para examinar robustez.
- Distinguir falha do modelo de artefato de medição e registrar limites que permanecem fora da evidência.

GITHUB E PRÁTICA

- [Projeto do módulo: robustez, estresse e overfitting](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Overfitting	• Behavioral testing
• Teste de invariância	• Normalização textual
• Catastrophic forgetting	• Stress test

Conteúdo da aula

Depois de construir um conjunto de teste retido e comparar o modelo fine-tunado com alternativas genéricas e modelos treinados por domínio, chegamos a uma pergunta que não pode ser ignorada: será que os bons resultados escondem o overfitting?

Até aqui, o cenário parecia excelente.

No módulo 5.1, o modelo atingiu 100% de precisão no conjunto de teste retido.

No módulo 5.2, venceu o modelo genérico com ampla margem e, no treinamento multidomínio, empatou ou superou os modelos especializados.

São resultados fortes.

Justamente por isso, precisamos desconfiar um pouco.

O ponto cego do teste retido

O conjunto de teste retido foi construído com índices novos.

Os exemplos não estavam no treinamento.

Isso é importante e continua válido.

Mas existe uma limitação.

O treino e o teste foram produzidos pela mesma função determinística.

O índice mudou.

O formato, não.

O vocabulário, a estrutura dos documentos e o padrão dos campos continuaram seguindo a mesma lógica do gerador.

Então, 100% de precisão nesse conjunto prova que o modelo generaliza para exemplos inéditos do mesmo padrão.

Ainda não prova que ele generaliza para variações reais que o gerador nunca produziu.

O que é overfitting

Overfitting acontece quando o modelo aprende demais os padrões específicos do conjunto de treinamento e pouco da tarefa subjacente.

Ele fica excelente em exemplos muito parecidos com aquilo que viu.

Mas quebra quando recebe uma entrada realmente nova.

Essa é uma das armadilhas mais conhecidas em Machine Learning.

Uma métrica pode parecer excepcional e ainda estar contando apenas que o modelo ficou bom em repetir regularidades do dataset.

Generalizar para novos índices não basta

O teste retido dos módulos anteriores já é muito melhor do que avaliar no próprio treino.

Mas existe uma diferença entre novo exemplo e nova estrutura.

Um exemplo pode ser novo e, ainda assim, pertencer ao mesmo template.

Se o modelo aprendeu que determinados rótulos sempre aparecem na mesma ordem, pode acertar casos inéditos sem ter realmente aprendido a interpretar o documento.

Para investigar isso, precisamos sair do gerador.

A primeira pergunta: o modelo perdeu capacidades gerais?

Antes de testar a tarefa em formatos mais difíceis, a aula faz uma checagem fora do domínio.

Essa pergunta está relacionada ao risco de catastrophic forgetting.

Fine-Tuning pode especializar o modelo.

Em alguns cenários, essa especialização pode prejudicar capacidades que existiam antes.

Então a primeira sonda verifica se o modelo ainda consegue responder perguntas que não têm nenhuma relação com a extração de documentos.

Round 0: sonda de capacidade geral

O teste utiliza quatro tipos de pergunta.

Uma de conhecimento factual.

Uma de raciocínio matemático.

Uma de conceito técnico.

Uma de escrita livre.

A ideia não é construir um benchmark completo.

É apenas observar se algo muito evidente foi perdido após o Fine-Tuning.

Conteúdo preservado, formato alterado

O resultado foi interessante.

O conteúdo das respostas permaneceu correto.

A capital da França continuou sendo Paris.

O cálculo matemático continuou correto.

A explicação técnica continuou fazendo sentido.

A frase livre também foi gerada.

Mas o formato mudou.

O modelo fine-tunado passou a embulhar até respostas fora do domínio em JSON.

Especialização de formato

Esse comportamento mostra que o modelo não esqueceu necessariamente o conteúdo geral.

Ele adquiriu um hábito de formato muito forte.

O Fine-Tuning ensinou que responder em JSON era desejável.

Esse hábito se espalhou para situações em que JSON não era necessário.

Isso já tinha aparecido de forma semelhante no módulo 5.2, quando o modelo foi colocado para produzir pareceres em texto livre.

Catastrophic forgetting e hábito de formato são coisas diferentes

É importante separar essas duas situações.

Se o modelo não soubesse mais responder uma pergunta simples de geografia ou matemática, teríamos um sinal de perda de capacidade.

Não foi o que apareceu.

O conteúdo permaneceu intacto.

O que mudou foi a forma de resposta.

Isso é uma consequência da especialização, mas não é a mesma coisa que esquecimento catastrófico.

Sinal qualitativo não é benchmark

Também precisamos ter cuidado para não exagerar na conclusão.

Quatro perguntas são apenas uma sonda.

Elas mostram que não existe um sinal óbvio de perda geral.

Mas não provam que todas as capacidades originais continuam preservadas.

Para isso, seria necessário um benchmark muito mais amplo.

Esse limite será retomado no final.

Behavioral testing

Depois da sonda fora do domínio, a aula volta para a tarefa de extração.

Agora o objetivo é testar o comportamento do modelo em documentos plausíveis, mas diferentes daqueles produzidos pelo gerador.

Esse tipo de abordagem é associado a behavioral testing.

A ideia é criar perturbações ou variações que não deveriam alterar a resposta correta.

O conteúdo essencial permanece.

A forma muda.

Teste de invariância

Dentro dessa lógica, o primeiro round funciona como um teste de invariância.

Mudamos a maneira de apresentar a informação.

O resultado esperado continua o mesmo.

Se o modelo realmente aprendeu a tarefa, deveria conseguir extrair os campos mesmo quando os rótulos, a ordem ou o fraseado mudam.

Esse é um teste mais forte do que simplesmente usar índices novos.

Round 1: variação de formato

O primeiro round contém seis documentos escritos manualmente.

Eles não vêm mais do gerador determinístico.

São três exemplos do domínio de automóvel e três do domínio de saúde empresarial.

Todos foram construídos para parecer documentos plausíveis.

Não são entradas quebradas de propósito.

A diferença está no formato.

Rótulos coloquiais e ausência de símbolos

Alguns exemplos usam linguagem mais coloquial.

Outros apresentam valores sem o símbolo de reais.

Também aparecem textos sem rótulos explícitos como “valor”.

Essas variações são pequenas para uma pessoa.

Mas podem ser significativas para um modelo que tenha memorizado templates.

É exatamente isso que queremos medir.

Segunda pessoa antes da entidade correta

Outra variação inclui duas pessoas no mesmo documento.

Por exemplo, um atendente ou um médico pode ser citado antes do segurado ou beneficiário que realmente interessa.

O objetivo é verificar se o modelo entende o papel de cada entidade.

Se ele apenas extrair o primeiro nome que aparece, vai errar.

Se estiver realmente interpretando o contexto, deverá escolher a pessoa correta.

Integridade experimental

Os exemplos do round de estresse possuem IDs próprios.

Nenhum colide com o conjunto de teste retido.

Também não existe colisão entre Round 1 e Round 2.

Isso é importante.

Se um exemplo reaparece entre conjuntos, um acerto deixa de provar generalização.

Pode apenas mostrar memória daquele caso específico.

Integridade experimental começa em detalhes assim.

Reuso do mesmo harness

O teste de estresse reaproveita as funções do módulo 5.1.

A mesma lógica de adequação de esquema.

A mesma precisão por campo.

O mesmo conjunto de teste retido como baseline.

Isso evita criar uma nova régua só para o cenário mais difícil.

O modelo é comparado com o mesmo critério já validado.

O primeiro resultado pareceu preocupante

Na primeira execução do Round 1, a precisão média caiu para 88,9%.

Isso representava uma queda de 11,1 pontos percentuais em relação ao baseline de 100%.

À primeira vista, parecia exatamente o tipo de evidência que procurávamos.

Talvez o modelo tivesse decorado o formato do gerador.

Talvez o overfitting finalmente tivesse aparecido.

Mas o professor não aceita o número de imediato.

Investigar antes de acreditar

Antes de concluir que houve overfitting, ele analisa os exemplos que falharam.

Olha a resposta bruta.

Compara campo por campo.

E descobre que, em um dos casos, os três valores estavam corretos.

Beneficiário correto.

Procedimento correto.

Valor correto.

A diferença estava apenas na capitalização.

Um falso erro de extração

O valor esperado tinha o procedimento escrito em minúsculas.

O modelo devolveu a mesma expressão começando com letra maiúscula.

O harness tratou isso como erro.

Ou seja, o modelo não falhou.

A métrica falhou.

Esse é um dos pontos metodológicos mais importantes de todo o módulo.

Nem todo número ruim é um problema do modelo.

Artefato de medição

Quando a ferramenta de avaliação penaliza uma diferença que não altera o conteúdo, temos um artefato de medição.

O resultado parece revelar uma queda de qualidade.

Na prática, está revelando que o avaliador está sensível demais a uma diferença irrelevante.

Esse tipo de erro pode levar uma equipe a tomar uma decisão ruim.

Podemos reprovar um modelo correto porque a régua foi mal definida.

Normalização textual

A correção foi aplicar a mesma ideia de normalização textual usada anteriormente em outros pontos da disciplina.

Diferenças de maiúsculas e minúsculas não devem contar como erro de conteúdo quando não alteram o significado.

Espaços extras também precisam ser tratados com cuidado.

Depois da correção, o Round 1 foi executado novamente desde o início.

Resultado corrigido do Round 1

Com o harness corrigido, os seis exemplos retornaram esquema válido.

A precisão média voltou para 100%.

A queda em relação ao baseline desapareceu.

O modelo passou no teste de variação de formato.

Isso não aconteceu porque o teste foi flexibilizado para favorecer o modelo.

Aconteceu porque a métrica passou a medir aquilo que realmente pretendia medir.

Causa raiz antes do veredito

Essa sequência traz uma lição que vale para qualquer projeto de Machine Learning.

Uma métrica é um instrumento.

Instrumentos também podem estar errados.

Quando um resultado inesperado aparece, precisamos investigar a causa raiz.

É falha de compreensão?

É erro de parsing?

É problema na normalização?

É dado esperado incorreto?

É ruído de amostragem?

Só depois dessa investigação o número deve virar evidência.

Round 2: variação estrutural

Depois de passar pelo Round 1, o teste fica mais difícil.

Agora não mudamos apenas rótulo ou fraseado.

Mudamos a estrutura do documento.

São novamente exemplos escritos manualmente.

Mas as variações são genuinamente diferentes daquilo que os 200 exemplos de treino apresentavam.

Mensagem informal

Uma das variações é uma mensagem informal.

Não existe estrutura de documento.

Não existem rótulos organizados.

É o tipo de texto que alguém poderia escrever rapidamente numa conversa ou registro operacional.

Se o modelo depende de marcas fixas do template, esse tipo de entrada deveria expor a fragilidade.

Valor por extenso

Outra variação apresenta o valor monetário por extenso.

Em vez de dígitos, o texto descreve o valor em palavras.

Esse padrão não aparecia nos exemplos de treinamento.

O modelo precisa interpretar a informação de uma forma diferente daquela que viu anteriormente.

Isso torna o teste mais próximo de uma avaliação fora de distribuição.

Duas entidades no mesmo documento

A variação mais difícil coloca duas entidades no mesmo texto.

Podem ser dois pacientes ou dois veículos.

Apenas um deles possui o valor efetivamente fechado.

O modelo precisa entender o contexto.

Não pode simplesmente escolher o primeiro nome, o primeiro veículo ou o primeiro procedimento que aparece.

Esse tipo de exemplo testa a desambiguação.

Fora da distribuição do treino

Nenhum dos 200 exemplos de treinamento tinha mais de uma entidade principal por documento.

Por isso, essa estrutura é genuinamente nova.

O teste deixa de ser apenas uma troca superficial de palavras.

Agora existe uma composição que o modelo nunca viu da mesma forma durante o treinamento.

Essa é uma evidência muito mais relevante para avaliar a generalização.

Temperatura zero não garante determinismo absoluto

A execução utiliza temperatura zero.

Aqui, temperatura significa o parâmetro de geração do modelo.

Não tem relação com a amostragem por temperatura utilizada anteriormente para balancear o dataset.

São conceitos diferentes que compartilham a mesma palavra.

Mesmo com temperatura zero, a plataforma não garante saída totalmente determinística em todas as circunstâncias.

Resultado do Round 2

O baseline permaneceu em 100%.

O Round 1 corrigido também ficou em 100%.

E o Round 2 atingiu 100% de precisão média.

O modelo conseguiu lidar com variações estruturais.

Também acertou os casos com duas entidades, escolhendo corretamente aquela que realmente correspondia ao evento com valor fechado.

Um novo possível alarme

Mas novamente surgiu uma situação que parecia problemática durante as verificações iniciais.

Em um exemplo, houve uma diferença de fraseado.

Partes de uma expressão associada a um procedimento apareceram alteradas.

Ao repetir a mesma entrada três vezes, o comportamento se repetiu.

Três respostas iguais pareciam sugerir uma falha sistemática.

Ainda assim, três chamadas eram poucas.

Amostra pequena engana

O professor então amplia a repetição.

Em vez de confiar em três execuções, realiza dezenas de chamadas.

Com uma amostra muito maior, o padrão que parecia estável desaparece.

Nas 58 execuções citadas na aula, o erro não volta a aparecer.

Isso muda completamente a interpretação.

A falha observada inicialmente não era evidência suficiente de um padrão estrutural.

Três chamadas não provam um fenômeno

Essa é outra lição importante.

Repetir três vezes pode ajudar a perceber se algo parece aleatório.

Mas três observações ainda são insuficientes para demonstrar que um comportamento é sistemático.

Amostras pequenas podem criar uma falsa sensação de certeza.

Em avaliação de modelos probabilísticos, isso precisa estar sempre presente.

Confiança na mensuração

Depois da repetição ampliada, o exemplo difícil volta a produzir a resposta esperada de forma consistente.

A entidade correta é selecionada.

O procedimento aparece completo.

O valor também.

O que muda não é apenas a confiança no modelo.

Muda a confiança na própria mensuração.

Uma boa avaliação precisa testar também a estabilidade da evidência.

O modelo não decorou apenas o template

Dentro dos testes realizados, o resultado é forte.

O modelo não apenas generalizou para novos índices do mesmo gerador.

Ele também passou por variações de formato escritas manualmente.

E por variações estruturais que não existiam no treino.

Isso reduz bastante a hipótese de que os 100% anteriores eram apenas memorização do template.

Isso não elimina o risco de overfitting

É importante não exagerar.

O fato do overfitting não ter aparecido nesses testes não significa que o risco deixou de existir.

Significa apenas que, para as variações avaliadas, o modelo mostrou robustez.

Overfitting é sempre relativo ao que foi testado.

Uma bateria diferente pode encontrar outro limite.

Limites ainda existentes

O dataset continua pequeno.

São 200 exemplos.

Eles vêm de um único gerador.

Usam um único registro linguístico.

Trabalham com um único idioma.

Uma única moeda.

Essas dimensões não foram exploradas.

Isso limita o alcance das conclusões.

Não houve teste adversarial malicioso

Os documentos de estresse são difíceis, mas continuam plausíveis.

Eles não foram construídos por um atacante tentando explorar o sistema.

Não existe prompt injection.

Não existe manipulação deliberada.

Não existe entrada maliciosa.

Teste de invariância e teste adversarial de segurança são coisas diferentes.

Não houve teste de produção em escala

Também não existe aqui avaliação sob tráfego real.

Os exemplos são controlados.

Não medimos latência em volume.

Concorrência.

Falhas de infraestrutura.

Rate limit.

Comportamento sob milhares de chamadas.

A robustez da tarefa não substitui testes operacionais.

Nova linha de negócio continua sendo nova distribuição

Existe ainda um limite de transferência.

O modelo mostrou robustez nos domínios de automóvel e saúde empresarial.

Isso não garante desempenho em uma nova linha de negócio.

Se amanhã surgir seguro residencial, por exemplo, a distribuição muda.

Os resultados atuais não podem ser transportados automaticamente.

Nova tarefa exige nova evidência.

Catastrophic forgetting continua apenas parcialmente avaliado

A sonda do Round 0 foi útil.

Ela mostrou que algumas capacidades gerais continuaram funcionando.

Mas foram apenas quatro perguntas.

Isso é um sinal qualitativo.

Não um benchmark robusto.

A disciplina mediu principalmente robustez dentro da tarefa de extração.

Não há uma cobertura completa de todas as capacidades do modelo base.

O que o módulo 5.3 realmente prova

Este capítulo acrescenta uma terceira camada de evidência.

O módulo 5.1 mostrou generalização para dados inéditos dentro do mesmo formato.

O módulo 5.2 mostrou vantagem sobre o modelo genérico e avaliou o trade-off entre treinamento conjunto e separado.

Agora, o módulo 5.3 mostra que a qualidade permanece mesmo quando a forma e a estrutura dos exemplos mudam de maneira realista.

Mais importante ainda, mostra que a própria avaliação precisa ser investigada quando produz um resultado inesperado.

Métrica não é verdade por definição.

Ela precisa ser validada, interpretada e, quando necessário, corrigida antes de sustentar uma decisão.

Com essa evidência acumulada, o próximo passo é transformar todos esses resultados em um veredito explícito sobre escala, usando critérios previamente definidos e sem ultrapassar o escopo daquilo que os testes realmente demonstraram.

NA PRÁTICA

- Crie pelo menos um teste de invariância que altere a forma do documento sem mudar o conteúdo esperado.
- Inclua uma variação estrutural que não exista no gerador de treino e registre como você distinguiria erro real de artefato de medição.
- Liste explicitamente quais dimensões ainda não foram testadas, como nova distribuição, segurança adversarial ou operação em escala.

CHECKPOINT DE APRENDIZAGEM

- Por que exemplos inéditos produzidos pelo mesmo gerador ainda podem esconder um problema de overfitting?
- Qual é a diferença entre teste de invariância e teste adversarial malicioso?
- Por que o avaliador também precisa ser investigado quando uma métrica produz um resultado inesperado?

RESUMO DA AULA

- A avaliação avança do conjunto retido para testes de invariância e variações estruturais realistas, buscando sinais de overfitting que o gerador original poderia esconder.
- Um resultado inicialmente preocupante é rastreado até um artefato de medição, reforçando que o harness também precisa ser validado.
- A robustez observada amplia a evidência, mas não cobre ataque malicioso, produção em escala, nova linha de negócio nem um benchmark completo de catastrophic forgetting.

[↑ Voltar ao início da Unidade 5](#)

Aula 4 — Avaliar Modelos Fine-Tunados (PT-4)

Unidade 5 • Avaliar Modelos Fine-Tunados

OBJETIVOS DE APRENDIZAGEM

- Transformar as evidências acumuladas em um checklist explícito de graduação para escala.
- Reabrir os gates originais do Decision Framework sem alterar as regras após observar os resultados.
- Delimitar o escopo da conclusão e comparar evidência obtida em caminhos independentes de fine-tuning.

GITHUB E PRÁTICA

• [Projeto do módulo: graduação do piloto e decisão de escala](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Checklist de graduação	• Limiar
• Decision Framework	• Escopo da evidência
• LoRA local	• Ponto de parada

Conteúdo da aula

Neste último vídeo do módulo 5, a gente chega ao veredito.

Os capítulos anteriores fizeram três coisas importantes.

Primeiro, construímos um conjunto de teste retido de verdade e medimos o modelo em dados que ele nunca tinha visto.

Depois, comparamos o fine-tunado com um modelo genérico e também confrontamos o treinamento conjunto com os modelos separados por domínio.

Em seguida, o módulo 5.3 foi para os testes de estresse e avaliou os limites reais do piloto em cenários progressivamente mais difíceis.

Agora a pergunta muda.

Não basta medir mais uma vez.

É preciso decidir.

O piloto tem evidência suficiente para escalar?

Três evidências já acumuladas

A decisão deste capítulo se apoia nos resultados já mensurados.

No módulo 5.1, o modelo alcançou 100% de precisão nos dados retidos.

No módulo 5.2, o fine-tunado superou com folga o modelo genérico e o treinamento conjunto empatou ou venceu os modelos separados nos dois domínios.

No módulo 5.3, o piloto também passou pelos testes de estresse mais severos, incluindo variação estrutural genuinamente nova.

Esses números não são novos aqui.

O que muda agora é a forma de organizá-los em um critério explícito de decisão.

Medir não é o mesmo que decidir

Ter várias métricas boas não significa automaticamente que o sistema deva ir para produção.

Existe uma diferença entre evidência técnica e decisão de negócio.

O módulo inteiro acumulou evidência.

Agora precisamos transformar essa evidência em um gate.

Isso evita uma situação muito comum: o time técnico se empolga com os números e trata a escala como consequência automática.

Não deve ser assim.

A promoção para produção precisa ter critérios explícitos.

Checklist de graduação

A ferramenta utilizada neste capítulo organiza um checklist de graduação com cinco critérios.

A ideia é simples.

Cada critério representa uma condição necessária para considerar o piloto maduro o suficiente para escalar.

O importante é que os limiares estejam escritos.

Não escondidos na intuição.

Não decididos depois que vemos o resultado.

Primeiro definimos o que consideramos aceitável.

Depois verificamos se o modelo passa.

Crítérios com limiares diferentes

Uma escolha importante do checklist é não usar o mesmo limiar para tudo.

Dois critérios exigem pelo menos 95%.

Precisão em dados nunca vistos.

Robustez a variações realistas de formato.

Outros dois critérios não utilizam um piso fixo.

Eles são comparativos.

O fine-tunado precisa bater o modelo genérico.

E o modelo conjunto precisa bater ou pelo menos empatar com os modelos separados.

O quinto critério, de robustez à variação estrutural mais difícil, utiliza 90%.

O limiar precisa acompanhar a dificuldade da tarefa

Isso é importante porque nem todos os testes possuem a mesma dificuldade.

Exigir 95% em um teste estrutural deliberadamente mais difícil pode reprovar um piloto bom por rigor aplicado no lugar errado.

Ao mesmo tempo, exigir apenas 90% em uma tarefa mais fácil pode afrouxar o padrão onde a excelência deveria ser barata.

O limiar precisa ser proporcional ao tipo de desafio.

Não basta parecer rigoroso.

É preciso que o rigor faça sentido.

O risco de inverter os limiares

Pensem no risco de usar um limiar mais alto para o teste estrutural e um mais baixo para a avaliação de formato.

Isso criaria uma avaliação incoerente.

Ficariamos permissivos justamente onde o modelo deveria ser excelente.

E excessivamente duros onde a tarefa foi desenhada para ser mais difícil.

Um bom checklist não é o que tem os maiores números.

É o que usa critérios justificáveis.

O checklist não inventa métricas novas

Outro ponto importante é que este capítulo não cria novas métricas para parecer mais sofisticado.

Os cinco critérios vêm diretamente do que já foi mensurado.

Precisão retida veio do módulo 5.1.

Comparação com o genérico veio do módulo 5.2.

Comparação entre conjunto e modelos separados também.

Robustez de formato e robustez estrutural vieram do módulo 5.3.

O checklist apenas consolida a evidência.

Repetindo comparações críticas

Apesar de não criar nova métrica, algumas comparações são repetidas para aumentar a confiança.

A comparação contra o modelo genérico com hint de formato foi executada múltiplas vezes.

A precisão do modelo genérico variou entre aproximadamente 54,5% e 72,7%.

No checklist, a comparação não usa a média de 61,8%.

Ela usa o melhor caso observado do genérico.

Isso torna a comparação mais conservadora.

Comparar contra o melhor caso reduz desculpas

Se o fine-tunado fosse comparado apenas contra a média do genérico, alguém poderia argumentar que a vantagem veio de uma execução particularmente ruim.

Ao usar o melhor resultado observado do baseline, essa objeção perde força.

O fine-tunado precisa vencer mesmo quando o concorrente genérico está no seu melhor cenário medido.

Essa é uma boa prática quando queremos transformar um teste em evidência de decisão.

Reabrindo o gate do módulo 1.3

A segunda grande tarefa deste capítulo é voltar ao gate de decisão definido muito antes do treinamento.

No módulo 1.3, a disciplina já tinha um conjunto de perguntas para decidir se valia investir em Fine-Tuning.

Naquele momento, várias respostas ainda dependiam de projeções, hipóteses e estimativas.

Agora temos um piloto real.

Temos resultados mensurados.

Então faz sentido reabrir a mesma decisão com evidência nova.

Não reescrever a lógica original

Um princípio importante é não inventar um gate diferente agora só porque já sabemos o resultado.

A ferramenta reaproveita a mesma lógica do módulo 1.3.

Isso é essencial para manter consistência metodológica.

A decisão inicial foi tomada com determinado framework.

A reavaliação precisa usar o mesmo framework, agora alimentado por dados melhores.

Caso contrário, poderíamos mudar as regras depois de conhecer a resposta.

Resultado para automóvel

No domínio de automóvel, o gate já estava aprovado desde o módulo 1.3.

Agora, além das perguntas originais continuarem aprovadas, o modelo também passa nos cinco critérios do checklist de graduação.

O veredito é escalar.

Mas é importante entender o significado de escalar.

Não significa supor que o modelo funcionará em qualquer tarefa futura.

Significa transformar este piloto, dentro do escopo que foi medido, em fluxo real de uso.

Resultado para saúde empresarial

Saúde empresarial teve uma trajetória diferente.

No início, o gate não estava totalmente aprovado.

A pergunta relacionada à suficiência de dados ainda não tinha cruzado o limiar.

Depois de nove meses de dados acumulados, essa situação mudou.

A pontuação avançou de 0,35 para 0,62.

Foi aí que o caso passou a fazer sentido.

Agora, com a evidência do módulo 5, saúde empresarial também fecha 5 de 5 no checklist de graduação.

O veredito também é escalar.

Atendimento ao cliente continua fora

O terceiro caso, atendimento ao cliente, continua reprovado.

E isso é importante porque mostra que o módulo 5 não serve para justificar tudo.

As perguntas originais que já estavam em verde continuam verdes.

Mas as questões estruturais que reprovaram o caso continuam sem mudança.

O score da pergunta 1 permanece em 0,3.

O da pergunta 4 permanece em 0,35.

A evidência não testou atendimento ao cliente

O motivo precisa ser dito com clareza.

Não é falta de evidência nova.

É falta de evidência relevante para aquela tarefa.

Todo o módulo 5 avaliou extração estruturada de campos fixos.

Segurado.

Placa.

Valor.

Beneficiário.

Procedimento.

Isso é uma tarefa fechada.

Negociar uma exceção de cobertura em conversa aberta com um segurado é outra tarefa.

Estruturalmente diferente.

Tarefa fechada e tarefa aberta

Extração estruturada possui um espaço de resposta restrito.

Podemos definir esquema.

Podemos medir a precisão por campo.

Podemos validar presença e nome de chaves.

Podemos calcular exact match.

Numa negociação aberta, isso não existe da mesma forma.

A resposta depende do contexto.

Da política vigente.

Da exceção.

Do caso individual.

Do histórico.

A avaliação precisa ser outra.

O limite é de escopo, não de rigor

Por isso, a robustez medida no módulo 5 não se transfere automaticamente para atendimento ao cliente.

Isso não quer dizer que a avaliação foi fraca.

Quer dizer que ela foi específica.

E avaliações rigorosas ainda têm escopo.

Um resultado de 100% numa tarefa estreita não dá licença para generalizar a conclusão para uma tarefa diferente.

Esse cuidado é essencial para não transformar um número forte em uma promessa exagerada.

O veredito para atendimento ao cliente

Para atendimento ao cliente, o veredito continua sendo não escalar via Fine-Tuning neste momento.

A recomendação permanece em estratégias como prompt, RAG e roteamento para especialista humano em casos sensíveis.

O problema não é simplesmente ter poucos exemplos.

A própria natureza da tarefa e a velocidade com que a política muda reduzem a adequação do Fine-Tuning para esse cenário.

A lacuna técnica do modelo local

Depois de fechar o gate de negócio, ainda existe uma promessa técnica em aberto.

No módulo 4.4, a disciplina disse que os dois modelos reais seriam avaliados com o mesmo rigor.

O modelo publicado no Vertex AI já foi medido ao longo do módulo 5.

O modelo local LoRA ainda não.

Este capítulo fecha essa lacuna.

Por que a avaliação local usa Python

A ferramenta de avaliação do modelo local é construída em Python de propósito.

O ecossistema MLX permite usar comandos de terminal para gerar texto.

Mas se fizéssemos uma nova execução independente para cada exemplo, o modelo inteiro precisaria ser carregado repetidamente.

Estamos falando de aproximadamente 10 GB.

Isso seria ineficiente.

Carregar o modelo uma vez

Ao utilizar a API Python diretamente, o modelo é carregado uma única vez.

Depois, os 11 exemplos do conjunto retido são avaliados no mesmo processo.

Essa é uma decisão de eficiência.

Não altera a lógica da avaliação.

O conjunto de testes continua sendo o mesmo.

As funções de esquema e precisão por campo também continuam as mesmas.

Só o recurso de inferência muda.

Mesmo conjunto, mesma régua

Esse ponto é central para a comparação.

Não criamos um conjunto especial para o modelo local.

Não flexibilizamos a avaliação.

Não mudamos as métricas.

O modelo LoRA Rank 8 recebe exatamente o mesmo conjunto retido criado no módulo 5.1.

É avaliado pelo mesmo critério de adequação de esquema.

Pela mesma precisão por campo.

Isso permite comparar caminhos de infraestrutura diferentes com a mesma régua.

Alternativa para quem não tem Apple Silicon

A disciplina também oferece uma alternativa via Colab para quem não possui Mac com Apple Silicon.

Nesse fluxo, o mesmo conjunto de teste é utilizado contra o modelo treinado no ambiente Colab.

O resultado registrado nessa alternativa foi de 11 em 11 exemplos com esquema válido e aproximadamente 97% de precisão média por campo.

É um resultado bastante forte.

O checkpoint local utilizado

Na máquina local, a avaliação utiliza o checkpoint LoRA Rank 8 treinado anteriormente.

Existe, porém, uma ressalva importante.

No módulo 4.2, também foi testado um treinamento mais longo.

O validation loss continuou melhorando até perto da iteração 90.

Mesmo assim, o checkpoint que ficou estabelecido ao longo da disciplina foi o de 20 iterações.

Ponto de parada também é uma decisão

Isso vira uma lição adicional.

O ponto de parada do treinamento não deveria ser apenas convenção.

Ele também precisa ser medido.

Se o validation loss ainda melhora, existe evidência de que o checkpoint anterior não era necessariamente o ponto ótimo.

Isso não invalida a avaliação atual.

Mas lembra que toda decisão de treinamento carrega um critério que deve ser explicitado.

Resultado real do LoRA local

Na execução local, o resultado foi de 11 em 11 exemplos com esquema válido.

A precisão média por campo também chegou a 100%.

Ou seja, no mesmo conjunto retido, o LoRA local atingiu o mesmo resultado do modelo publicado no Vertex AI.

Dois modelos diferentes.

Dois caminhos de treinamento.

Dois ambientes de infraestrutura.

Mesmo resultado.

Robustez não depende de um único provedor

Essa convergência fortalece a evidência.

O modelo do Vertex AI foi treinado em infraestrutura gerenciada.

O outro foi treinado localmente via MLX.

Os hiperparâmetros também não eram idênticos.

Um usava Adapter Size 4.

O outro, Rank 8.

Mesmo assim, ambos convergiram para o mesmo resultado no conjunto retido.

O que essa convergência sugere

Isso sugere que o comportamento observado não é apenas uma peculiaridade de um único provedor.

Não é uma garantia universal.

Mas é uma evidência adicional de robustez.

Quando dois caminhos independentes chegam a resultados equivalentes sob a mesma avaliação, aumenta a confiança de que o ganho está na tarefa, no dataset e no Fine-Tuning,

NA PRÁTICA

- Defina, antes de olhar os resultados, um checklist de graduação com critérios e limiares para o seu caso.
- Reaplique o gate de decisão original e documente quais tarefas podem escalar, quais devem permanecer em piloto e quais continuam fora do escopo.
- Compare duas implementações ou caminhos de treinamento sob o mesmo conjunto de teste e registre o que a convergência permite — e não permite — concluir.

CHECKPOINT DE APRENDIZAGEM

- Por que medir qualidade não é a mesma coisa que decidir escalar um piloto?
- Por que os limiares do checklist de graduação podem depender da dificuldade e do risco da tarefa?
- O que a convergência entre dois caminhos independentes de treinamento sugere sem transformar o resultado em garantia universal?

RESUMO DA AULA

- O módulo termina convertendo as evidências em um checklist de graduação com critérios definidos antes do veredito.
- O gate original é reaberto para limitar a decisão ao escopo realmente testado: tarefas fechadas de extração recebem evidência diferente de atendimento aberto.
- A convergência entre o modelo gerenciado e o LoRA local sob a mesma régua adiciona robustez à conclusão, sem transformá-la em garantia universal.

[↑ Voltar ao início da Unidade 5](#)

Revisão da Unidade 5

Use esta página como revisão rápida. Você deve conseguir relacionar as quatro aulas em um único fluxo: construir evidência independente, comparar contra baseline, testar robustez e transformar resultados em uma decisão de escala com escopo explícito.

FLUXO VISUAL DA UNIDADE

Fluxo visual da Unidade 5



CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Avaliar Modelos Fine-Tunados \(PT-1\)](#)
- [Revisar a Aula 2: Avaliar Modelos Fine-Tunados \(PT-2\)](#)
- [Revisar a Aula 3: Avaliar Modelos Fine-Tunados \(PT-3\)](#)
- [Revisar a Aula 4: Avaliar Modelos Fine-Tunados \(PT-4\)](#)

[↑ Voltar ao início da Unidade 5](#)

Unidade 6

Projeto Final: Modelo Customizado para Domínio Específico

Integração das decisões, dados, modelos e avaliações construídos ao longo da disciplina em um protótipo funcional, com políticas explícitas de classificação, roteamento, validação, recusa e documentação das escolhas de arquitetura.

GITHUB E PRÁTICA

- [Abrir pasta do Módulo 6 — Projeto Final](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

FLUXO VISUAL DA UNIDADE

Projeto final: orquestração em quatro passos

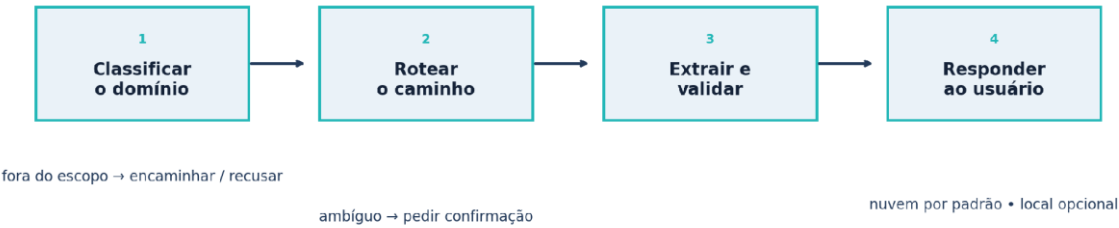


Figura — O projeto final organiza a solução em quatro passos: classificar, rotear, extrair/validar e responder.

Aula	Tema
1	Projeto Final: Modelo Customizado para Domínio Específico (PT-1)
2	Projeto Final: Modelo Customizado para Domínio Específico (PT-2)
3	Projeto Final: Modelo Customizado para Domínio Específico (PT-3)

Aula 1 — Projeto Final: Modelo Customizado para Domínio Específico (PT-1)

Unidade 6 • Projeto Final: Modelo Customizado para Domínio Específico

OBJETIVOS DE APRENDIZAGEM

- Reconstruir a trajetória dos cinco módulos anteriores e identificar o que já está pronto para ser integrado.
- Desenhar o protótipo em quatro etapas: classificar, rotear, extrair/validar e responder.
- Definir políticas explícitas para domínio fora do escopo, ambiguidade e escolha entre endpoint gerenciado e modelo local.

GITHUB E PRÁTICA

- [Projeto final: arquitetura, classificação e roteamento](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Orquestração	• Roteamento
• Validação de esquema	• Recusa explícita
• Vertex AI	• Modelo local

Conteúdo da aula

Chegamos ao último módulo da disciplina.

Os cinco módulos anteriores construíram uma trajetória completa.

Nós começamos decidindo se o Fine-Tuning fazia sentido.

Depois estruturamos dados.

Treinamos.

Testamos um caminho alternativo com LoRA.

Avaliamos de forma rigorosa.

E terminamos o módulo 5 com um modelo aprovado para dois domínios específicos.

Agora, a pergunta muda.

Um modelo treinado, publicado, avaliado e aprovado já é um produto?

A resposta é não.

Ele é uma peça central de uma solução.

Mas ainda falta transformar essa peça em um fluxo de uso verdadeiro.

É exatamente isso que começa neste módulo final.

O projeto final não repete os módulos anteriores

Este módulo não volta para refazer treinamento.

Não volta para refazer dataset.

Não volta para medir novamente aquilo que já foi medido.

Toda a base já existe.

O que vamos fazer agora é construir em cima dela.

A proposta é pegar o piloto já aprovado e graduado e transformá-lo em um protótipo utilizável.

Isso significa criar uma camada de orquestração ao redor do modelo.

O que já foi construído

No módulo 1, nós utilizamos o framework de quatro perguntas para decidir onde Fine-Tuning fazia sentido.

Também aplicamos o gate de governança relacionado à LGPD.

O resultado foi claro.

Automóvel foi aprovado desde o início.

Saúde empresarial ficou inicialmente bloqueada por falta de dados.

Atendimento ao Cliente foi reprovado por um problema estrutural, não por falta de volume.

Essa diferença é importante porque continua valendo agora.

O módulo 2 preparou o dado

No módulo 2, trabalhamos com deduplicação e balanceamento.

Utilizamos MinHash e LSH para reduzir o esforço de comparação.

Foram 549 comparações reduzidas para 20.

Uma redução de 96,4% no trabalho de comparação.

Encontramos três duplicatas reais.

O recall foi perfeito.

Depois, trabalhamos balanceamento com temperatura e entropia de Shannon para evitar que uma única fonte dominasse o conjunto final.

O módulo 3 colocou o modelo em produção

No módulo 3, o treinamento aconteceu de verdade.

Partimos de 305 exemplos brutos.

Depois da deduplicação, ficamos com 300.

O conjunto final de treinamento foi balanceado para 200 exemplos.

Foram 120 de automóvel e 80 de saúde empresarial.

O treinamento utilizou três épocas, taxa de aprendizado 5 e posto 4.

O job real rodou na Vertex AI por cerca de 45 minutos.

Depois, o endpoint foi publicado.

Esse endpoint será reutilizado agora.

O módulo 4 criou um caminho alternativo

No módulo 4, testamos LoRA localmente.

Foram comparadas configurações com postos 4, 8 e 16, além de Full Fine-Tuning limitado às últimas 16 camadas.

O posto 8 foi utilizado no checkpoint que passou a ser reaproveitado no restante da disciplina.

Para esta tarefa, ele produziu o mesmo resultado observável do Full Fine-Tuning, apesar de utilizar 153,5 vezes menos parâmetros treináveis.

Esse resultado também volta a ser importante no projeto final.

O módulo 5 mediu de verdade

O módulo 5 trouxe uma avaliação rigorosa.

Pela primeira vez, utilizamos um conjunto realmente retido.

Também fizemos comparação A/B contra o modelo genérico.

Depois, aplicamos dois rounds de estresse com dificuldade crescente.

Na linha de base, o modelo atingiu 100% de precisão.

Depois de investigar e descartar um alarme falso causado por uma amostra pequena e por uma regra de avaliação inadequada, o resultado permaneceu em 100% também sob estresse.

Checklist de graduação

No final do módulo 5, nós não apenas olhamos para uma métrica.

Criamos um checklist de graduação com cinco critérios.

O modelo foi aprovado nos cinco.

Também validamos localmente o mesmo resultado com uma implementação independente.

Isso é importante porque o projeto final não parte de uma hipótese.

Ele parte de um modelo que já possui evidência de qualidade.

O veredito que chega ao módulo 6

Automóvel foi aprovado.

Saúde empresarial também foi aprovado depois que o volume de dados deixou de ser um bloqueio.

Atendimento ao cliente continua fora.

O motivo é o mesmo desde o início.

A tarefa é estruturalmente diferente.

Ela envolve negociação aberta e não foi coberta pelas evidências de robustez de extração construídas na disciplina.

Escalar não significa apenas testar outro domínio

Até o módulo 5, quando falávamos em escalar, a ideia principal era levar o mesmo protocolo de avaliação para um novo caso.

Isso continua válido.

Mas a emenda oficial deste módulo final exige uma segunda forma de escala.

Transformar o piloto aprovado em um fluxo de uso prático.

Não apenas medir.

Orquestrar.

Um endpoint não é um produto

Esse é um dos pontos mais importantes deste capítulo.

Um endpoint publicado pode funcionar perfeitamente.

Pode estar estável.

Pode ter sido aprovado em todos os testes.

Mesmo assim, ele ainda não é um produto completo.

Para utilizá-lo diretamente, alguém precisaria saber montar o prompt correto.

Também precisaria saber qual API chamar.

Precisaria saber autenticar.

Precisaria conhecer o formato esperado.

Depois, precisaria interpretar o JSON retornado.

Isso exige conhecimento técnico demais para ser considerado um fluxo de uso final.

O motor sem carroceria

A analogia utilizada é a de um motor de carro.

Podemos ter um motor perfeitamente calibrado.

Testado.

Com todos os números corretos em dinamômetro.

Ele funciona.

Mas ninguém dirige um motor sozinho.

Ainda faltam carroceria.

Volante.

Pedais.

Controles.

O modelo fine-tunado é esse motor.

O projeto final constrói a estrutura ao redor dele.

Fine-Tuning precisa de arquitetura

A aula traz casos reais para mostrar que isso não é apenas uma analogia acadêmica.

A EXL Service Holdings, empresa de tecnologia e BPO para seguradoras, publicou uma solução em parceria com a NVIDIA utilizando modelo fine-tunado sobre dados proprietários de sinistros.

O objetivo incluía extração e apoio a underwriting.

São domínios muito próximos ao cenário simulado pela Amplitude Seguros.

Segundo o caso apresentado na aula, o ganho reportado foi de 30% de acurácia sobre o modelo genérico e redução de 30% no custo operacional.

Outro exemplo em produção

A aula também cita o Nubank.

Nesse caso, a empresa treinou um transformer sobre sequências de transações bancárias.

Não era texto.

O modelo era utilizado para tarefas como prever inadimplência após aumento de limite, entre outras.

No teste offline, houve ganho relativo de 1,25% de AUC contra o modelo de referência baseado em LightGBM.

Depois, o modelo foi colocado em produção.

Segundo o caso apresentado, o ganho de longo prazo chegou a 32% na métrica de negócio utilizada pelo banco.

A mesma lição nos dois casos

Os dois exemplos possuem contextos diferentes.

Seguros.

Banco.

Extração.

Predição.

Mesmo assim, a lição é a mesma.

Fine-Tuning não é o produto final.

É um componente que precisa ser integrado a uma arquitetura.

Sem essa camada, o modelo continua restrito a quem já conhece seu funcionamento técnico.

O objetivo oficial deste módulo

A exigência agora é integrar o modelo customizado em um fluxo prático de uso.

Pode ser um chatbot.

Pode ser um agente.

Pode ser outro tipo de interface.

Nesta disciplina, o fluxo será transformado em um protótipo funcional em JavaScript.

A chamada será feita contra o modelo real.

Não haverá mock.

Não haverá simulação.

O protótipo chamará o endpoint treinado na disciplina.

Reutilizar é uma decisão de engenharia

O projeto final não deve começar do zero.

O endpoint do módulo 3 já existe.

O harness do módulo 5 também.

As funções de avaliação já foram testadas.

Por isso, vamos reutilizar tudo que já está confiável.

Essa é uma decisão deliberada.

Reescrever lógica validada apenas para ter um arquivo novo aumenta o risco.

O contrato dos dois domínios

Antes de criar uma orquestração, precisamos lembrar o contrato de dados.

Para automóvel, a resposta esperada contém três campos.

Segurado.

Placa.

Valor.

Para saúde empresarial, também temos três campos.

Beneficiário.

Procedimento.

Valor.

O número de campos é igual.

Mas o significado é diferente.

O classificador não conhece a resposta antes de classificar

Esse detalhe é essencial.

O primeiro passo do fluxo recebe apenas texto bruto.

Ele ainda não possui o JSON de saída.

Portanto, ele precisa identificar o domínio usando o vocabulário e o contexto presentes no texto de entrada.

Só depois disso será possível escolher a instrução e o esquema corretos.

O fluxo de quatro passos

Projeto final: orquestração em quatro passos

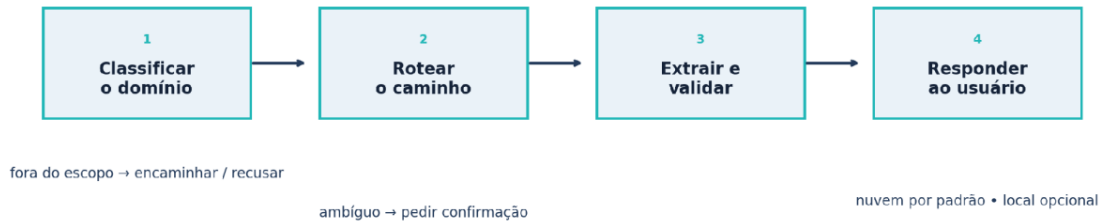


Figura — Fluxo operacional do protótipo final.

O protótipo será estruturado em quatro etapas.

Primeiro, classificar o domínio.

Segundo, rotear para o modelo correto.

Terceiro, extrair e validar o esquema.

Quarto, responder ao usuário em linguagem natural.

A ordem é importante.

Não é apenas uma lista de funções independentes.

Cada etapa depende da anterior.

Passo 1: classificar o domínio

O primeiro passo precisa decidir se o texto pertence ao domínio de automóvel ou ao domínio de saúde empresarial.

Essa classificação define todo o restante do fluxo.

Ela determina qual instrução será utilizada.

Qual esquema será esperado.

E como a resposta final será construída.

Passo 2: rotear para o modelo

Depois de classificar o domínio, o fluxo precisa escolher como executar a inferência.

O caminho padrão utiliza o endpoint publicado na Vertex AI no módulo 3.

Opcionalmente, será possível trocar para o checkpoint local treinado com LoRA no módulo 4.

Essa possibilidade será controlada por uma flag.

Passo 3: extrair e validar esquema

Receber um JSON não é suficiente.

O protótipo precisa validar se os campos esperados realmente estão presentes.

Também precisa verificar se não existem campos extras.

A função de adequação de esquema já existe no harness do módulo 5.

Ela será reutilizada.

Uma segunda camada de validação

A validação de esquema não detecta tudo.

Um campo pode existir e ainda assim estar vazio.

O campo valor pode existir e conter algo que não é numérico.

Por isso, o projeto final acrescenta uma nova camada de validação.

Ela verifica consistência dos valores.

Essa parte é um código novo.

Passo 4: responder ao usuário

Depois que o JSON estiver validado, o protótipo não deve simplesmente despejar o objeto bruto na tela.

A pessoa que utiliza a solução não deveria precisar interpretar a estrutura interna da API.

A resposta será transformada em uma confirmação em linguagem natural.

É a etapa que fecha o loop com o usuário.

A ordem protege o fluxo

Classificar vem antes de rotear.

Sem domínio, não sabemos qual instrução ou caminho utilizar.

Validar vem antes de responder.

Se formatarmos uma resposta bonita sobre um JSON incorreto, apenas escondemos o erro.

A sequência protege o sistema de produzir uma aparência de sucesso quando o dado está errado.

O que será novo e o que será reutilizado

O passo de classificação será escrito do zero.

O passo de resposta também.

O roteamento reutiliza integralmente o endpoint de nuvem já existente.

No modo local, porém, haverá nova lógica de orquestração para chamar o checkpoint LoRA.

A validação reutiliza a função de adequação de esquema, mas adiciona a checagem de consistência de valores.

Reuso evita divergência

Existe uma razão prática para reutilizar a função de validação.

Se a mesma regra for copiada para dois lugares, as implementações podem divergir.

Um bug pode ser corrigido em um arquivo e continuar existindo no outro.

Ao reutilizar o harness, uma correção feita na lógica original é herdada pelo protótipo.

Isso é disciplina de engenharia.

O fluxo é propositalmente pequeno

O protótipo terá poucas peças novas.

Isso é intencional.

O trabalho pesado já foi feito.

Treinamento.

Validação.

Medição.

Comparação.

Stress test.

Checklist.

Agora precisamos apenas construir a cola entre o modelo e uma pessoa real utilizando a solução.

Primeira decisão de arquitetura

Antes de codificar, duas decisões precisam ser justificadas por escrito.

A primeira é qual modelo será chamado por padrão.

Nós temos duas opções que já foram medidas.

Vertex AI.

Checkpoint local com LoRA rank 8.

As duas produziram o mesmo resultado no conjunto retido.

Vertex AI e modelo local produziram o mesmo resultado

No teste comparável, ambas as implementações atingiram 11 de 11 respostas com esquema válido.

A precisão média foi de 100%.

São dois caminhos de treinamento e execução diferentes chegando ao mesmo comportamento observável.

Isso cria uma decisão arquitetural real.

Trade-off do Vertex AI

Vertex AI é gerenciado.

Não exige manutenção de hardware local.

Já foi utilizado ao longo do módulo 5.

É o caminho mais próximo de um piloto de produção.

Mas possui custo por chamada.

Também depende da rede e do provedor.

Trade-off do modelo local

O modelo local não possui custo por chamada.

Pode operar offline.

Evita dependência de rede durante a inferência.

Por outro lado, exige que o checkpoint esteja disponível.

Também exige hardware compatível.

A máquina responsável pelo protótipo precisa conseguir executar o modelo.

Decisão adotada

O protótipo utilizará Vertex AI por padrão.

Esse é o piloto real publicado durante a disciplina.

O modo local ficará disponível por meio de uma flag opcional.

Essa escolha também prova que a camada de orquestração não precisa ficar presa a um único provedor.

Segunda decisão de arquitetura

A segunda decisão é ainda mais importante.

O que fazer quando o classificador não tem certeza?

Esse problema pode aparecer de formas diferentes.

E a arquitetura precisa definir o comportamento antes de o código ser escrito.

Caso fora do escopo

O primeiro cenário é receber uma solicitação de atendimento ao cliente.

Esse domínio não foi aprovado.

Não foi graduado.

Não passou pelas avaliações que justificam o uso do modelo fine-tunado.

Portanto, ele não pode ser forçado para automóvel ou saúde empresarial.

Não forçar um esquema inadequado

Se o protótipo tentar encaixar atendimento ao cliente em um dos dois esquemas aprovados, produzirá uma saída estruturalmente incorreta.

O problema seria ainda pior se essa saída parecesse válida.

Por isso, o sistema precisa reconhecer explicitamente que aquele caso está fora do escopo.

Roteamento para humano

Para atendimento ao cliente, a recomendação continua exatamente a mesma do módulo 5.4.

Prompt.

RAG.

Roteamento para especialista humano nos casos sensíveis.

O protótipo do módulo 6 não muda essa decisão.

Ele torna essa exclusão explícita na arquitetura.

Caso ambíguo

Existe também um segundo cenário.

O texto pode conter sinais de automóvel e saúde empresarial ao mesmo tempo.

Ou pode não apresentar nenhuma pista suficientemente clara.

Um classificador simples pode empatar.

Nesse caso, o sistema também não deve escolher no chute.

Pedir confirmação é melhor que inventar certeza

Quando não existe confiança clara, o protótipo pede confirmação ao usuário.

Isso gera um pequeno atrito.

Mas esse atrito é preferível a extrair campos do domínio errado.

Rejeitar temporariamente um caso válido é um erro menor do que devolver um dado corrompido como se fosse confiável.

Incerteza não vira resposta forçada

Esse é o princípio arquitetural que deve atravessar o projeto final.

Incerteza não deve ser escondida.

Quando o sistema não sabe, ele precisa sinalizar.

Quando o caso está fora do escopo, precisa encaminhar.

Quando existe ambiguidade, precisa pedir confirmação.

Um protótipo que responde sempre alguma coisa não é necessariamente melhor.

A arquitetura já está definida antes do código

Ao final desta primeira parte, ainda não escrevemos a implementação final.

Mas já sabemos exatamente como ela deve funcionar.

Temos dois domínios aprovados.

Temos um domínio explicitamente fora do escopo.

Temos dois caminhos possíveis de inferência.

Temos uma política para ambiguidade.

Temos um fluxo em quatro etapas.

E sabemos quais partes serão reutilizadas e quais serão novas.

No próximo capítulo, esses quatro passos e essas duas decisões passam do desenho para a execução. O protótipo será implementado em JavaScript e fará chamadas reais ao endpoint construído ao longo da disciplina.

NA PRÁTICA

- Desenhe o fluxo de quatro passos para um caso do seu contexto e marque onde o sistema deve recusar, pedir confirmação ou seguir.
- Liste quais componentes você reutilizaria de etapas anteriores e quais precisariam ser novos.
- Registre os trade-offs entre um caminho gerenciado em nuvem e um caminho local para a mesma tarefa.

CHECKPOINT DE APRENDIZAGEM

- Por que um endpoint publicado ainda não é um produto completo?
- Por que casos fora do escopo e ambíguos não devem ser forçados para um dos domínios treinados?
- Quais partes do protótipo podem ser reutilizadas e quais precisam ser implementadas agora?

RESUMO DA AULA

- O projeto final não refaz treinamento ou preparação de dados; ele integra os artefatos já construídos em uma solução utilizável.
- A arquitetura é organizada em quatro passos e inclui políticas explícitas de recusa e confirmação para evitar respostas forçadas.
- O caminho padrão utiliza o endpoint gerenciado, enquanto o modelo local permanece como alternativa opcional sob o mesmo contrato de saída.

[↑ Voltar ao início da Unidade 6](#)

Aula 2 — Projeto Final: Modelo Customizado para Domínio Específico (PT-2)

Unidade 6 • Projeto Final: Modelo Customizado para Domínio Específico

OBJETIVOS DE APRENDIZAGEM

- Implementar em JavaScript a orquestração definida na aula anterior.
- Integrar o endpoint da Vertex AI e o checkpoint LoRA local preservando o mesmo contrato para o restante do fluxo.
- Aplicar classificação conservadora, validação em duas camadas e testes automatizados para caminhos de sucesso, recusa e ambiguidade.

GITHUB E PRÁTICA

- [Projeto final: implementação JavaScript e caminho local](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• JavaScript	• Python + MLX
• Classificador por palavras-chave	• Validação em duas camadas
• Testes automatizados	• Portabilidade

Conteúdo da aula

Na primeira parte deste módulo final, nós desenhamos o protótipo antes de implementá-lo.

Definimos quatro passos.

Classificar.

Rotear.

Validar.

Responder.

Também tomamos duas decisões de arquitetura antes de escrever a implementação.

O caminho padrão utilizaria o endpoint publicado na Vertex AI, mantendo o modelo local como alternativa opcional.

E qualquer caso fora do escopo ou ambíguo não seria forçado para um domínio.

Agora esse desenho deixa de existir apenas no papel.

Nesta parte, o protótipo roda de verdade contra os dois modelos que a disciplina treinou.

Sem simulação.

Sem mock no caminho principal.

Com chamadas reais ao endpoint publicado e ao checkpoint local.

Do desenho para a implementação

O protótipo é implementado em JavaScript no arquivo principal da aplicação.

Ele concentra a orquestração dos quatro passos definidos anteriormente.

Ao mesmo tempo, não tenta reescrever tudo aquilo que os módulos anteriores já construíram.

Essa é uma característica importante do projeto.

A maior parte da robustez vem justamente do reuso.

O passo de classificação é novo.

O caminho padrão de nuvem reaproveita o endpoint já publicado.

A validação de esquema reaproveita o harness do módulo 5.

A formatação da resposta é nova.

E o caminho local adiciona uma pequena ponte para o modelo treinado com LoRA.

Um segundo arquivo para o modelo local

O caminho local depende de um segundo arquivo escrito em Python.

Isso acontece porque o checkpoint local foi treinado e executado com MLX, biblioteca voltada para Apple Silicon.

Não existe uma implementação equivalente em JavaScript usada neste projeto.

Em vez de esconder essa diferença, a arquitetura assume explicitamente que cada tecnologia deve ser utilizada onde faz sentido.

JavaScript orquestra o protótipo.

Python executa o modelo local.

Integração entre JavaScript e Python

Quando a execução recebe a opção de utilizar o modelo local, o processo JavaScript chama o script Python como um programa externo.

O texto de entrada é enviado para o processo Python.

A comunicação utiliza entrada e saída padrão.

O JavaScript envia os dados em formato JSON.

O Python processa a solicitação com o checkpoint LoRA e devolve uma resposta textual.

Para o restante do protótipo, os dois caminhos continuam apresentando o mesmo contrato.

Tanto Vertex AI quanto o processo local devolvem texto bruto que será interpretado e validado posteriormente.

Uma implementação bloqueante por escolha

A comunicação local utiliza uma chamada síncrona.

Isso significa que o processo JavaScript espera o Python carregar o checkpoint e produzir a resposta.

Enquanto isso, ele não executa outras atividades em paralelo.

Para um sistema de produção com muitas requisições simultâneas, essa escolha não seria adequada.

Mas para um protótipo didático que processa um caso de cada vez, ela simplifica leitura, depuração e compreensão.

O que mudaria em produção

Em um sistema real, poderíamos substituir essa abordagem por uma comunicação assíncrona.

Outra possibilidade seria manter o processo Python ativo por mais tempo, evitando carregar o checkpoint a cada solicitação.

Esse processo poderia receber pedidos por fila ou por alguma interface de serviço.

A aula deixa essa limitação explícita porque simplificação de protótipo não deve ser confundida com arquitetura definitiva de produção.

Alternativa para quem não possui Apple Silicon

O caminho local da disciplina foi construído para MLX em Apple Silicon.

Mas isso não significa que o conceito dependa de um Mac.

Existe um material complementar que documenta alternativas via Colab ou GPU CUDA local.

A ideia é preservar o mesmo contrato de entrada e saída.

A implementação muda.

A orquestração não precisa mudar.

Isso reforça novamente a separação entre arquitetura do fluxo e infraestrutura de execução.

O classificador é a primeira peça totalmente nova

A função de classificação decide se o texto pertence ao domínio de automóvel ou de saúde empresarial.

Ela trabalha com vocabulário específico.

O detalhe mais importante não está apenas nas palavras escolhidas.

Também está nas palavras que foram deliberadamente excluídas.

A palavra “sinistro” foi retirada por segurança

“Sinistro” parece uma palavra natural para o domínio automotivo.

Mas ela também aparece em atendimento ao cliente.

Por exemplo, uma pessoa pode contestar uma negativa de cobertura de sinistro.

Esse domínio foi reprovado desde o módulo 1 e continua fora do escopo.

Se “sinistro” fosse considerado uma palavra decisiva para automóvel, o classificador poderia gerar um voto falso.

O protótipo acabaria encaminhando uma tarefa não aprovada para o modelo fine-tunado.

Por isso, a palavra é excluída do vocabulário de classificação.

Vocabulário realmente discriminativo

Para automóvel, são utilizadas palavras mais específicas do domínio.

Placa.

Veículo.

Oficina.

Funilaria.

Para saúde empresarial, aparecem termos como beneficiário.

Procedimento.

Clínica.

A ideia não é construir um classificador sofisticado.

É utilizar sinais fortes e claramente associados aos dois domínios aprovados.

Variações com e sem acento

As listas também consideram formas com e sem acentuação.

Isso não é redundância desnecessária.

Textos reais muitas vezes chegam a ser digitados rapidamente.

Acentos podem ser omitidos.

Se o classificador dependesse da grafia perfeita, criaria uma fragilidade artificial.

Por isso, o protótipo considera variações simples do vocabulário.

Regra de pontuação conservadora

A classificação conta quantas palavras específicas de cada domínio aparecem no texto.

Se automóvel recebe mais pontos, o domínio é automóvel.

Se saúde empresarial recebe mais pontos, o domínio é saúde.

Mas existem duas exceções importantes.

Zero a zero significa fora do escopo.

Empate maior que zero significa ambiguidade.

Fora do escopo

Quando nenhuma pista específica aparece, o protótipo não chama o modelo.

Ele reconhece que não possui evidência suficiente para classificar o texto em um dos dois domínios aprovados.

Essa recusa não é uma falha.

É o comportamento esperado.

Ela impede que uma solicitação desconhecida seja forçada em um esquema inadequado.

Ambiguidade

Quando os dois lados possuem a mesma pontuação, também não existe chamada ao modelo.

O sistema pede confirmação ao usuário.

Essa política foi definida na PT-1 e agora aparece na implementação.

A regra continua sendo a mesma.

Sem confiança clara, não existe decisão forçada.

Por que não usar outro LLM como classificador

Seria possível utilizar outro modelo de linguagem para classificar o domínio.

Mas isso adicionaria uma nova chamada de API.

Mais custo.

Mais latência.

Mais dependência.

Mais um ponto de falha.

Neste projeto, existem apenas dois domínios aprovados e vocabulários bastante distintos.

Por isso, um classificador simples de palavras-chave é proporcional ao problema.

Simplicidade também é decisão de arquitetura

Usar um LLM porque existe um LLM disponível não é necessariamente boa engenharia.

A solução deve ser proporcional à tarefa.

Se uma regra simples resolve de forma clara e verificável, ela pode ser preferível a uma camada mais complexa.

Esse tipo de decisão precisa ser documentado.

Ele será retomado no próximo capítulo.

A suíte de testes automatizados

Antes das demonstrações manuais, o protótipo possui uma suíte com 14 testes automatizados.

Treze deles não chamam rede.

Testam comportamento local.

Classificação.

Formatação.

Consistência de valores.

Caminhos de recusa.

Roteamento.

Falhas de JSON.

Esses testes utilizam dependências injetadas para evitar chamadas reais quando elas não são necessárias.

Um único teste chama o endpoint de verdade

Dos 14 testes, apenas um faz uma chamada real ao endpoint publicado.

Ele utiliza um caso de automóvel escrito manualmente.

Não é um exemplo gerado pelo template determinístico utilizado anteriormente.

Isso permite verificar que o fluxo continua conectado ao modelo real.

Ao mesmo tempo, evita transformar toda a suíte em um conjunto caro e lento de chamadas remotas.

Testar barato o que pode ser testado barato

Essa divisão é importante.

Nem todo teste precisa tocar o modelo.

Se o objetivo é verificar o classificador, não há razão para pagar uma chamada de inferência.

Se o objetivo é verificar formatação, também não.

Apenas o que precisa confirmar integração real utiliza rede.

Isso reduz custos sem reduzir a cobertura lógica.

O caminho padrão de roteamento

Quando o domínio está claro e o modo local não foi solicitado, o protótipo chama o mesmo endpoint utilizado nos módulos de avaliação.

Não existe um novo endpoint para o projeto final.

O módulo 6 reaproveita exatamente a infraestrutura que já foi validada.

Essa continuidade é uma das provas de que os módulos anteriores construíram peças reutilizáveis.

O caminho local

Quando a opção local está ativa, o roteamento chama o script Python.

Esse script carrega o checkpoint LoRA rank 8 do módulo 4.

Depois devolve a resposta no mesmo formato esperado pelo JavaScript.

A camada seguinte não precisa saber se a inferência veio da nuvem ou da máquina local.

Essa intercambialidade é intencional.

Erros do Python são propagados

Se o processo local falhar, o erro não é escondido atrás de uma mensagem genérica.

Se o checkpoint não existir.

Se houver problema de execução.

Se o Python retornar erro.

O JavaScript captura esse erro real e o propaga.

Isso melhora o diagnóstico.

Uma arquitetura que esconde a causa de uma falha torna a manutenção muito mais difícil.

Validação em duas camadas

Depois da inferência, o protótipo entra no terceiro passo.

A primeira camada é a validação de esquema.

Ela reaproveita a mesma lógica do módulo 5.1.

A resposta deve ter exatamente os campos esperados.

Nem faltando.

Nem sobrando.

A validação de esquema não resolve tudo

Um JSON pode possuir as chaves corretas e ainda estar semanticamente ruim.

O campo valor pode conter algo não numérico.

Um campo textual pode estar vazio.

Por isso, o módulo 6 adiciona uma segunda camada.

A validação de consistência de valores.

Consistência de valores

Essa nova checagem verifica se campos textuais possuem conteúdo e se o valor é válido.

A lógica complementa o harness anterior.

O objetivo não é substituir a validação do módulo 5.

É acrescentar uma verificação que só faz sentido agora, no fluxo de uso.

Falha de validação não vira chute

Se qualquer uma das duas camadas reprovar, o protótipo não tenta corrigir silenciosamente.

Também não tenta inventar o campo ausente.

Ele informa que o resultado precisa ser tentado novamente.

Esse comportamento mantém o princípio usado desde os módulos anteriores.

Erro detectado deve ser tratado como erro.

Não como oportunidade para mascarar o problema.

Resposta em linguagem natural

Depois de validar os dados, entra o quarto passo.

O JSON não é mostrado diretamente ao usuário.

O protótipo monta uma frase de confirmação em português.

A pessoa recebe os campos já interpretados.

O valor é formatado em reais.

Com separador de milhar.

Vírgula decimal.

O usuário pode simplesmente confirmar ou corrigir.

Dados estruturados para máquinas, linguagem natural para pessoas

Essa camada fecha o problema que motivou o módulo 6.

O modelo fine-tunado devolve uma estrutura útil para sistemas.

Mas uma pessoa não deveria precisar ler chaves de JSON para utilizar a solução.

O protótipo transforma a saída técnica em uma interação compreensível.

Demonstração real com automóvel

A aula executa um caso escrito manualmente.

O texto contém o nome do segurado.

A placa.

Dois valores parciais.

Peças.

Mão de obra.

E um valor total.

O objetivo é verificar se o modelo não confunde os valores distratores com o valor final.

Extração correta apesar dos distratores

O protótipo classifica o domínio como automóvel.

Chama o endpoint real.

Valida a resposta.

Depois devolve uma confirmação com segurado, placa e valor total corretos.

Os valores parciais de peças e mão de obra são ignorados.

Isso demonstra que o fluxo está operando sobre texto corrido e não apenas sobre exemplos artificiais perfeitamente estruturados.

Recusa fora do escopo

A aula testa então uma mensagem de contestação de negativa de cobertura.

Ela fala de sinistro.

Mas não contém as palavras específicas utilizadas para automóvel ou saúde.

O resultado é zero a zero.

O protótipo reconhece que o caso está fora do escopo.

E recomenda encaminhamento para um especialista humano.

Nenhuma chamada de API acontece na recusa

Esse detalhe é central.

O modelo fine-tunado não é chamado.

A recusa acontece antes do roteamento.

Isso reduz custo.

Reduz latência.

E, principalmente, evita que um domínio não aprovado seja processado como se estivesse dentro da cobertura do modelo.

Recusa por ambiguidade

O segundo caso de recusa mistura sinais dos dois domínios.

O texto menciona elementos de automóvel e de saúde empresarial.

A pontuação empata.

O protótipo não escolhe arbitrariamente.

Ele pede ao usuário que confirme qual dos dois domínios se aplica.

Recusar pode ser mais barato e mais seguro

Os dois casos de recusa respondem praticamente de forma imediata.

Não existe chamada de rede.

Não existe custo de inferência.

Isso torna visível uma ideia importante.

Recusar corretamente pode ser mais barato do que aceitar errado.

E também pode ser muito mais seguro.

Três entradas, três comportamentos

Ao final dessas demonstrações, temos três tipos de resposta.

Caso válido.

O protótipo extrai e pede confirmação.

Caso fora do escopo.

O protótipo encaminha para o humano.

Caso ambíguo.

O protótipo pede esclarecimento.

Os três comportamentos são diferentes porque a arquitetura foi desenhada para lidar com incerteza explicitamente.

Comparação entre nuvem e modelo local

A aula executa o mesmo texto de automóvel primeiro contra Vertex AI e depois contra o modelo local.

A única diferença entre as execuções é a opção de linha de comando que seleciona o caminho local.

O restante da orquestração permanece igual.

Classificar.

Rotear.

Validar.

Responder.

Mesmo resultado com dois caminhos independentes

Os dois modelos devolvem os mesmos três campos para o exemplo apresentado.

Segurado.

Placa.

Valor.

O texto não fazia parte do conjunto conhecido anteriormente.

Mesmo assim, os dois caminhos convergem para a mesma resposta observável.

Isso reforça o resultado medido no módulo 5.4.

A orquestração não depende do provedor

Esse é um dos resultados mais importantes desta parte.

Nenhuma lógica central precisa ser alterada quando trocamos Vertex AI pelo checkpoint local.

O classificador não muda.

A validação não muda.

O formatador não muda.

Somente a implementação do passo de inferência é substituída.

Essa separação reduz o acoplamento.

O custo real do modo local aparece na demonstração

O modelo local demora mais para responder nesse formato.

O motivo é simples.

Cada execução carrega o checkpoint novamente.

Isso gera custo de inicialização.

No caminho gerenciado da nuvem, essa responsabilidade fica escondida pelo serviço.

No caminho local, ela aparece de forma explícita.

Não é um defeito do conceito local

Essa latência não significa que o modelo local seja necessariamente pior.

Ela é consequência da forma simplificada como o protótipo foi implementado.

Se o processo Python permanecesse vivo entre requisições, o checkpoint não precisaria ser recarregado a cada chamada.

Essa otimização é real.

Mas está fora do escopo deste protótipo demonstrativo.

O que os 14 testes realmente provam

A suíte não serve apenas para mostrar números verdes no terminal.

Ela verifica se a implementação corresponde às decisões da PT-1.

O classificador precisa respeitar fora de escopo e ambiguidade.

As recusas não podem chamar a API.

A validação precisa rejeitar saídas ruins.

O formatador precisa produzir uma resposta utilizável.

O roteamento precisa alternar entre nuvem e local.

O protótipo fecha dois requisitos oficiais

Com esse fluxo rodando ponta a ponta, o requisito de integrar o modelo customizado em um fluxo prático está cumprido.

Também está cumprido o requisito de implementar em JavaScript um protótipo funcional utilizando o modelo treinado via API real.

O projeto final deixa de ser apenas documentação.

Ele funciona.

O protótipo é pequeno porque os módulos anteriores fizeram o trabalho pesado

A implementação central possui pouco mais de 200 linhas de lógica, além da suíte de testes.

Isso só é possível porque o protótipo não tenta reconstruir tudo.

O endpoint existe porque o módulo 3 o publicou.

O caminho local existe porque o módulo 4 treinou o checkpoint.

A validação existe porque o módulo 5 construiu um harness reutilizável.

O projeto final apenas conecta essas peças.

Fine-Tuning vale a pena também pelo que permite construir

Desde o módulo 1, a disciplina pergunta se Fine-Tuning vale a pena.

Até aqui, a resposta vinha de custo, qualidade, disponibilidade de dados e capacidade de generalização.

Agora aparece uma dimensão adicional.

O que conseguimos construir depois que o modelo está treinado?

Neste caso, conseguimos criar um protótipo pequeno, previsível, com dois provedores intercambiáveis, validação estruturada e política explícita de recusa.

Essa é uma resposta prática.

O que falta para encerrar a disciplina

Depois de implementar, ainda falta documentar.

As escolhas de arquitetura não devem ficar escondidas dentro do código.

Por que palavras-chave e não outro LLM?

Por que o Vertex AI como padrão?

Por que modo local opcional?

Por que uma chamada síncrona no protótipo?

Por que recusar ambiguidade?

Por que reutilizar o harness?

Essas decisões precisam ser justificadas por escrito.

É isso que a próxima parte do módulo final vai organizar, encerrando o projeto e a disciplina com a documentação técnica das escolhas que transformaram o modelo aprovado em um protótipo funcional.

NA PRÁTICA

- Implemente uma função de classificação simples que consiga retornar domínio conhecido, fora do escopo ou ambíguo.
- Escreva testes que garantam que casos recusados não chamem o provedor de inferência.
- Defina um contrato único para que dois provedores diferentes possam ser trocados sem alterar a lógica de validação e resposta.

CHECKPOINT DE APRENDIZAGEM

- Por que a implementação local usa Python enquanto a orquestração permanece em JavaScript?
- Por que a palavra “sinistro” foi removida do vocabulário discriminativo do classificador?
- O que a suíte de testes prova sobre recusa, validação e portabilidade entre os dois caminhos?

RESUMO DA AULA

- A implementação transforma o desenho em um protótipo real, com JavaScript orquestrando e Python executando o checkpoint local quando necessário.
- O classificador é conservador: fora do escopo e ambiguidade são tratados explicitamente, evitando chamadas desnecessárias e decisões por chute.
- A validação e o contrato comum permitem alternar entre Vertex AI e modelo local sem alterar o restante da orquestração.

[↑ Voltar ao início da Unidade 6](#)

Aula 3 — Projeto Final: Modelo Customizado para Domínio Específico (PT-3)

Unidade 6 • Projeto Final: Modelo Customizado para Domínio Específico

OBJETIVOS DE APRENDIZAGEM

- Documentar as decisões de arquitetura e preservar o raciocínio por trás de cada escolha.
- Reavaliar o comportamento após escalar o dataset e interpretar corretamente um resultado pior como evidência, não como falha a esconder.
- Fechar a disciplina distinguindo o que o case prova, o que não prova e como reproduzir a missão prática em outro contexto.

GITHUB E PRÁTICA

- [Projeto final: documentação e decisões de arquitetura](#)

Sugestão de estudo: abra o artefato indicado, reproduza o fluxo da aula e registre uma pequena alteração ou verificação antes de consultar os demais arquivos do módulo.

CONCEITOS-CHAVE

• Architecture Decision Record	• Escala de dados
• Reavaliação	• Escopo da evidência
• Rigor experimental	• Documentação

Conteúdo da aula

Chegamos ao último vídeo do módulo e da disciplina.

Até aqui, o projeto final já fez duas coisas importantes.

Primeiro, desenhou a arquitetura do protótipo.

Depois, executou esse protótipo de verdade contra os dois caminhos de inferência construídos ao longo da disciplina.

Faltava uma última etapa.

Documentar as decisões.

Não apenas registrar o que foi feito.

Registrar por que foi feito daquela maneira.

Esse é o ponto que fecha o projeto.

Código funcionando não encerra um projeto profissional

Um protótipo pode funcionar perfeitamente e ainda assim ser difícil de manter.

Isso acontece quando as decisões estão apenas na cabeça de quem desenvolveu.

Por que utilizamos Vertex AI por padrão?

Por que o classificador usa palavras-chave?

Por que recusamos em vez de adivinhar?

Por que reutilizamos funções existentes?

Por que escolhemos LoRA rank 8?

Se essas respostas não estiverem registradas, alguém que herdar o projeto no futuro terá de reconstruir o raciocínio.

E pode reconstruí-lo de forma diferente.

Documentação preserva contexto

Imagine alguém abrindo o projeto daqui a um ano.

Essa pessoa encontra um classificador simples baseado em vocabulário.

A primeira reação pode ser perguntar por que não utilizar um LLM para isso.

Sem documentação, a justificativa desaparece.

Custo.

Latência.

Complexidade.

Proporcionalidade com o problema.

Tudo isso existiu na decisão original.

Mas, se não estiver escrito, deixa de fazer parte do sistema.

Um documento de decisões de arquitetura

A disciplina encerra o case com um documento específico de decisões de arquitetura.

Ele registra as oito decisões mais importantes de toda a jornada.

Uma por módulo do 1 ao 5.

E três decisões do módulo 6.

Cada decisão segue o mesmo formato.

Escolha.

Alternativa rejeitada.

Por quê.

Resultado real.

Esse formato cria consistência e facilita auditoria.

Escolha

A primeira parte registra o que foi adotado.

Não precisa ser um texto longo.

A ideia é deixar claro qual caminho foi escolhido.

Por exemplo, utilizar um gate ponderado com AHP.

Ou adotar MinHash mais LSH.

Ou utilizar Vertex AI como caminho padrão.

Alternativa rejeitada

Toda decisão relevante possui outra possibilidade.

Registrar apenas a escolha final esconde metade do raciocínio.

Por isso, o documento também registra o que foi considerado e rejeitado.

Checklist simples.

Comparação par a par.

Infraestrutura própria.

Full Fine-Tuning.

Outro LLM para classificação.

A alternativa rejeitada ajuda a explicar o contexto da decisão.

Por quê

Depois vem a justificativa.

Essa parte não pode ser uma opinião vaga.

Ela precisa ligar a escolha ao problema real.

Custo.

Escala.

Manutenção.

Qualidade.

Latência.

Risco.

Disponibilidade de dados.

A justificativa mostra qual critério pesou na decisão.

Resultado real

O quarto item é o mais importante para manter rigor.

Resultado real não é “achamos que ficou melhor”.

É um número.

Uma métrica.

Um comportamento observado.

Algo que pode ser rastreado até um experimento executado.

A disciplina inteira foi construída dessa forma.

Decisão 1: gate ponderado por AHP

No módulo 1, a decisão foi utilizar um framework de quatro perguntas com pesos e limiares por pergunta.

A alternativa rejeitada foi um checklist binário ou uma decisão por maioria simples.

Essa diferença é importante porque saúde empresarial e atendimento ao cliente ficavam relativamente próximos no score agregado.

Saúde empresarial ficou em 0,60.

Atendimento ao cliente ficou em 0,66.

Uma leitura superficial poderia tratar os dois como casos semelhantes.

O score agregado não conta toda a história

O problema só fica claro quando analisamos cada pergunta individualmente.

Saúde empresarial reprovava por um problema temporário.

Faltava dado suficiente naquele momento.

Atendimento ao cliente reprovava por problemas estruturais.

A tarefa era mais aberta e instável.

Por isso, olhar apenas para uma média esconderia a natureza real da decisão.

Decisão 2: MinHash mais LSH

No módulo 2, a escolha foi utilizar MinHash combinado com LSH para deduplicação.

A alternativa rejeitada foi comparar todos os exemplos par a par.

O motivo foi escalabilidade.

Comparação exaustiva funciona em datasets pequenos.

Mas cresce rapidamente.

No experimento da disciplina, 549 comparações foram reduzidas para 20.

Uma redução de 96,4% no trabalho.

Com recall perfeito.

Decisão 3: API gerenciada

No módulo 3, a escolha foi utilizar uma API gerenciada na Vertex AI.

A alternativa seria começar montando infraestrutura própria de treinamento e publicação.

A justificativa foi simples.

Primeiro precisamos provar o caso de negócio.

Depois podemos decidir se vale investir em infraestrutura mais complexa.

O objetivo naquele momento era validar Fine-Tuning profissional com um pipeline real.

Decisão 4: LoRA rank 8

No módulo 4, a escolha foi LoRA com rank 8.

A alternativa considerada foi Full Fine-Tuning nas últimas 16 camadas.

O rank 8 não foi escolhido porque alguém assumiu que era o melhor custo-benefício.

Ele foi testado.

E, para aquela tarefa, empatou com o Full Fine-Tuning no comportamento observado.

Ao mesmo tempo, treinou 153,5 vezes menos parâmetros.

A importância de não reescrever a história

Documentação técnica precisa registrar o que realmente aconteceu.

Não o que pareceria mais elegante depois.

O rank 8 também era o valor default da ferramenta utilizada.

O fato de ter funcionado bem foi confirmado pelos experimentos.

Essa honestidade é importante.

A decisão só pode ser auditável se o documento representar o processo real.

Decisão 5: protocolo de avaliação em três frentes

No módulo 5, a decisão foi não confiar em uma única métrica forte.

A avaliação combinou três frentes.

Teste retido.

Comparação contra baseline.

Testes de estresse escritos à mão.

Essa escolha evitou que um resultado inicial fosse interpretado como prova absoluta de generalização.

Avaliação precisa resistir a novos cenários

Um modelo pode funcionar muito bem em um conjunto simples e falhar quando recebe textos mais difíceis.

Por isso, o protocolo foi aumentando a dificuldade.

A avaliação foi desenhada para procurar falhas.

Não para confirmar uma expectativa.

Esse princípio aparece novamente neste capítulo final.

Decisão 6: classificador por palavras-chave

No módulo 6, a primeira decisão foi utilizar um classificador simples baseado em vocabulário específico.

A alternativa rejeitada foi chamar outro LLM apenas para classificar o domínio.

A justificativa foi a proporcionalidade.

Existiam dois domínios aprovados.

Os vocabulários eram suficientemente distintos.

Adicionar outro modelo aumentaria custo, latência e complexidade sem necessidade comprovada.

Decisão 7: Vertex AI por padrão, modelo local opcional

A segunda decisão do módulo 6 foi utilizar Vertex AI como caminho padrão.

O modelo local permaneceu disponível por uma flag.

O endpoint gerenciado era o caminho já publicado e validado durante a disciplina.

O modelo local oferecia independência de rede e custo por chamada zero.

A arquitetura manteve os dois.

A troca de provedor não altera a orquestração

Essa decisão mostrou um resultado arquitetural importante.

Classificação.

Validação.

Formatação.

Política de recusa.

Nada disso muda quando trocamos a origem da inferência.

Apenas o componente responsável pela chamada ao modelo é substituído.

Isso reduz acoplamento.

Decisão 8: recusar em vez de adivinhar

A terceira decisão do módulo 6 foi não transformar incerteza em resposta.

Casos fora do escopo são encaminhados.

Casos ambíguos pedem confirmação.

A alternativa rejeitada seria escolher um domínio mesmo sem confiança.

Esse comportamento reduz custo e evita corrupção silenciosa de dados.

A decisão transversal de reuso

Além das oito decisões, existe uma escolha que atravessa a disciplina inteira.

Reusar código validado em vez de copiá-lo.

O framework do módulo 1.

A lógica de preparação do módulo 2.

O harness do módulo 5.

Quando uma função já existe e foi testada, ela é importada diretamente.

Não é duplicada.

Por que copiar é perigoso

Duas cópias da mesma regra podem divergir.

Uma recebe correção.

A outra não.

Uma muda um detalhe.

A outra fica para trás.

Reuso reduz esse risco.

O módulo 3 é diferente porque o artefato reutilizado não é um bloco de código local.

É um endpoint publicado.

Nesse caso, o reuso acontece por chamada de API.

O modelo escalado para 3.000 exemplos

Depois do veredito do módulo 5, o case ganhou um novo experimento.

Um novo job foi executado com 3.000 exemplos.

Quinze vezes mais dados que o piloto de 200 exemplos.

Esse modelo representa uma tentativa de escalar o piloto.

E, novamente, a disciplina não assume que mais dado significa automaticamente melhor resultado.

Reavaliar com o mesmo protocolo

O modelo de 3.000 exemplos foi testado com os mesmos conjuntos utilizados anteriormente.

Teste retido.

Round 1.

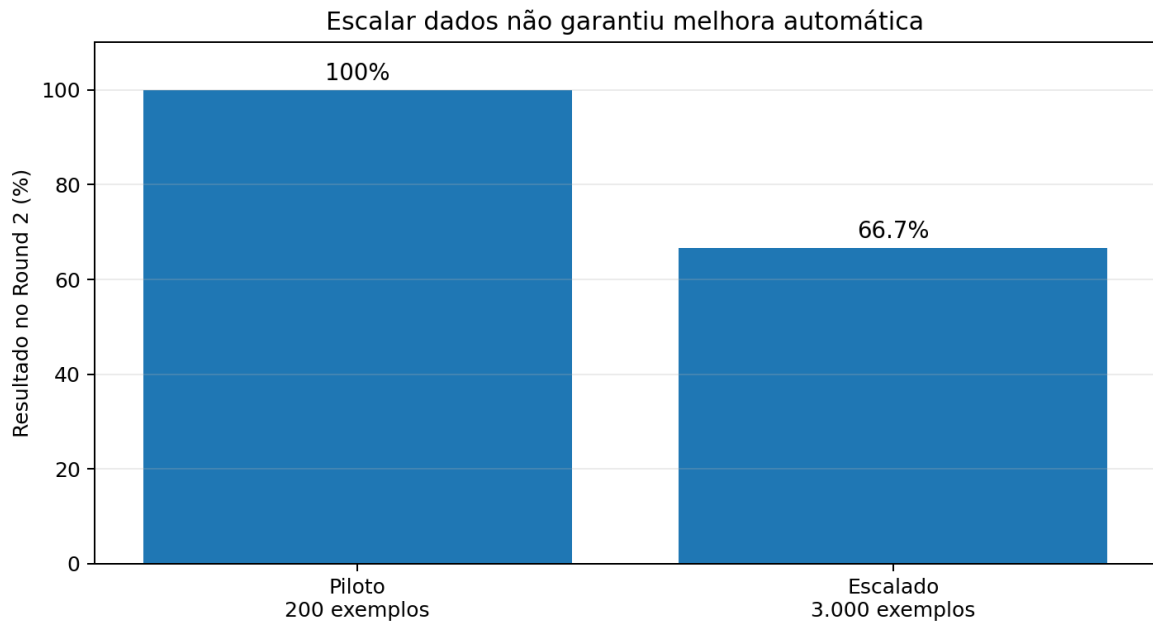
Round 2.

Isso permite uma comparação direta com o piloto.

A regra é importante.

Se mudarmos o protocolo junto com o modelo, perdemos a capacidade de comparar.

Escalar não significa melhorar automaticamente



Comparação da execução apresentada na aula; o material alerta para não generalizar o resultado.

Figura — Resultado apresentado na aula ao comparar o piloto com a execução escalada; a própria transcrição alerta para não generalizar essa observação.

No teste retido e no primeiro round de estresse, o modelo escalado continuou apresentando desempenho forte.

Mas no round 2 apareceu uma regressão.

Em uma das execuções apresentadas, o resultado caiu de 100% para 66,7%.

Esse é um resultado importante justamente porque é incômodo.

Um resultado pior também é evidência

Seria fácil assumir que 3.000 exemplos necessariamente superariam 200.

O experimento mostra que isso não pode ser garantido.

Mais dados podem introduzir distribuição diferente.

Podem alterar comportamento.

Podem criar novas fragilidades.

A escala precisa ser medida.

Não presumida.

O que o case prova

O documento final precisa ser preciso sobre o alcance das evidências.

O case mostra que um piloto curado com 200 exemplos reais, submetido a um protocolo de avaliação rigoroso, generalizou para dados nunca vistos.

Também superou o modelo genérico por margem clara no cenário testado.

Resistiu às avaliações de formato e estrutura.

E foi integrado a um protótipo funcional.

O que o modelo escalado prova

O modelo com 3.000 exemplos mostra que a abordagem pode ser escalada e reavaliada com o mesmo protocolo.

Ele preservou bons resultados em parte dos testes.

Mas não manteve desempenho perfeito em todas as condições.

Essa diferença precisa fazer parte da documentação.

Não deve ser escondida.

O que o case não prova

O projeto não prova que 3.000 exemplos são suficientes para qualquer cenário de produção.

Não há prova de generalização para outro idioma.

Outra moeda.

Outro padrão de documento.

Outro volume de tráfego.

Não prova resistência a entradas maliciosas.

Não prova que aumentar dados sempre preserva desempenho.

Essas perguntas exigem testes próprios.

Escopo é parte do rigor

Dizer o que um experimento não prova não enfraquece o trabalho.

Faz o contrário.

Delimita onde a evidência é válida.

Esse é um hábito essencial em inteligência artificial.

Resultados precisam ser interpretados dentro do cenário em que foram produzidos.

Casos de mercado ajudam a contextualizar

A aula também apresenta referências de mercado para mostrar que o Fine-Tuning continua sendo utilizado profissionalmente.

Existem casos em atendimento ao cliente.

Saúde.

Jurídico.

Finanças.

Ferramentas de desenvolvimento.

Varejo.

Mídia.

O material complementar reúne diversos exemplos com suas respectivas referências.

Fine-Tuning não desaparece porque modelos de fronteira melhoram

A pergunta sobre a utilidade de Fine-Tuning continua relevante.

Modelos genéricos ficam melhores.

Ao mesmo tempo, empresas continuam possuindo dados proprietários.

Vocabulário específico.

Processos próprios.

Tarefas estreitas.

Em alguns desses casos, especialização continua produzindo vantagem.

Mas a decisão precisa ser medida.

O custo real do experimento escalado

O último grande número da disciplina não é uma métrica de precisão.

É o custo de executar o experimento escalado.

O job de 3.000 exemplos levou 24 minutos e 53 segundos no caso apresentado.

O job anterior de 200 exemplos havia levado 45 minutos e 42 segundos.

Isso não significa que mais dados treinam mais rápido.

A diferença provavelmente está relacionada à variação de fila e infraestrutura.

Não transformar observação em regra

Esse exemplo é útil porque ensina novamente a evitar generalização indevida.

Uma execução mais rápida não cria uma lei.

É apenas o resultado daquela execução.

O custo financeiro reportado para o novo job foi de R\$41,40, verificado diretamente no billing do Google Cloud.

O valor não cresceu na mesma proporção dos dados.

O dataset de produção também é documentado

A montagem dos 3.000 exemplos não fica escondida.

Existe documentação específica explicando como o dataset de produção foi criado.

Isso é importante porque resultado de modelo sem rastreabilidade de dados é incompleto.

A documentação fecha não apenas a arquitetura.

Fecha também a origem do experimento escalado.

A missão prática final

Depois de acompanhar o case da Amplitude Seguros durante seis módulos, a disciplina termina pedindo que cada aluno repita o arco em um caso próprio.

O objetivo não é copiar o case.

É aplicar o método.

A atividade possui quatro etapas principais.

Primeiro: aplicar o framework de decisão

O aluno deve escolher um caso real, preferencialmente do próprio trabalho.

Depois, aplicar as quatro perguntas do módulo 1.

Fine-Tuning não começa treinando.

Começa decidindo se vale a pena.

Se o gate reprovar, isso já é um resultado válido.

Segundo: construir um piloto pequeno

Se o caso for aprovado, o próximo passo é criar um dataset curado.

Deduplicado.

Balanceado.

Com origem conhecida.

Depois, executar um treino real quando possível.

Se não houver acesso à infraestrutura, a disciplina permite documentar uma simulação com o mesmo rigor.

Dados sintéticos exigem cuidado

Quando não existe dado suficiente, há um material complementar sobre geração sintética.

A ideia é utilizar métodos inspirados em abordagens como Self-Instruct e Magpie.

Mas existe uma preocupação central.

O modelo não pode simplesmente decorar o padrão do próprio gerador.

Por isso, a geração sintética precisa ser planejada e avaliada.

Terceiro: avaliar em três frentes

O piloto precisa ser avaliado com o mesmo protocolo central do módulo 5.

Teste retido.

Comparação contra baseline.

Teste de estresse escrito manualmente.

O teste de estresse não deve ser apenas uma variação automática do template de treinamento.

Ele precisa procurar pontos fracos.

Quarto: documentar cada decisão

A última entrega repete o formato deste capítulo.

Escolha.

Alternativa rejeitada.

Por quê.

Resultado real.

Esse documento precisa permitir que outra pessoa compreenda as decisões sem assistir às aulas ou perguntar ao autor.

Esse é o fechamento profissional do trabalho.

O campo continua mudando

A aula encerra também olhando para o que vem depois.

World models.

Sistemas agênticos.

Protocolos de integração.

Mais autonomia.

Mais ferramentas.

E, junto com tudo isso, novos riscos.

O ponto não é prever exatamente qual tecnologia dominará.

É entender que o profissional precisa continuar aprendendo.

Mais autonomia exige mais controle

À medida que agentes ganham capacidade de agir, auditoria e governança se tornam ainda mais importantes.

Sandboxing.

Permissões.

Monitoramento.

Rastreabilidade.

Segurança.

Esses temas deixam de ser extras.

Passam a fazer parte do desenho.

A disciplina seguinte aprofunda justamente a segurança e governança.

Disponibilidade de Fine-Tuning muda

Outro cuidado prático é verificar a disponibilidade atual dos provedores.

Fine-Tuning pode mudar de status.

Modelos podem perder suporte.

As APIs podem ser descontinuadas.

Novas alternativas podem surgir.

Por isso, existem materiais complementares específicos sobre disponibilidade de provedores e validade dos modelos utilizados na disciplina.

Ferramenta certa, não ferramenta mais sofisticada

Se existe um fio que conecta toda a disciplina, é esse.

A solução mais sofisticada não venceu por padrão.

AHP foi utilizado porque separava melhor os casos.

MinHash e LSH porque escalavam melhor.

API gerenciada porque era proporcional ao piloto.

LoRA porque entregou o resultado necessário com muito menos parâmetros treináveis.

Palavras-chave porque o problema de classificação era simples.

Recusa porque incerteza não deveria virar dado incorreto.

Mensurar antes de confiar

O segundo fio que atravessa a disciplina é a mensuração.

Toda métrica precisa ser verificada.

Toda conclusão precisa ter fonte.

Todo resultado incômodo precisa ser investigado.

Foi assim quando apareceu um alarme falso na avaliação.

Foi assim quando o modelo escalado regrediu no segundo round de estresse.

Rigor não é apenas defender o resultado que queremos.

É aceitar o resultado quando ele contradiz nossa expectativa.

Documentar antes de assumir

A última lição deste módulo fecha o ciclo.

Decisões precisam sobreviver a quem as tomou.

Código muda.

Equipes mudam.

Modelos mudam.

Provedores mudam.

O documento de arquitetura preserva o motivo da escolha.

Permite auditar.

Permite discordar.

Permite repetir o experimento.

E permite melhorar o sistema sem reconstruir toda a história.

Ao final de seis módulos, o projeto começou perguntando quando Fine-Tuning vale a pena e terminou com um protótipo real, dois caminhos de inferência, política explícita de recusa, avaliação reproduzível e documentação das decisões. A técnica específica pode mudar. O princípio permanece: escolher com evidência, medir antes de confiar e registrar aquilo que sustentou cada escolha.

Ciclo completo da disciplina



decisão → dados → treinamento → eficiência → avaliação → protótipo e documentação

Figura — Ciclo completo da disciplina: decidir, preparar dados, treinar, otimizar, avaliar, integrar e documentar.

NA PRÁTICA

- Registre uma decisão técnica no formato escolha, alternativa rejeitada, por quê e resultado real.
- Ao escalar um dataset, repita o mesmo protocolo de avaliação antes de concluir que o aumento de volume melhorou o modelo.
- Liste separadamente o que seu experimento prova e quais perguntas permanecem fora do escopo.

CHECKPOINT DE APRENDIZAGEM

- Por que código funcionando não encerra um projeto profissional?
- O que significa obter um resultado pior depois de escalar o dataset?
- Por que documentar limites e decisões é parte do rigor e não apenas uma tarefa administrativa?

RESUMO DA AULA

- O módulo termina registrando as decisões de arquitetura para que o raciocínio sobreviva a mudanças de equipe, modelo e provedor.
- A escala do dataset é reavaliada com a mesma régua; um resultado pior é tratado como evidência que precisa ser investigada, e não generalizada.
- A disciplina fecha o ciclo entre decisão, dados, treinamento, eficiência, avaliação, integração e documentação, mantendo explícito o escopo das conclusões.

[↑ Voltar ao início da Unidade 6](#)

Revisão da Unidade 6

Use o checklist abaixo para retornar diretamente às aulas que precisam de reforço.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Projeto Final: Modelo Customizado para Domínio Específico \(PT-1\)](#)
- [Revisar a Aula 2: Projeto Final: Modelo Customizado para Domínio Específico \(PT-2\)](#)
- [Revisar a Aula 3: Projeto Final: Modelo Customizado para Domínio Específico \(PT-3\)](#)

[↑ Voltar ao início da Unidade 6](#)

Revisão final da disciplina

A disciplina percorre seis etapas conectadas: decidir quando fine-tuning faz sentido, preparar dados, executar treinamento gerenciado, explorar LoRA/PEFT, avaliar generalização e, por fim, integrar o modelo aprovado em um protótipo documentado e reavaliável.

FLUXO GERAL

Ciclo completo da disciplina



decisão → dados → treinamento → eficiência → avaliação → protótipo e documentação

Figura — Fluxograma das seis unidades presentes no material fornecido.

SÍNTESE PARA REVISÃO

- Fine-tuning é tratado como uma alternativa posterior a técnicas mais baratas, e não como ponto de partida.
- RAG é associado à recuperação de conhecimento; fine-tuning é associado a comportamento, formato e especialização repetida.
- A qualidade do treinamento depende de relevância, representatividade, governança, validação, deduplicação e diversidade do dataset.
- No treinamento gerenciado, configuração, estados do job, hiperparâmetros, automação e linhagem precisam ser observáveis e testáveis.
- LoRA e PEFT reduzem a quantidade de parâmetros treináveis e permitem comparar alternativas locais com Full Fine-Tuning usando métricas de custo, memória e qualidade.
- A decisão final entre técnicas precisa ser sustentada pelas medições do caso, e não por uma regra universal.

PERGUNTAS DE REVISÃO FINAL

- Em que situação o Decision Framework deve interromper uma recomendação de fine-tuning?
- Por que um dataset individualmente válido ainda pode ser inadequado como conjunto?
- Como a comparação entre configuração solicitada e configuração aplicada reduz risco operacional de uma API de treinamento?
- Por que o hash do dataset e o Model Card são importantes para a linhagem do modelo?
- Como rank e scale alteram a decisão de um treinamento LoRA?
- Em que condições o ganho adicional do Full Fine-Tuning justificaria seu custo de memória e armazenamento?

[↑ Voltar ao mapa da disciplina](#)

Anexo A — GitHub da disciplina

Repositório oficial informado para esta disciplina:

Ao longo das aulas, o bloco “GITHUB E PRÁTICA” aponta para o projeto, ferramenta ou material complementar mais diretamente relacionado ao exemplo estudado; quando o repositório não expõe um arquivo único para a etapa, o link aponta para a pasta oficial do módulo.

<https://github.com/unipds-engenharia-de-ia-aplicada/engenharia-de-software-com-ia-aplicada/tree/main/modulo09-processamento-de-dados-e-fine-tuning-de-modelos>



Links diretos por módulo

Módulo 1 — Decision Framework: [abrir no GitHub](#)

Módulo 2 — Preparação de Datasets: [abrir no GitHub](#)

Módulo 3 — Fine-Tuning via API: [abrir no GitHub](#)

Módulo 4 — LoRA e PEFT: [abrir no GitHub](#)

Módulo 5 — Avaliação de Modelos: [abrir no GitHub](#)

Módulo 6 — Projeto Final: [abrir no GitHub](#)